

DevOps Lab Report

Anshuman Mohapatra (500119016)

- 1 Containerization-and-Devops-Lab
 - 1.1 LAB
 - 1.2 THEORY
 - 1.3 ASSIGNMENT

1 Containerization-and-Devops-Lab

Name - Anshuman Mohapatra

Sap ID - 500119016

Batch - 3

1.1 LAB

#	Experiment	Link
0	Install WSL	View
1	Vagrant & NGINX Setup	View
2	Docker Container Setup	View
3	Deploying NGINX Using Different Base Images & Layer Comparison	View
4	Docker Essentials	View
5	Docker Volumes, Environment Variables, Monitoring and Networks	View
6	Docker Run vs Docker Compose	View
7	CI/CD Pipeline using Jenkins, GitHub and Docker Hub	View
9	Ansible Configuration Management	View
10	SonarQube — Static Code Analysis	View
11	Orchestration using Docker Compose & Docker Swarm	View
12	Container Orchestration using Kubernetes	View

1.2 THEORY

- 1. [Lecture -1 : Data Management - Docker Volume](#)
- 2. [Lecture -2 : Docker Networking Concepts](#)
- 3. [Lecture -3 : Python Flask Dockerization](#)
- 4. [Lecture -4 : Node.js Docker Application](#)
- 5. [Lecture -5 : Networking Concepts](#)
- 6. [Lecture -6 : Docker Swarm](#)
- 7. [Lecture -7 : Redis](#)

1.3 ASSIGNMENT

Assignment - 1 : <https://anshuman9016.github.io/Assignment-1/>

Repository URL: <https://github.com/Anshuman9016/Containerization-and-Devops-Lab>

Hosted URL: <https://anshuman9016.github.io/Containerization-and-Devops-Lab>

2 Containerization-and-Devops-Lab

3 EXPERIMENT - 1

3.1 Comparison of Virtual Machines (VMs) and Containers using Ubuntu and Nginx

3.2 Aim

To study, implement, and compare Virtual Machines (VMs) and Containers by deploying an Ubuntu-based Nginx web server in both environments and analyzing their resource utilization, performance, and operational characteristics.

3.3 Objectives

- To understand the conceptual difference between Virtual Machines and Containers
 - To install and configure a Virtual Machine using VirtualBox and Vagrant
 - To install and configure Containers using Docker inside WSL
 - To deploy Nginx web server on both VM and Container
 - To compare performance, startup time, and resource usage of VMs and Containers
-

3.4 System Requirements

3.4.1 Hardware Requirements

- 64-bit system with virtualization enabled in BIOS
- Minimum 8 GB RAM (4 GB acceptable)
- Stable internet connection

3.4.2 Software Requirements (Windows Host)

- Oracle VirtualBox
- Vagrant

- Windows Subsystem for Linux (WSL 2)
 - Ubuntu (WSL distribution)
 - Docker Engine (Docker CLI)
-

4 Theory

4.1 Virtual Machine (VM)

A Virtual Machine emulates a complete physical computer, including its own operating system, kernel, libraries, and hardware drivers. Each VM runs on top of a hypervisor such as VirtualBox.

4.1.1 Characteristics:

- Full OS per VM
 - Strong isolation
 - High resource usage
 - Slower startup time
-

4.2 Container

Containers virtualize at the operating system level. They share the host OS kernel while isolating applications and dependencies in user space.

4.2.1 Characteristics:

- Shared kernel
 - Lightweight
 - Fast startup
 - Efficient resource utilization
-

5 Experiment Setup

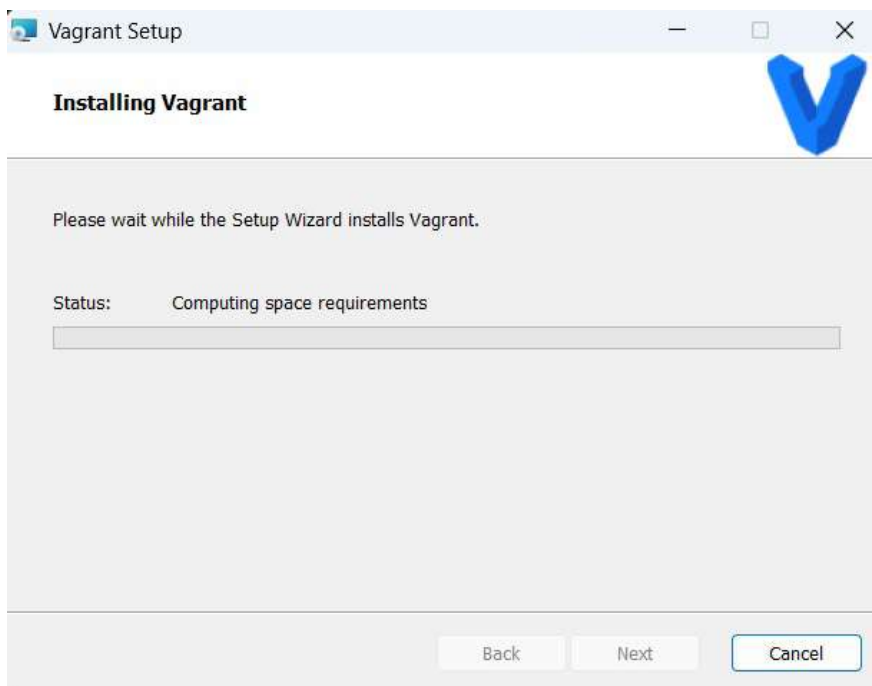
6 PART A: Virtual Machine Setup (Using VirtualBox & Vagrant)

6.1 Step 1: Install VirtualBox

Download from:

<https://www.virtualbox.org>

Install using default settings and restart if prompted.



```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> vagrant --version
Vagrant 2.4.9
PS C:\WINDOWS\system32>
```

6.3 Step 3: Create Ubuntu VM using Vagrant

Create new directory:

```
mkdir vm-lab
cd vm-lab
```

```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> vagrant --version
Vagrant 2.4.9
PS C:\WINDOWS\system32> mkdir vm-lab

Directory: C:\WINDOWS\system32

Mode                LastWriteTime         Length Name
----                -
d-----          2/16/2025   5:50 PM             vm-lab

PS C:\WINDOWS\system32> cd vm-lab
PS C:\WINDOWS\system32\vm-lab>
```

Initialize Ubuntu box:

vagrant init ubuntu/jammy64

```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> vagrant --version
Vagrant 2.4.9
PS C:\WINDOWS\system32> mkdir vm-lab

Directory: C:\WINDOWS\system32

Mode                LastWriteTime         Length Name
----                -
d-----          2/16/2026   5:50 PM             vm-lab

PS C:\WINDOWS\system32> cd vm-lab
PS C:\WINDOWS\system32\vm-lab> vagrant init ubuntu/jammy64
A 'Vagrantfile' has been placed in this directory. You are now
ready to 'vagrant up' your first virtual environment! Please read
the comments in the Vagrantfile as well as documentation on
'vagrantup.com' for more information on using Vagrant.
PS C:\WINDOWS\system32\vm-lab>
```

Start Virtual Machine:

vagrant up

```
Administrator: Windows PowerShell
PS C:\WINDOWS\system32\vm-lab> vagrant up
Bringing machine 'default' up with 'virtualbox' provider...
==> default: Box 'ubuntu/jammy64' could not be found. Attempting to find and install...
    default: Box Provider: virtualbox
    default: Box Version: >= 0
--> default: Loading metadata for box 'ubuntu/jammy64'
    default: URL: https://vagrantcloud.com/api/v2/vagrant/ubuntu/jammy64
==> default: Adding box 'ubuntu/jammy64' (v20241002.0.0) for provider: virtualbox
    default: Downloading: https://vagrantcloud.com/ubuntu/boxes/jammy64/versions/20241002.0.0/providers/virtualbox/unknown/vagrant.box
    default:
==> default: Successfully added box 'ubuntu/jammy64' (v20241002.0.0) for 'virtualbox'!
==> default: Importing base box 'ubuntu/jammy64'...
==> default: Matching MAC address for NAT networking...
==> default: Checking if box 'ubuntu/jammy64' version '20241002.0.0' is up to date...
--> default: Setting the name of the VM: vm-lab default 1771244808070_26027
Vagrant is currently configured to create VirtualBox synced folders with
the 'SharedFoldersEnableSymlinksCreate' option enabled. If the Vagrant
guest is not trusted, you may want to disable this option. For more
information on this option, please refer to the VirtualBox manual:

    https://www.virtualbox.org/manual/ch04.html#sharedfolders

This option can be disabled globally with an environment variable:

    VAGRANT_DISABLE_VBOXSYMLINKCREATE=1

or on a per folder basis within the Vagrantfile:

    config.vm.synced_folder '/host/path', '/guest/path', SharedFoldersEnableSymlinksCreate: false
```

Access VM:

vagrant ssh

```
PS C:\WINDOWS\system32\vm-lab> vagrant ssh
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-134-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/support

System information as of Thu Feb 16 12:56:04 UTC 2026

System load:  0.27
Usage of /:   3.9% of 36.76GB
Memory usage: 20%
Swap usage:   0%
Processes:    113
Users logged in: 0
IP address for ethp0: 10.0.2.15
IP address for ethp0: 5.00 (10.43.11; feed:55at)

Disabling Security Maintenance for Applications is not enabled.
A update can be applied immediately.
2 of these updates is a standard security update.
To see these additional updates run: apt list --upgradable
Enable (SN) Apps to receive additional future security updates.
See https://ubuntu.com/son or run: sudo apt update

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
New release '24.04.4 (Noble)' available.
Run 'do-release-upgrade' to upgrade to it.

vagrant@ubuntu-jammy:~$
```

6.4 Step 4: Install Nginx inside VM

[illegible]

```
curl localhost
```

```

sage@rhel6centos:~$ sudo systemctl start nginx
sage@rhel6centos:~$ sudo systemctl status nginx
nginx.service - nginx web server
Loaded: loaded (/etc/systemd/system/nginx.service; vendor preset: enabled)
Active: active (running) since Mon 2016-08-01 14:00:00 EDT; 1min 1s ago
Main PID: 1000
CGroup: /systemd/system/nginx.service
        └─ nginx: /usr/sbin/nginx: -c /etc/nginx/nginx.conf

sage@rhel6centos:~$ cat /etc/nginx/nginx.conf
user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log;
pid /var/run/nginx.pid;

events {
    worker_connections 1024;
}

http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    # Log on to the stderr
    access_log /var/log/nginx/access.log stderr;

    # Load dynamic configuration if you have the shared objects (modules)
    # installed on your system.
    # #include /usr/share/nginx/modules/*.conf;

    include /etc/nginx/conf.d/*.conf;
    include /etc/nginx/sites-enabled/*;
}

```

```
vagrant halt
vagrant destroy
```

[illegible]

7.1 Step 1: Install WSL

7.1 Step 1: Install WSL

```
wsl --install
```

Reboot system.

7.2 Step 2: Install Ubuntu on WSL

```
wsl --install -d Ubuntu
```

7.3 Step 3: Install Docker Engine inside WSL

```
sudo apt update
```

```
sudo apt install -y docker.io
```

```
sudo systemctl start docker
```

```
sudo usermod -aG docker $USER
```

```

$ sudo apt-get install -y docker.io
(only) package for amd64
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
bridge-utils containerd cni-plugin docker-buildx docker-compose-docker-plugin docker-ce-cli docker-ce-rootless-extras
Suggested packages:
iproute2 iptables kubernetes-cni kubelet kubeadm kubectl kubernetes-kubelet-plugin kube-proxy kubelet-plugin kubemark kubelet-plugin-sriov-networking
The following NEW packages will be installed:
bridge-utils containerd cni-plugin docker-buildx docker-compose-docker-plugin docker-ce-cli docker-ce-rootless-extras
0 upgraded, 8 newly installed, 0 to remove and 2 not upgraded.
Need to get 81.1 MB of archives.
After this operation, 268 MB of additional disk space will be used.
Do you want to continue? [Y/n]
Selecting previously unselected package bridge-utils.
(Reading database ... 10731 files and directories currently installed.)
Preparing to unpack .../0-bridge-utils_1.7.1-1ubuntu0.20.04.1_amd64.deb ...
Unpacking bridge-utils (1.7.1-1ubuntu0.20.04.1) ...
Selecting previously unselected package containerd.
Preparing to unpack .../1-containerd_1.2.8-ubuntu~20.04.2_amd64.deb ...
Unpacking containerd (1.2.8-ubuntu~20.04.2) ...
Selecting previously unselected package cni-plugin.
Preparing to unpack .../2-cni-plugin_0.3.1-3ubuntu~20.04.2_amd64.deb ...
Unpacking cni-plugin (0.3.1-3ubuntu~20.04.2) ...
Selecting previously unselected package docker-buildx.
Preparing to unpack .../3-docker-buildx_0.11.1-1ubuntu~20.04.2_amd64.deb ...
Unpacking docker-buildx (0.11.1-1ubuntu~20.04.2) ...
Selecting previously unselected package docker-compose-docker-plugin.
Preparing to unpack .../4-docker-compose-docker-plugin_1.29.1-1ubuntu~20.04.2_amd64.deb ...
Unpacking docker-compose-docker-plugin (1.29.1-1ubuntu~20.04.2) ...
Selecting previously unselected package docker-ce-cli.
Preparing to unpack .../5-docker-ce-cli_20.10.14~3-0ubuntu~20.04.2_amd64.deb ...
Unpacking docker-ce-cli (20.10.14~3-0ubuntu~20.04.2) ...
Selecting previously unselected package docker-ce-rootless-extras.
Preparing to unpack .../6-docker-ce-rootless-extras_20.10.14~3-0ubuntu~20.04.2_amd64.deb ...
Unpacking docker-ce-rootless-extras (20.10.14~3-0ubuntu~20.04.2) ...
Setting up bridge-utils (1.7.1-1ubuntu0.20.04.1) ...
Setting up containerd (1.2.8-ubuntu~20.04.2) ...
Setting up cni-plugin (0.3.1-3ubuntu~20.04.2) ...
Setting up docker-buildx (0.11.1-1ubuntu~20.04.2) ...
Setting up docker-compose-docker-plugin (1.29.1-1ubuntu~20.04.2) ...
Setting up docker-ce-cli (20.10.14~3-0ubuntu~20.04.2) ...
Setting up docker-ce-rootless-extras (20.10.14~3-0ubuntu~20.04.2) ...

```

Logout and login again to apply group changes.

7.4 Step 4: Run Nginx Container

Pull Nginx image:

```
docker pull nginx
```

Run container:

```
docker run -d -p 8080:80 --name nginx-container nginx
```

[illegible]

Check running containers:

docker ps

7.5 Step 5: Verify Nginx Container

curl localhost:8080

```
root@kali:~# curl localhost:8080
(DEVTYPE html)
html
head:
[[[Welcome to nginx!]]]
title
html { color:blue; font-size: 14px; }
body { color:blue; font-weight: bold; }
font-family: Ubuntu, Verdana, Arial, sans-serif; }
/HTML
/body
/body
[[[Welcome to nginx!]]]
[[[If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.]]]
[[[For online documentation and support please refer to
http://nginx.org/en/doc/
Commercial support is available at
http://nginx.com]]]
[[[Thank you for using nginx.]]]
/body
/body
root@kali:~#
```

If Nginx default page appears, container deployment is successful.

8 Resource Utilization Observation

8.1 VM Observation Commands

```
free -h
htop
systemd-analyze
```

8.2 Container Observation Commands

```
docker stats
free -h
```

9 Common Troubleshooting

9.1 VirtualBox Issues

- VT-x / AMD-V not enabled → Enable virtualization in BIOS
- Hyper-V conflict → Disable Hyper-V in Windows

9.2 Vagrant Issues

- Box download failure → Check firewall
- SSH timeout → Run `vagrant reload`

9.3 Docker Issues (WSL)

- Permission denied → Ensure user added to docker group
- Docker service not running:

```
sudo systemctl start docker
```

10 Result

The experiment successfully demonstrated that containers are lightweight, faster, and more resource-efficient than virtual machines, while virtual machines provide stronger isolation and full OS abstraction.

11 Conclusion

Containers are best suited for modern cloud-native and DevOps workflows due to their speed and efficiency, whereas Virtual Machines are preferred when full isolation and OS-level control are required.

12 Viva-Voice Questions

1. What is the main difference between VM and container?
 2. Why do containers start faster than VMs?
 3. What role does a hypervisor play?
 4. Can containers run different OS kernels?
 5. Why is Docker considered lightweight?
-

13 References

- VirtualBox Documentation
- Vagrant Documentation
- Docker Official Documentation

14 Containerization-and-Devops-Lab

15 EXPERIMENT - 2

15.1 Docker Installation, Configuration, and Running Images

15.2 Aim

To install and configure Docker, pull Docker images, run containers, and manage the container lifecycle using Docker commands.

15.3 Objectives

- To pull Docker images from Docker Hub
 - To run containers with port mapping
 - To verify running containers
 - To manage container lifecycle (start, stop, remove)
-

16 Theory

Docker is an open-source containerization platform that allows applications to be packaged along with their dependencies into lightweight, portable containers. Containers run on a shared operating system kernel, making them faster and more resource-efficient than traditional virtual machines.

- A **Docker Image** is a read-only template used to create containers.
- A **Docker Container** is a running instance of a Docker image.

Docker follows a **client-server architecture**, where:

- The Docker Client sends commands
 - The Docker Daemon builds, runs, and manages containers
-

17 Software Requirements

- Windows OS
 - Docker Desktop with WSL integration
 - Ubuntu (WSL distribution)
-

18 Procedure / Steps to Perform the Experiment

18.1 Step 1: Pull Docker Image

Pull the Nginx image from Docker Hub:

```
docker pull nginx
```



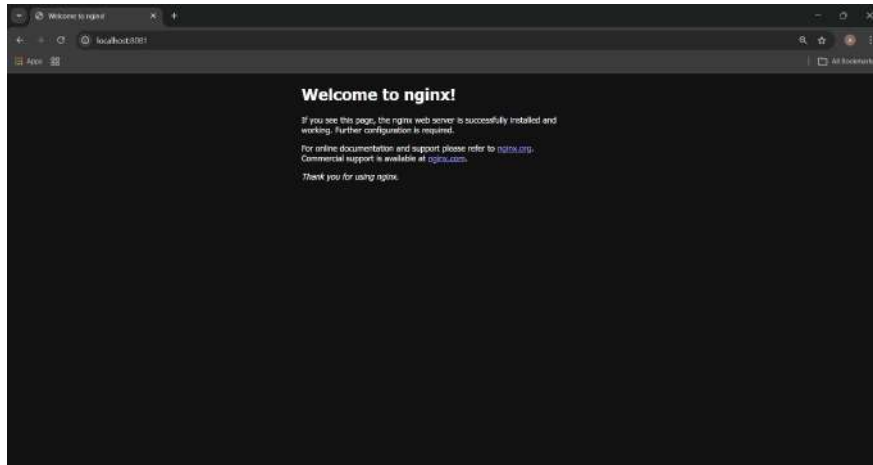
```
root@kali:~/Documents# docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
Digest: sha256:343d99f308c527b996d1f4a252a6d0a95a8a2d208f6d11208
Status: Image is up to date for nginx:latest
docker.io/library/nginx:latest
root@kali:~/Documents#
```

This command downloads the latest official Nginx image to the local system.

18.2 Step 2: Run Container with Port Mapping

Run the Nginx container in detached mode with port mapping:

```
docker run -d -p 8080:80 nginx
```



18.2.1 Explanation:

- -d → Runs container in background
 - -p 8080:80 → Maps host port 8080 to container port 80
 - nginx → Docker image name
-

18.3 Step 3: Verify Running Containers

Check running containers:

```
docker ps
```



This displays:

- Container ID
 - Image name
 - Status
 - Port mapping
-

18.4 Step 4: Stop and Remove Container

To stop the running container:

```
docker stop <container_id>
```

To remove the container:

```
docker rm <container_id>
```

18.5 Step 5: Remove Docker Image

To remove the downloaded image:

```
docker rmi nginx
```

This frees disk space by deleting the unused image.

19 Result

Docker images were successfully pulled, containers were executed, and container lifecycle management commands were performed successfully.

20 Conclusion

This experiment demonstrated the use of Docker for application deployment using containers. Docker provides a lightweight, efficient, and portable environment for running applications, making it suitable for modern DevOps and cloud-native applications.

21 Viva-Voce Questions

1. What is a Docker image?
 2. What is a Docker container?
 3. Difference between `docker run` and `docker start`?
 4. What is the purpose of port mapping in Docker?
 5. Why are containers lightweight compared to VMs?
-

22 Overall Conclusion of Lab

This lab demonstrated virtualization using Vagrant + VirtualBox and containerization using Docker, highlighting clear performance and resource efficiency differences. Containers are better suited for rapid deployment and microservices, while VMs provide stronger isolation.

23 Containerization-and-Devops-Lab

24 EXPERIMENT - 3

24.1 Deploying NGINX Web Server Using Official, Ubuntu, and Alpine Base Images and Comparing Image Layers

24.2 Aim

To deploy the NGINX web server using different Docker base images and compare their image size, performance, layers, and security impact.

24.3 Objectives

After completing this experiment, students will be able to:

- Deploy NGINX using Docker containers
 - Build custom images using Dockerfile
 - Understand Docker image layers
 - Compare Ubuntu, Alpine, and Official images
 - Analyze size, speed, and security differences
 - Explain real-world uses of NGINX in containers
-

24.4 Prerequisites

- Docker installed and running
 - Basic Linux commands
 - Knowledge of:
 - `docker pull`
 - `docker run`
 - `docker build`
 - Dockerfile
 - Port mapping
-

25 Theory

25.1 What is NGINX?

NGINX is a high-performance:

- Web Server
- Reverse Proxy
- Load Balancer
- API Gateway

It uses an event-driven asynchronous architecture, making it faster and more scalable than traditional servers like Apache.

25.2 What is Docker?

Docker is a containerization platform that:

- Packages applications with dependencies
 - Ensures portability
 - Provides lightweight virtualization
 - Speeds up deployment
-

25.3 What are Docker Image Layers?

Docker images are built in layers.

Each instruction in a Dockerfile creates a layer:

- FROM
- RUN
- COPY
- ADD

25.3.1 Importance of Layers

- More layers → Bigger image
 - Bigger images → Slower pull time
 - Larger images → More vulnerabilities
 - Fewer layers → Faster and more secure
-

25.4 Base Image Types

Base Image	Description
Official nginx	Pre-built optimized production image
Ubuntu	Full Linux OS with tools
Alpine	Lightweight minimal Linux

26 Experiment Procedure

27 Part 1 — Deploy NGINX Using Official Image

27.0.1 Step 1: Pull Image

```
docker pull nginx:latest
```

FROM ubuntu:22.04


```
CMD ["nginx", "-g", "daemon off;"]
```

```
docker build -t nginx-ubuntu .
```

```
docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu
```

docker images nginx-ubuntu

- Large image (~220MB+)
- Many layers
- Includes full OS
- Slower startup
- Larger attack surface

```
CMD ["nginx", "-q", "daemon off;"]
```

29.0.2 Step 2: Build Image

```
docker build -t nginx-alpine .
```

```

C:\docker-nginx>dir

Directory: C:\docker-nginx

Name                Length
----                -
3/15/2025, 7:41 PM  101 Dockerfile

C:\docker-nginx>docker build -t nginx-nginx .

[+] Building 2.9s (7/0) FINISH
   # [internal] Load build definition from Dockerfile                                Docker Desktop Linux 0.0s
   # => transferring dockerfile: 1.00k                                              0.0s
   # [internal] Load metadata for docker.io/library/nginx:latest                  0.0s
   # => http://library/nginx:latest taken for registry-3.docker.io                  0.0s
   # [internal] Load metadata for docker.io/library/nginx:latest                  0.0s
   # => transferring context: 28                                                     0.0s
   # [2/2] FROM docker.io/library/nginx:latest@sha256:2180164c72b6d73e8318a812724949e3a0710d19770a3c80110c35f90e 1.3s
   # => FROM docker.io/library/nginx:latest@sha256:2180164c72b6d73e8318a812724949e3a0710d19770a3c80110c35f90e 0.0s
   # => sha256:5900000eb0ed21a10411b0d477e0c35c5c8e0000000000000000000000000000000000000000000000000000000000000000 1.0s
   # => sha256:5900000eb0ed21a10411b0d477e0c35c5c8e000000000000000000000000000000000000000000000000000000000000000 1.0s
   # [2/2] RUN apt-get --no-cache install -y nginx                                  1.2s
   # => exporting to image                                                            0.0s
   # => exporting layers                                                              0.2s
   # => exporting manifest sha256:0f3010a1c02040401344150000000000000000000000000000000000000000000000000000000000000 0.0s
   # => exporting manifest sha256:0f3010a1c02040401344150000000000000000000000000000000000000000000000000000000000000 0.0s
   # => exporting manifest sha256:0f3010a1c02040401344150000000000000000000000000000000000000000000000000000000000000 0.0s
   # => exporting manifest sha256:0f3010a1c02040401344150000000000000000000000000000000000000000000000000000000000000 0.0s
   # => exporting manifest sha256:0f3010a1c02040401344150000000000000000000000000000000000000000000000000000000000000 0.0s
   # => exporting to docker.io/library/nginx:latest                                0.0s
   # CANCELED

```

29.0.3 Step 3: Run Container

```
docker run -d --name nginx-alpine -p 8082:80 nginx-alpine
```

[illegible]

29.0.4 Observations

```
docker images nginx-alpine
```

```
PS C:\cluster-nglino> docker images nginx-alpine
```

IMAGE	ID	DATE	SIZE	EXTRA
nginx-alpine:latest	0962c1010105	10h	4.74MB	

- Very small (~25-30MB)
- Minimal dependencies
- Faster pull time
- Faster startup
- More secure

30 Part 4 — Compare Image Sizes

```
docker images | grep nginx
```

[illegible]

30.0.1 Sample Output

Image Type	Size
nginx:latest	~140MB
nginx-ubuntu	~220MB+
nginx-alpine	~25MB

31 Part 5 — Inspect Image Layers

```
docker history nainx
```

```
docker history nginx-ubuntu
```

```
docker history nginx-alpine
```

[illegible]

31.0.1 Observations

- Ubuntu → Many filesystem layers
- Alpine → Minimal layers
- Official → Optimized layers

32 Part 6 — Serve Custom HTML Page

32.0.1 Step 1: Create HTML

```
mkdir html
```

```
echo "<h1>Hello from Docker NGINX</h1>" > html/index.html
```

```

C:\Users\aglin>ls /s
Directory: C:\Users\aglin

Mode                LastWriteTime         Length Name
----                -
d-----          1/30/2025   7:36 PM             Mail

```

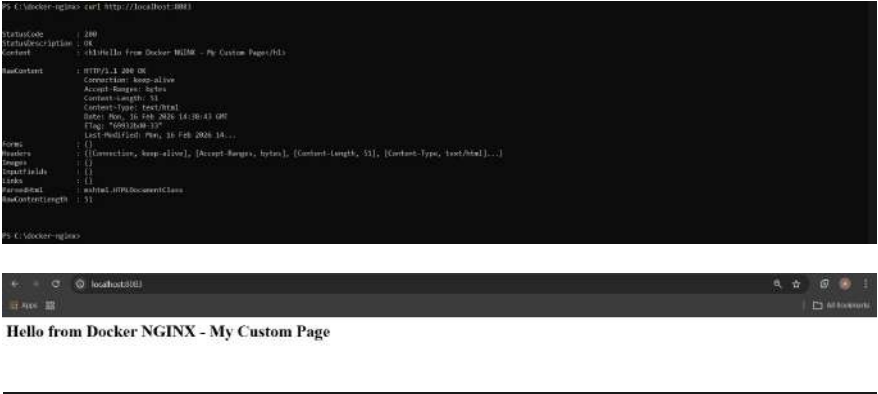
32.0.2 Step 2: Run

```
docker run -d \
  -p 8083:80 \
  -v $(pwd)/html:/usr/share/nginx/html \
  nginx
```

32.0.3 Step 3: Verify

Open:

http://localhost:8083



33 Part 7 — Real World Uses of NGINX

NGINX is commonly used for:

- Static websites
- Reverse proxy
- Load balancing
- SSL termination
- API gateway
- Kubernetes ingress controller
- Microservices frontend

34 Comparison Summary

Feature	Official	Ubuntu	Alpine
Size	Medium	Large	Very Small
Startup	Fast	Slow	Very Fast
Security	Medium	Low	High
Debugging	Limited	Good	Minimal
Production	Yes	Rare	Yes

35 When to Use What

35.0.1 Official Image

- Production deployments
- Reverse proxy
- Standard hosting

35.0.2 Ubuntu Image

- Learning purposes
- Debugging tools required
- Heavy dependencies

35.0.3 Alpine Image

- Cloud environments
 - Microservices
 - CI/CD
 - Kubernetes
-

36 Assignment Tasks

- Measure image pull time
- Add custom NGINX configuration
- Change default port
- Enable basic authentication
- Reduce layers and rebuild

Explain:

- Why Alpine is smaller
 - Why Ubuntu is not preferred in production
-

37 Viva Questions

1. What is NGINX?
 2. What is Docker?
 3. What are Docker layers?
 4. Why are Alpine images smaller?
 5. Difference between Ubuntu and Alpine?
 6. Why use official images in production?
 7. What is reverse proxy?
 8. How does NGINX improve performance?
-

38 Learning Outcomes

After completing this experiment, students can:

- Deploy web servers in containers
 - Build custom Docker images
 - Optimize image size
 - Understand layer structure
 - Improve security practices
 - Use NGINX in production systems
-

39 Conclusion

This experiment demonstrates that:

- Official images are stable and production ready
- Ubuntu images are large and slower
- Alpine images are lightweight, faster, and secure

Therefore, Alpine or Official images are preferred for real-world deployments.

40 Containerization-and-Devops-Lab

40.1 EXPERIMENT - 4

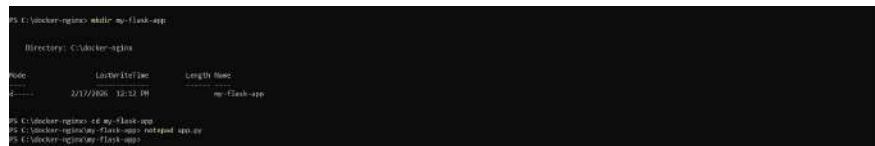
40.2 Docker Essentials

41 Part 1: Containerizing Applications with Dockerfile

41.1 Step 1: Create a Simple Application

41.1.1 Python Flask App

```
mkdir my-flask-app  
cd my-flask-app
```

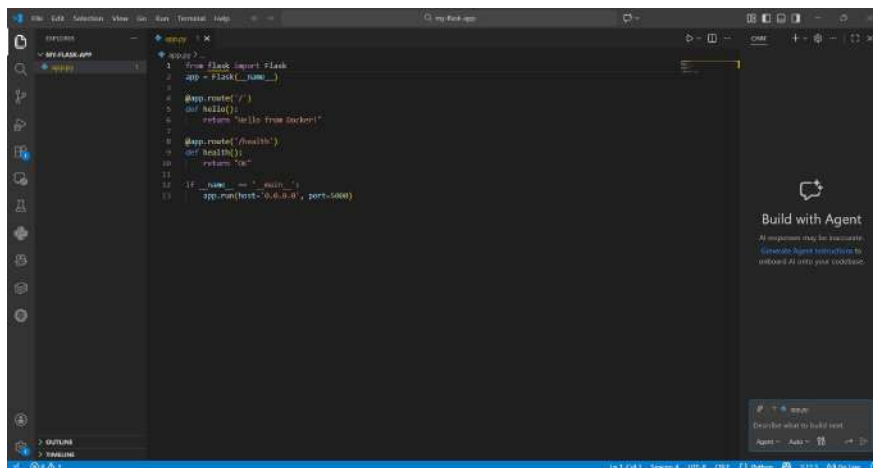


A terminal window showing the creation of a directory 'my-flask-app' and a subsequent 'ls' command. The output shows a directory listing for 'my-flask-app' with columns for mode, last modification time, and file name.

```
PS C:\docker-nginx> mkdir my-flask-app  
Directory: C:\docker-nginx  
Mode                LastWriteTime         Length Name  
----                -  
d-----            2/17/2026   12:12 PM             my-flask-app  
PS C:\docker-nginx> cd my-flask-app  
PS C:\docker-nginx> ls  
PS C:\docker-nginx> cd my-flask-app  
PS C:\docker-nginx> cd my-flask-app  
PS C:\docker-nginx> cd my-flask-app
```

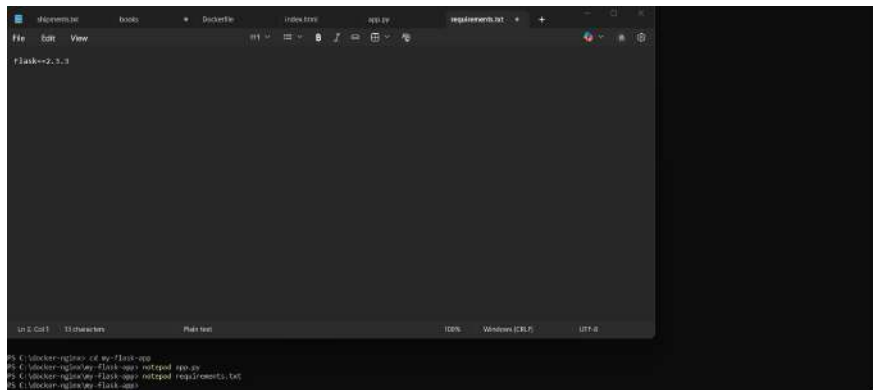
41.1.2 app.py

```
from flask import Flask  
app = Flask(__name__)  
  
@app.route('/')  
def hello():  
    return "Hello from Docker!"  
  
@app.route('/health')  
def health():  
    return "OK"  
  
if __name__ == '__main__':  
    app.run(host='0.0.0.0', port=5000)
```



41.1.3 requirements.txt

Flask==2.3.3



41.2 Step 2: Create Dockerfile

```
# Use Python base image
FROM python:3.9-slim

# Set working directory
WORKDIR /app

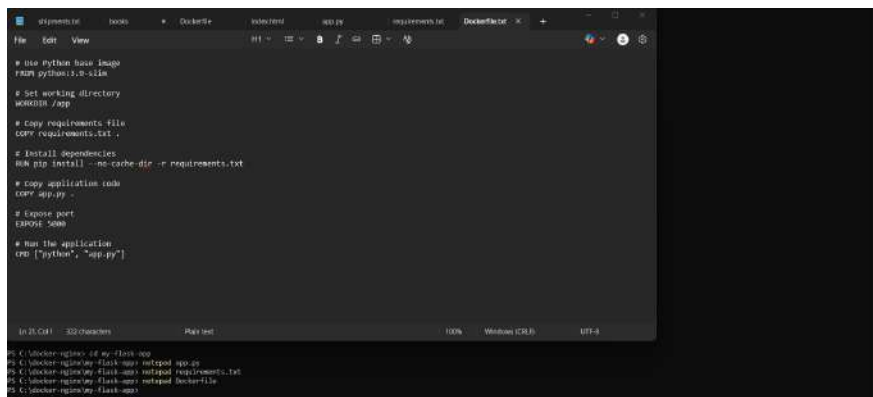
# Copy requirements file
COPY requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY app.py .

# Expose port
EXPOSE 5000

# Run the application
CMD ["python", "app.py"]
```



```
FROM python:3.8-slim
# Set working directory
WORKDIR /app
# Copy requirements file
COPY requirements.txt .
# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt
# Copy application code
COPY app.py .
# Expose port
EXPOSE 5000
# Run the application
CMD ["python", "app.py"]
```

in 2.5s | 332 characters | Plain text | 100% | Windows (CRLF) | UTF-8

1. C:\docker\ignores\if my flask app
2. C:\docker\ignores\if flask app\retaped app.py
3. C:\docker\ignores\if flask app\retaped requirements.txt
4. C:\docker\ignores\if flask app\retaped Dockerfile
5. C:\docker\ignores\if flask app\retaped

42 Part 2: Using .dockerignore

42.1 Step 1: Create .dockerignore File

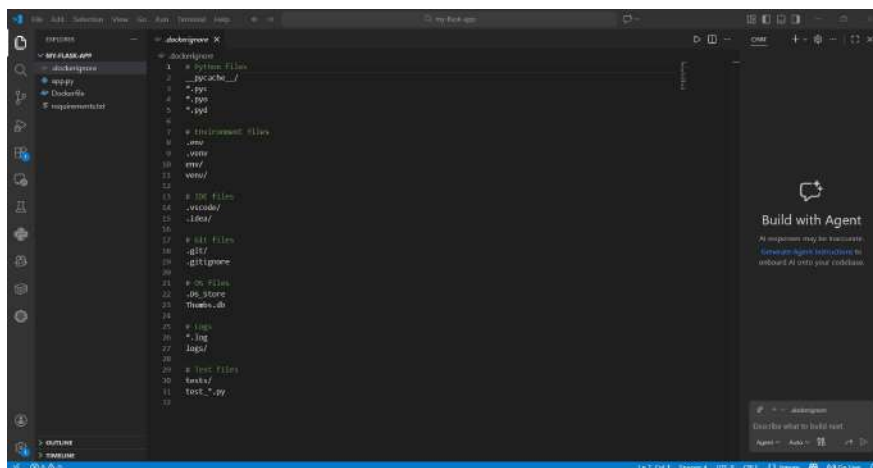
```
# Python cache files
__pycache__/*
*.pyc

# Virtual environments
.env
.venv
env/
venv/

# IDE files
.vscode/
.idea/

# Git files
.git/
.gitignore

# Logs
*.log
logs/
```



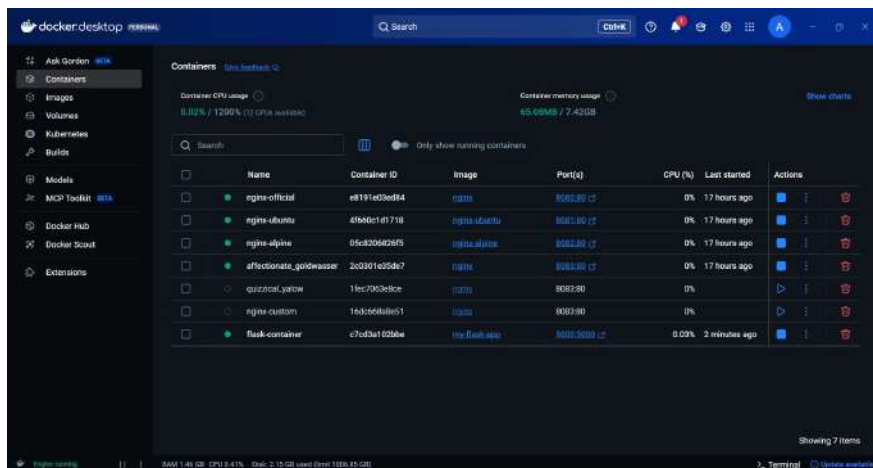
44.1 Step 1: Run Container

```
docker run -d -p 5000:5000 --name flask-container my-flask-app
```

Test application:

localhost:5103

```
docker ps
docker logs flask-container
```



44.2 Step 2: Manage Containers

```
docker stop flask-container
docker start flask-container
docker rm flask-container
docker rm -f flask-container
```

45 Part 5: Multi-stage Builds

45.1 Why Multi-stage Builds?

- Smaller final image size
- Better security
- Separate build and runtime environment

45.2 Example Multi-stage Dockerfile

```
# STAGE 1: Builder
FROM python:3.9-slim AS builder

WORKDIR /app
COPY requirements.txt .
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN pip install --no-cache-dir -r requirements.txt

# STAGE 2: Runtime
FROM python:3.9-slim

WORKDIR /app
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

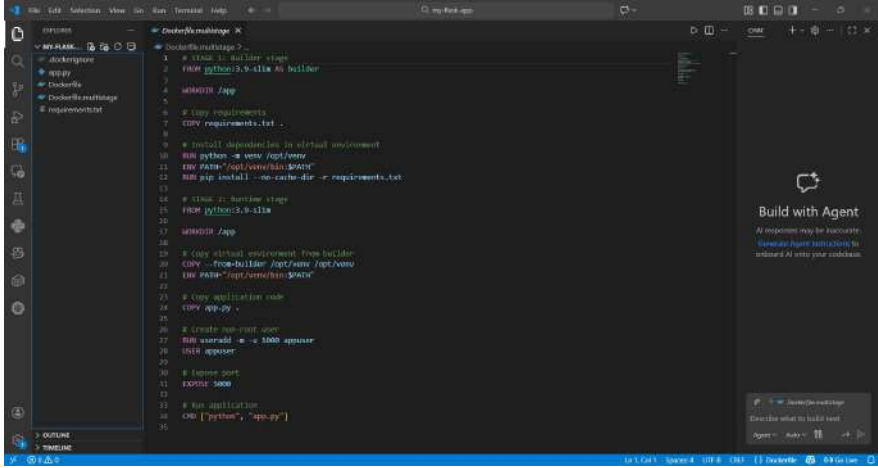
COPY app.py .

RUN useradd -m -u 1000 appuser
```

USER appuser

EXP0SE 5000

```
CMD ["python", "app.py"]
```

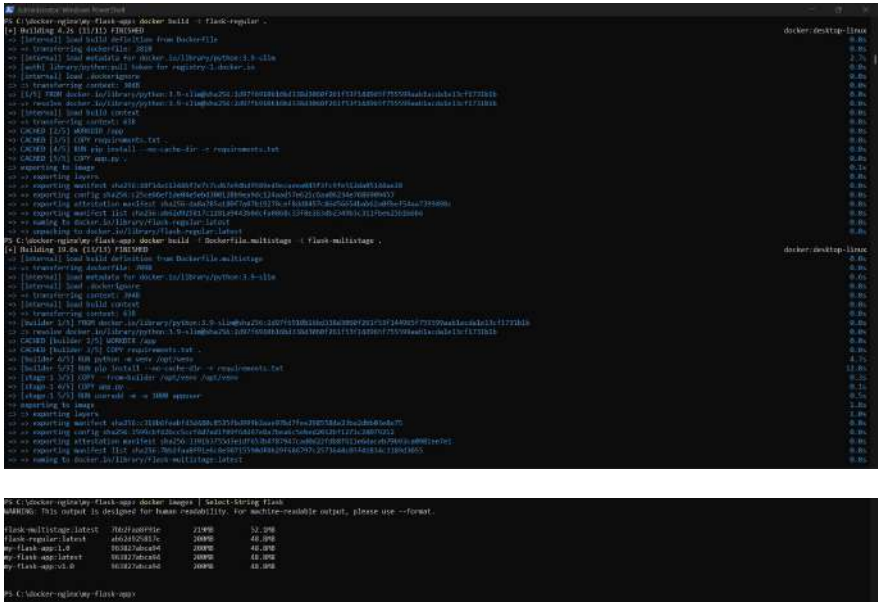


45.3 Build and Compare

```
docker build -t flask-regular .
```

```
docker build -f Dockerfile.multistage -t flask-multistage .
```

```
docker images | grep flask
```



Expected:

- flask-regular → ~250MB
- flask-multistage → ~150MB

46 Part 6: Publishing to Docker

Hub

46.1 Login & Tag

```
docker login
```

```
docker tag my-flask-app:latest username/my-flask-app:1.0
```

```
docker push username/my-flask-app:1.0
```

[illegible][illegible]

46.2 Pull on Another Machine

```
docker pull username/my-flask-app:latest
```

```
docker run -d -p 5000:5000 username/my-flask-app:latest
```

[illegible]

47 Part 7: Node.js Example

47.1 Node.js App (app.js)

```
const express = require('express');
const app = express();
const port = 3000;
```

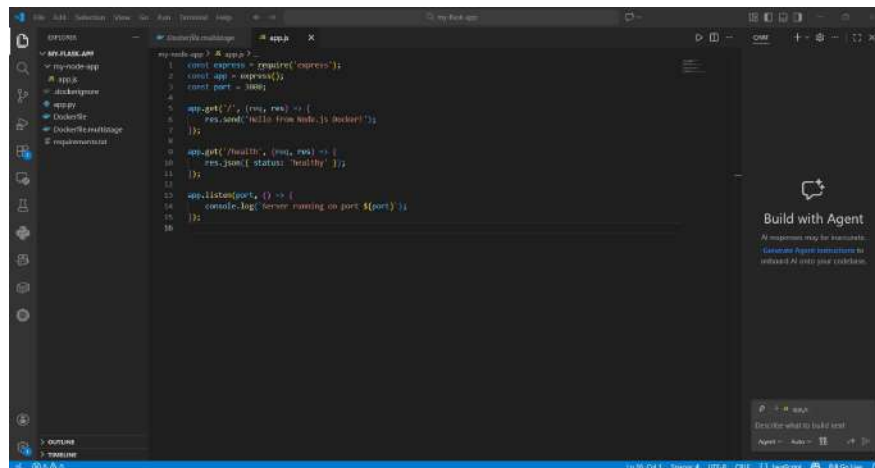
```

app.get('/', (req, res) => {
  res.send('Hello from Node.js Docker!');
});

app.get('/health', (req, res) => {
  res.json({ status: 'healthy' });
});

app.listen(port, () => {
  console.log(`Server running on port ${port}`);
});

```



47.2 Dockerfile

FROM node:18-alpine

WORKDIR /app

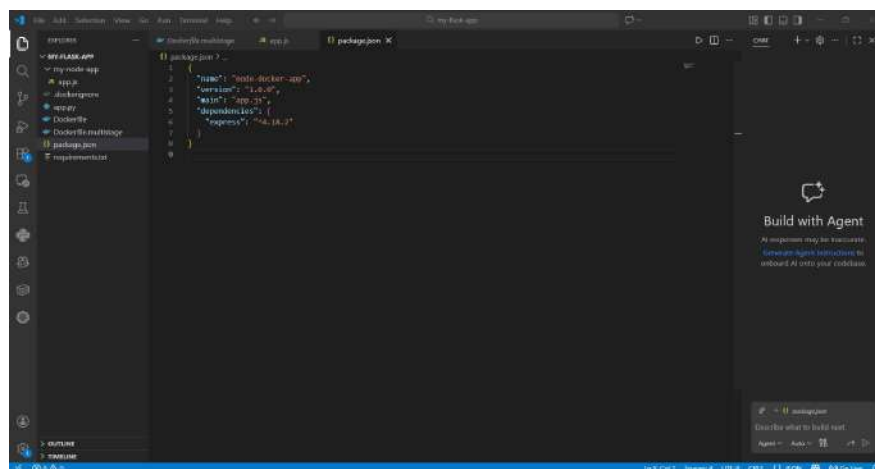
COPY package*.json ./

RUN npm install --only=production

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]



48 Essential Docker Commands Cheatsheet

Command	Purpose	Example
docker build	Build image	docker build -t myapp .
docker run	Run container	docker run -p 3000:3000 myapp
docker ps	List containers	docker ps -a
docker images	List images	docker images
docker tag	Tag image	docker tag myapp:v1 myapp:v2
docker push	Push image	docker push username/myapp
docker pull	Pull image	docker pull username/myapp
docker rm	Remove container	docker rm container-name
docker rmi	Remove image	docker rmi image-name
docker logs	View logs	docker logs container-name
docker exec	Execute command	docker exec -it container-name bash

49 Common Workflow Summary

49.1 Development Workflow

docker build -t myapp .
docker run -p 8080:8080 myapp
docker tag myapp:latest myapp:v1.0
docker push myapp:v1.0

49.2 Production Workflow

docker pull myapp:v1.0
docker run -d -p 80:8080 --name prod-app myapp:v1.0
docker logs -f prod-app

```

$ docker run --rm --name myapp-dev docker build -t myapp:latest -f Dockerfile .
$ docker run -p 8080:8080 myapp
$ docker tag myapp:latest myapp:v1.0
$ docker push myapp:v1.0

$ docker pull myapp:v1.0
$ docker run -d -p 80:8080 --name prod-app myapp:v1.0
$ docker logs -f prod-app
```



```
PS C:\docker-engine\my-flask-app\my-node-app> Measure-Command { docker build -t cached-app . }
```

- Dockerfile defines how to build your image
- .dockerignore improves performance and security
- Tagging helps version management
- Multi-stage builds reduce image size
- Docker Hub enables sharing images
- Always test locally before publishing

[illegible]

53 Experiment 5: Docker - Volumes, Environment Variables, Monitoring & Networks

55 List volumes

docker volume ls

56 Shows: mydata

57 Inspect volume

docker volume inspect mydata

![alt text](/Containerization-and-Devops-Lab/lab/experiment-5/images/2.png)

3. Bind Mounts (Host Directory)

```Bash

# Create directory on host

mkdir ~/myapp-data

# Mount host directory to container

docker run -d -v ~/myapp-data:/app/data --name web3 nginx

# Add file on host

echo "From Host" > ~/myapp-data/host-file.txt

# Check in container

docker exec web3 cat /app/data/host-file.txt

# Shows: From Host

```
PS C:\Users\anshu> mkdir ~/nginx-config

Directory: C:\Users\anshu

Mode LastWriteTime Length Name
---- -
d----- 4/24/2026 8:01 PM nginx-config

PS C:\Users\anshu> echo 'server {
>> listen 80;
>> server_name localhost;
>> location / {
>> return 200 "Hello from mounted config!";
>> }
>> }' > ~/nginx-config/nginx.conf
PS C:\Users\anshu> @
>> server {
>> listen 80;
>> server_name localhost;
>> location / {
>> return 200 "Hello from mounted config!";
>> }
>> }
>> "@ > nginx-config/nginx.conf
```

```

PS C:\Users\anshu> notepad nginx-config\nginx.conf
PS C:\Users\anshu> docker rm -f nginx-custom
nginx-custom
PS C:\Users\anshu> docker run -d --name nginx-custom -p 8080:80 -v $(PWD)/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf nginx
a7b512a971a8b4e8994dc4b46dbcb4b970b2a519dc6ca51eb0779a85
PS C:\Users\anshu> docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
a7b512a971a8 nginx "/docker-entrypoint. 4 seconds ago Up 4 seconds 0.0.0.0:8080->80/tcp nginx-custom
e7269e24f978e nginx "/docker-entrypoint. 33 minutes ago Up 33 minutes 80/tcp web2
6ad79e54b635 assignment-1-backend "/docker-entrypoint.s 4 weeks ago Up 39 minutes 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp backend_app
281acc59a80 assignment-1-database "/docker-entrypoint.s 4 weeks ago Up 39 minutes postgres_db

Security Warning: Script Execution Risk
Invoke-MobRequest parses the content of the web page. Script code in the
web page might be run when the page is parsed.
Recommended ACTION:
Use the -UseBasicParsing switch to avoid script code execution.
Do you want to continue?
[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help
(default is "N")y

StatusCode : 200
StatusDescription : OK
Content : {2, 181, 188, 188...}
RawContent : HTTP/1.1 200 OK
Connection : keep-alive
Content-Length : 26
Content-Type : application/octet-stream
Date : Fri, 24 Apr 2026 14:43:55 GMT
Server : nginx/1.20.3

Headers
: [[Connection, keep-alive], [Content-Length, 26],
[Content-Type, application/octet-stream], [Date, Fri,
24 Apr 2026 14:43:55 GMT]...]
RawContentLength : 26

```

## 57.0.1 Lab 3: Practical Volume Examples

### Example 1: Database with Persistent Storage

```

MySQL with named volume
docker run -d \
 --name mysql-db \
 -v mysql-data:/var/lib/mysql \
 -e MYSQL_ROOT_PASSWORD=secret \
 mysql:8.0

```

```

Check data persists
docker stop mysql-db
docker rm mysql-db

```

```

New container with same volume
docker run -d \
 --name new-mysql \
 -v mysql-data:/var/lib/mysql \
 -e MYSQL_ROOT_PASSWORD=secret \
 mysql:8.0

```

# Data is preserved!

### Example 2: Web App with Configuration Files

```

Create config directory
mkdir ~/nginx-config

Create nginx config file
echo 'server {
 listen 80;
 server_name localhost;
 location / {
 return 200 "Hello from mounted config!";
 }
}' > ~/nginx-config/nginx.conf

```

```

Run nginx with config bind mount
docker run -d \
 --name nginx-custom \
 -p 8080:80 \
 -v ~/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf \
 nginx

```

# Test

```
curl http://localhost:8080
```

```
PS C:\Users\anshu> docker volume ls
DRIVER VOLUME NAME
local assignment-1_postgres_data
local mydata
PS C:\Users\anshu> docker volume create app-volume
app-volume
PS C:\Users\anshu> docker volume inspect app-volume
[
 {
 "CreatedAt": "2026-04-24T14:48:03Z",
 "Driver": "local",
 "Labels": null,
 "Mountpoint": "/var/lib/docker/volumes/app-volume/_data",
 "Name": "app-volume",
 "Options": null,
 "Scope": "local"
 }
]
PS C:\Users\anshu> docker volume prune
WARNING! This will remove anonymous local volumes not used by at least one c
ontainer.
Are you sure you want to continue? [y/N] y
Total reclaimed space: 0B

PS C:\Users\anshu> docker volume rm volume-name
Error response from daemon: get volume-name: no such volume
PS C:\Users\anshu> docker volume rm app-volume
app-volume
PS C:\Users\anshu> echo "hello" > local-file.txt
PS C:\Users\anshu> docker cp local-file.txt web2:/tmp/local-file.txt
Successfully copied 2.05kB to web2:/tmp/local-file.txt
PS C:\Users\anshu> docker exec web2 cat /tmp/local-file.txt
♦♦hello
PS C:\Users\anshu> |
```

#### Lab 4: Volume Management Commands

*# List all volumes*

```
docker volume ls
```

*# Create a volume*

```
docker volume create app-volume
```

*# Inspect volume details*

```
docker volume inspect app-volume
```

*# Remove unused volumes*

```
docker volume prune
```

*# Remove specific volume*

```
docker volume rm volume-name
```

*# Copy files to/from volume*

```
docker cp local-file.txt container-name:/path/in/volume
```

```

PS C:\Users\lanhuo> Set-Content -Path .env -Value @{
 >> "DATABASE_HOST=localhost",
 >> "DATABASE_PORT=5432",
 >> "API_KEY=secret123"
}
PS C:\Users\lanhuo> .env
DATABASE_HOST=localhost
DATABASE_PORT=5432
API_KEY=secret123
PS C:\Users\lanhuo> docker run -d --name env-test --env-file .env alpine sleep 3600
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
6a8ac1d17861: Pull complete
e0eb34cc3297: Download complete
a03813f19ccb: Download complete
Digest: sha256:5b16f432cf3da1b8d4c7ab6c487f2f5a8f996bc92140e0878d64a5019afde11
Status: Downloaded newer image for alpine:latest
2ba8cee4fa6f41816897948e14936184634981c07411546517edc2ae2ee47cf
PS C:\Users\lanhuo> docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
2ba8cee4fa6f alpine "sleep 3600" 5 seconds ago Up 4 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
e17b519a91da nginx "/docker-entrypoint. 13 minutes ago Up 13 minutes 80/tcp
e7269e21978e nginx "/docker-entrypoint. 47 minutes ago Up 47 minutes 80/tcp
8ad79e54b635 assignment-1-backend "docker-entrypoint.s 4 weeks ago Up 53 minutes 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
20b1eed594b6 assignment-1-database "docker-entrypoint.s 4 weeks ago Up 53 minutes
PS C:\Users\lanhuo> docker exec env-test printenv DATABASE_HOST
localhost
PS C:\Users\lanhuo> docker exec env-test printenv
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=2ba8cee4fa6f
DATABASE_HOST=localhost
DATABASE_PORT=5432
API_KEY=secret123
NONE/true
PS C:\Users\lanhuo> Set-Content -Path .env.secrets -Value @{
 >> "SECRET_KEY=abc123",
 >> "DEBUG=true"
}
PS C:\Users\lanhuo> docker run -d --name env-test2 --env-file .env --env-file .env.secrets alpine sleep 3600
a0918cd5372ba48b7685136224a8746f1372042f2b99acd133cb2c50546e53
PS C:\Users\lanhuo> docker exec env-test2 printenv

```

```

PS C:\Users\lanhuo> docker exec env-test2 printenv DATABASE_HOST
localhost
PS C:\Users\lanhuo> docker exec env-test2 printenv
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=2ba8cee4fa6f
DATABASE_HOST=localhost
DATABASE_PORT=5432
API_KEY=secret123
NONE/true
PS C:\Users\lanhuo> Set-Content -Path .env.secrets -Value @{
 >> "SECRET_KEY=abc123",
 >> "DEBUG=true"
}
PS C:\Users\lanhuo> docker run -d --name env-test2 --env-file .env --env-file .env.secrets alpine sleep 3600
a0918cd5372ba48b7685136224a8746f1372042f2b99acd133cb2c50546e53
PS C:\Users\lanhuo> docker exec env-test2 printenv
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=a0918cd5372b
DATABASE_HOST=localhost
DATABASE_PORT=5432
API_KEY=secret123
SECRET_KEY=abc123
DEBUG=true
NONE/true
PS C:\Users\lanhuo> docker exec env-test2 printenv

```

## 57.1 Part 2: Environment Variables

### Lab 1: Setting Environment Variables

## 57.2 Method 1: Using -e flag

```

Single variable
docker run -d \
 --name app1 \
 -e DATABASE_URL="postgres://user:pass@db:5432/mydb" \
 -e DEBUG="true" \
 -p 3000:3000 \
 my-node-app

Multiple variables
docker run -d \
 -e VAR1=value1 \
 -e VAR2=value2 \
 -e VAR3=value3 \
 my-app

```

### 57.2.1 Method 2: Using -env-file

```

Create .env file
echo "DATABASE_HOST=localhost" > .env
echo "DATABASE_PORT=5432" >> .env
echo "API_KEY=secret123" >> .env

Use env file
docker run -d \
 --env-file .env \

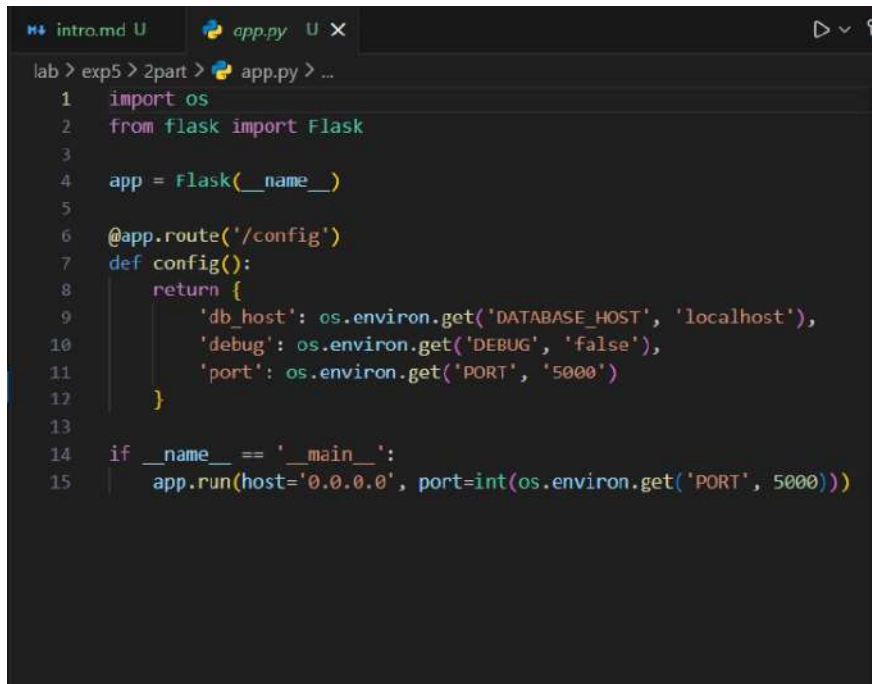
```

```

--name app2 \
my-app

Use multiple env files
docker run -d \
--env-file .env \
--env-file .env.secrets \
my-app

```



```

lab > exp5 > 2part > app.py > ...
1 import os
2 from flask import Flask
3
4 app = Flask(__name__)
5
6 @app.route('/config')
7 def config():
8 return {
9 'db_host': os.environ.get('DATABASE_HOST', 'localhost'),
10 'debug': os.environ.get('DEBUG', 'false'),
11 'port': os.environ.get('PORT', '5000')
12 }
13
14 if __name__ == '__main__':
15 app.run(host='0.0.0.0', port=int(os.environ.get('PORT', 5000)))

```

### 57.2.2 Method 3: In Dockerfile

```

Set default environment variables
ENV NODE_ENV=production
ENV PORT=3000
ENV APP_VERSION=1.0.0
Can be overridden at runtime

```

## 57.3 Lab 2: Environment Variables in Applications

Python Flask Example

```

app.py
import os
from flask import Flask

app = Flask(__name__)
Read environment variables
db_host = os.environ.get('DATABASE_HOST', 'localhost')
debug_mode = os.environ.get('DEBUG', 'false').lower() == 'true'
api_key = os.environ.get('API_KEY')

@app.route('/config')
def config():

```

```

 return {
 'db_host': db_host,
 'debug': debug_mode,
 'has_api_key': bool(api_key)
 }

if __name__ == '__main__':
 port = int(os.environ.get('PORT', 5000))
 app.run(host='0.0.0.0', port=port, debug=debug_mode)

```

```

1 FROM python:3.9-slim
2 WORKDIR /app
3 RUN pip install flask
4 COPY app.py .
5 ENV PORT=5000
6 CMD ["python", "app.py"]

```

### 57.3.1 Dockerfile with Environment Variables

```

Dockerfile
FROM python:3.9-slim
Set environment variables at build time
ENV PYTHONUNBUFFERED=1
ENV PYTHONDONTWRITEBYTECODE=1

WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY app.py .

Default runtime environment variables
ENV PORT=5000
ENV DEBUG=false

EXPOSE 5000
CMD ["python", "app.py"]

```

The screenshot shows a Windows PowerShell terminal window with the following commands and output:

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\anshu> mkdir flask-env-app

Directory: C:\Users\anshu

Mode LastWriteTime Length Name
---- -
d----- 4/24/2026 8:32 PM flask-env-app

PS C:\Users\anshu> cd flask-env-app
PS C:\Users\anshu\flask-env-app> notepad app.py
PS C:\Users\anshu\flask-env-app> echo flask > requirements.txt
PS C:\Users\anshu\flask-env-app> notepad Dockerfile

```



```

PS C:\Users\anshu\Flask-env-app> docker build -t flask-env-app .
[+] Building 9.6s (18/18) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 216B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [internal] load dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:240746991813db
=> => resolve docker.io/library/python:3.9-slim@sha256:240746991813db
=> [internal] load build context
=> => transferring context: 451B
=> [2/5] WORKDIR /app
=> [3/5] COPY requirements.txt
=> [4/5] RUN pip install -r requirements.txt
=> [5/5] COPY app.py
=> => exporting layers
=> => exporting manifest sha256:651b9c3b99e279d6c3e289d900166e6c9d33c
=> => exporting config sha256:991e8d81650716404b95d7b511bc7743d3f1a
=> => exporting sha256 manifest sha256:07a5d6523b7d20330311e2f
=> => exporting manifest list sha256:861122b6f626d56e55956f7911ah
=> => naming to docker.io/library/flask-env-app:latest
=> => unpacking to docker.io/library/flask-env-app:latest
PS C:\Users\anshu\Flask-env-app> docker run -d --name flask-app -p 5000:5000 --e DATABASE_HOST="prod-db.example.com" --e DEBUG="true" --e PORT="5000" flask-env-app
0218931d6bcb3b54f3628071748fbd376dad2550d499a867cf4ec30846557a
PS C:\Users\anshu\Flask-env-app> docker ps

```

| CONTAINER ID  | IMAGE                 | COMMAND                 | CREATED        | STATUS           | PORTS                                       | NAMES      |
|---------------|-----------------------|-------------------------|----------------|------------------|---------------------------------------------|------------|
| 0218931d6bcb  | flask-env-app         | "python app.py"         | 4 seconds ago  | Up 5 seconds     | 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp | flask-app  |
| 4d918cd5512b  | alpine                | "sleep 3600"            | 7 minutes ago  | Up 7 minutes     |                                             | env-test2  |
| 3b6c6e0f46f   | alpine                | "sleep 3600"            | 8 minutes ago  | Up 8 minutes     |                                             | env-test   |
| e17b519d91de  | nginx                 | "/docker-entrypoint..." | 21 minutes ago | Up 21 minutes    | 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp     | nginx-cust |
| ef7209e2f978e | nginx                 | "/docker-entrypoint..." | 53 minutes ago | Up 53 minutes    | 80/tcp                                      | web2       |
| 4ed79eb4b635  | assignment-1-backend  | "docker-entrypoint..."  | 4 weeks ago    | Up About an hour | 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp | backend_ap |
| 20b1ecd594b9  | assignment-1-database | "docker-entrypoint..."  | 4 weeks ago    | Up About an hour |                                             | postgres_d |

```

PS C:\Users\anshu\Flask-env-app> curl http://localhost:5000/config
Security Warning: Script Execution Risk
Invoke-MobRequest parses the content of the web page. Script code in the
web page might be run when the page is parsed.
RECOMMENDED ACTION:
Use the -UseBasicParsing switch to avoid script code execution.
Do you want to continue?
[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help
(default is "N"):y
StatusCode : 200
StatusDescription : OK
Content : [{"db_host":"prod-db.example.com","debug":"true","port":
"5000"}]
RawContent : HTTP/1.1 200 OK
Connection: close
Content-Length: 63
Content-Type: application/json
Date: Fri, 24 Apr 2026 15:05:25 GMT
Server: Werkzeug/3.1.0 Python/3.9.25
[{"db_host":"prod-db.example.com","d...
Forms : {}
Headers : [{"Connection", "close"}, [{"Content-Length", "63"},
[{"Content-Type", "application/json"}, [{"Date", "Fri, 24 Apr
2026 15:05:25 GMT"}]...
Images : {}
InputFields : {}
Links : {}
ParsedHtml : [Html.HtmlDocumentClass
RawContentLength : 63

```

```

b
PS C:\Users\anshu\Flask-env-app> curl http://localhost:5000/config
Security Warning: Script Execution Risk
Invoke-MobRequest parses the content of the web page. Script code in the
web page might be run when the page is parsed.
RECOMMENDED ACTION:
Use the -UseBasicParsing switch to avoid script code execution.
Do you want to continue?
[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help
(default is "N"):y
StatusCode : 200
StatusDescription : OK
Content : [{"db_host":"prod-db.example.com","debug":"true","port":
"5000"}]
RawContent : HTTP/1.1 200 OK
Connection: close
Content-Length: 63
Content-Type: application/json
Date: Fri, 24 Apr 2026 15:05:25 GMT
Server: Werkzeug/3.1.0 Python/3.9.25
[{"db_host":"prod-db.example.com","d...
Forms : {}
Headers : [{"Connection", "close"}, [{"Content-Length", "63"},
[{"Content-Type", "application/json"}, [{"Date", "Fri, 24 Apr
2026 15:05:25 GMT"}]...
Images : {}
InputFields : {}
Links : {}
ParsedHtml : [Html.HtmlDocumentClass
RawContentLength : 63

```

## 57.3.2 Lab 3: Test Environment Variables

*# Run with custom env vars*

```

docker run -d \
 --name flask-app \
 -p 5000:5000 \
 -e DATABASE_HOST="prod-db.example.com" \
 -e DEBUG="true" \
 -e PORT="8080" \
 flask-app

```

*# Check environment in running container*

```

docker exec flask-app env
docker exec flask-app printenv DATABASE_HOST

```

*# Test the endpoint*

```

curl http://localhost:5000/config

```

| CONTAINER ID | NAME         | CPU % | MEM USAGE / LIMIT   | MEM % | NET I/O         |
|--------------|--------------|-------|---------------------|-------|-----------------|
| 9218931d6bc3 | flask-app    | 0.02% | 20.36MiB / 7.595GiB | 0.26% | 1.57 kB / 842B  |
| 4d918cd55320 | env-test2    | 0.00% | 348KiB / 7.595GiB   | 0.00% | 176kB / 127kB   |
| 2ba8cee4fa6f | env-test     | 0.00% | 356KiB / 7.595GiB   | 0.00% | 41kB / 0B       |
| e17b519a91da | nginx-custom | 0.00% | 10.43MiB / 7.595GiB | 0.13% | 0B / 0B         |
| e7269e2f078e | web2         | 0.00% | 17.29MiB / 7.595GiB | 0.22% | 16.4kB / 4.1kB  |
| 04d79e54b635 | backend_app  | 0.00% | 18.77MiB / 7.595GiB | 0.24% | 14.4MB / 12.3kB |
| 20b1ecd594b9 | postgres_db  | 0.01% | 30.54MiB / 7.595GiB | 0.39% | 52.1MB / 0B     |
|              |              |       |                     |       | 40.8MB / 680kB  |

### 57.3.3 Part 3: Docker Monitoring

#### 57.3.4 Lab 1: Basic Monitoring Commands

#### 57.3.5 docker stats - Real-time Container Metrics

```
Live stats for all containers
docker stats

Live stats for specific containers
docker stats container1 container2

Specific format output
docker stats --format "table \t\t\t"

No-stream (single snapshot)
docker stats --no-stream

All containers (including stopped)
docker stats --all

Useful Format Options:
Custom format
docker stats --format "Container: | CPU: | Memory: "
JSON output
docker stats --format json --no-stream
Wide output
docker stats --no-stream --no-trunc
```

```
PS C:\Users\lanhuo> docker run -d --name monitor-test nginx
2eab8fd8bd72877b6132a88dc47d4aa5d8bf618dc3549ca69f45e7a82e4
PS C:\Users\lanhuo> docker logs monitor-test
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/04/24 15:11:18 [notice] 1#1: using the "epoll" event method
2026/04/24 15:11:18 [notice] 1#1: nginx/1.29.5
2026/04/24 15:11:18 [notice] 1#1: built by gcc 18.2.0 (Debian 18.2.0-19)
2026/04/24 15:11:18 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/04/24 15:11:18 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/04/24 15:11:18 [notice] 1#1: start worker processes
2026/04/24 15:11:18 [notice] 1#1: start worker process 29
2026/04/24 15:11:18 [notice] 1#1: start worker process 30
2026/04/24 15:11:18 [notice] 1#1: start worker process 31
2026/04/24 15:11:18 [notice] 1#1: start worker process 32
2026/04/24 15:11:18 [notice] 1#1: start worker process 33
2026/04/24 15:11:18 [notice] 1#1: start worker process 34
2026/04/24 15:11:18 [notice] 1#1: start worker process 35
2026/04/24 15:11:18 [notice] 1#1: start worker process 36
2026/04/24 15:11:18 [notice] 1#1: start worker process 37
2026/04/24 15:11:18 [notice] 1#1: start worker process 38
2026/04/24 15:11:18 [notice] 1#1: start worker process 39
2026/04/24 15:11:18 [notice] 1#1: start worker process 40
PS C:\Users\lanhuo> docker logs -f monitor-test
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/04/24 15:11:18 [notice] 1#1: using the "epoll" event method
```

### 57.3.6 Lab 2: docker top - Process Monitoring

```
View processes in container
docker top container-name

View with full command line
docker top container-name -ef

Compare with host processes
ps aux | grep docker
```

### 57.3.7 Lab 3: docker logs - Application Logs

```
View logs
docker logs container-name

Follow logs (like tail -f)
docker logs -f container-name

Last N lines
docker logs --tail 100 container-name

Logs with timestamps
docker logs -t container-name

Logs since specific time
docker logs --since 2024-01-15 container-name

Combine options
docker logs -f --tail 50 -t container-name
```

```
PS C:\Users\anshu> docker network ls
NETWORK ID NAME DRIVER SCOPE
f21c08ba398e assignment-1_synnetwork bridge local
2e97e3ba3763 bridge bridge local
b09b8e23f0ff host host local
bda7ad7f6c4d mynetwork macvlan local
85f6695f719c none null local
PS C:\Users\anshu> docker network create my-network
2a833fed58be198c62e2ddcf6e53fb69a15e868ac85c22c9754e84cc85bca8a0
PS C:\Users\anshu> docker network ls
NETWORK ID NAME DRIVER SCOPE
f21c08ba398e assignment-1_synnetwork bridge local
2e97e3ba3763 bridge bridge local
b09b8e23f0ff host host local
2a833fed58be mynetwork macvlan local
bda7ad7f6c4d mynetwork macvlan local
85f6695f719c none null local
PS C:\Users\anshu> docker network inspect my-network
[
 {
 "Name": "my-network",
 "Id": "2a833fed58be198c62e2ddcf6e53fb69a15e868ac85c22c9754e84cc85bca8a0",
 "Created": "2020-04-24T13:13:19.332422809Z",
 "Scope": "local",
 "Driver": "bridge",
 "EnableIPv6": true,
 "EnableIPV6": false,
 "IPAM": {
 "Driver": "default",
 "Options": {},
 "Config": [
 {
 "Subnet": "172.19.0.0/16",
 "Gateway": "172.19.0.1"
 }
]
 },
 "Internal": false,
 "Attachable": false,
 "Ingress": false,
 "ConfigFrom": {}
 }
]
```

```
Windows PowerShell
{
 "Name": "my-network",
 "Id": "2a833fed58be198c62e2ddcf6e53fb69a15e868ac85c22c9754e84cc85bca8a0",
 "Created": "2020-04-24T13:13:19.332422809Z",
 "Scope": "local",
 "Driver": "bridge",
 "EnableIPv6": true,
 "EnableIPV6": false,
 "IPAM": {
 "Driver": "default",
 "Options": {},
 "Config": [
 {
 "Subnet": "172.19.0.0/16",
 "Gateway": "172.19.0.1"
 }
]
 },
 "Internal": false,
 "Attachable": false,
 "Ingress": false,
 "ConfigFrom": {
 "Network": ""
 },
 "ConfigOnly": false,
 "Options": {
 "com.docker.network.enable_ipv6": "true",
 "com.docker.network.enable_ipv6": "false"
 },
 "Labels": {},
 "Containers": {},
 "Status": {
 "IPAM": {
 "Subnets": {
 "172.19.0.0/16": {
 "IPIndex": 3,
 "DynamicIPAvailable": 65533
 }
 }
 }
 }
}
```

## 57.3.8 Lab 4: Container Inspection

# Detailed container info

docker inspect container-name

# Specific information

docker inspect --format='{{ .Name }}' container-name

docker inspect --format='{{ .Image }}' container-name

docker inspect --format='{{ .Status }}' container-name

# Resource limits

docker inspect --format='{{ .MemLimit }}' container-name

docker inspect --format='{{ .CPUShares }}' container-name

Lab 5: Events Monitoring

Bash

# Monitor Docker events in real-time

docker events

# Filter events

docker events --filter 'type=container'

docker events --filter 'event=start'

docker events --filter 'event=die'

# Since specific time

docker events --since '2024-01-15'

# Format output

```
docker events --format ' '
```

### 57.3.9 Lab 6: Practical Monitoring Script

```
#!/bin/bash
monitor.sh
Simple Docker monitoring

echo "=== Docker Monitoring Dashboard ==="
echo "Time: $(date)"
echo

echo "1. Running Containers:"
docker ps --format "table \t\t"
echo

echo "2. Resource Usage:"
docker stats --no-stream --format "table \t\t\t"
echo

echo "3. Recent Events:"
docker events --since '5m' --until '0s' --format ' ' | tail -5
echo

echo "4. System Info:"
docker system df
```

### 57.3.10 Part 4: Docker Networks

#### 57.3.11 Lab 1: Understanding Docker Network Types

```
Default networks
docker network ls

Output:
NETWORK ID NAME DRIVER SCOPE
abc123 bridge bridge local
def456 host host local
ghi789 none null local
Lab 2: Network Types Explained
1. Bridge Network (Default)
Bash
Containers on bridge network can communicate
Each container gets own IP, isolated from host

Create custom bridge network
docker network create my-network

Inspect network
docker network inspect my-network

Run containers on custom network
docker run -d --name web1 --network my-network nginx
docker run -d --name web2 --network my-network nginx

Containers can communicate using container names
docker exec web1 curl http://web2
```

```

PS C:\Users\lanshu> docker run -d --name host-app --ip 8888:88 nginx
889ebd2dc93628e3ffad4e1badc980e41b914b1b84870740095dd38404eade5f
PS C:\Users\lanshu> curl http://localhost:8888

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the
web page might be run when the page is parsed.
RECOMMENDED ACTION:
Use the -useBasicParsing switch to avoid script code execution.
Do you want to continue?
[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help
(default is "N"):y

StatusCode : 200
StatusDescription : OK
Content : <!DOCTYPE html>
 <html>
 <head>
 <title>Welcome to nginx</title>
 <style>
 html { color:scheme: light dark; }
 body { width: 350px; margin: 0 auto;
 font-family: Tahoma, Verdana, Arial, sans-serif; }
 </style>...
RawContent : HTTP/1.1 200 OK
 Connection: keep-alive
 Accept-Ranges: bytes
 Content-Length: 615
 Content-Type: text/html
 Date: Fri, 24 Apr 2026 15:19:55 GMT
 ETag: "60835104-267"
 Last-Modified: Wed, 04 Feb 2026 ...
Forms : {}
Headers : [[Connection, keep-alive], [Accept-Ranges, bytes],
 [Content-Length, 615], [Content-Type, text/html]...]
Images : {}
InputFields : {}
Links : [[InnerHTML=nginx.org], [InnerText=nginx.org]]

```

## 57.3.12 2. Host Network

# Container uses host's network directly  
# No network isolation, shares host's IP

`docker run -d --name host-app --network host nginx`

# Access directly on host port 80  
`curl http://localhost`

```

PS C:\Users\lanshu> docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
ed6b2c3b432 alpine "sleep 3600" 22 seconds ago Up 22 seconds host-app
889ebd2dc936 nginx "/docker-entrypoint..." About a minute ago Up About a minute 0.0.0.0:8080->88/tcp, [::]:8080->88/tcp host-app
2ac233fcc0e9 nginx "/docker-entrypoint..." 7 minutes ago Up 7 minutes 88/tcp web1
3aabb8fd88d7 nginx "/docker-entrypoint..." 9 minutes ago Up 9 minutes 88/tcp monit
or-test
9218931d8b3c flask-env-app "python app.py" 16 minutes ago Up 16 minutes 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp flask-app
4c918cd5532b alpine "sleep 3600" 23 minutes ago Up 23 minutes env-t
wa22 alpine "sleep 3600" 24 minutes ago Up 23 minutes env-t
2ba8c4e4fa5f alpine "sleep 3600" 24 minutes ago Up 23 minutes env-t
c7269e2f078e nginx "/docker-entrypoint..." About an hour ago Up About an hour 88/tcp web2
88d79e54b635 assignment-1-backend "docker-entrypoint..." 4 weeks ago Up About an hour 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp backe
nd-app
20b1ecd59409 assignment-1-database "docker-entrypoint.s..." 4 weeks ago Up About an hour postg
res_db

PS C:\Users\lanshu> docker exec isolated-app ifconfig
lo
 link-local: 127.0.0.1 Mask: 255.0.0.0
 inet addr: 172.17.0.1 Scope: host
 UP LOOBACK RUNNING MTU: 65536 Metric: 1
 RX packets: 0 errors: 0 dropped: 0 overruns: 0 frame: 0
 TX packets: 0 errors: 0 dropped: 0 overruns: 0 carrier: 0
 collisions: 0 timeout: 1000
 RX bytes: 0 (0.0 B) TX bytes: 0 (0.0 B)

PS C:\Users\lanshu> docker exec isolated-app ping google.com
ping: bad address 'google.com'
PS C:\Users\lanshu> docker inspect isolated-app
[
 {

```

## 57.3.13 3. None Network

# No network access

`docker run -d --name isolated-app --network none alpine sleep 3600`

# Test no network interfaces

`docker exec isolated-app ifconfig`

```

PS C:\Users\anshu> docker swarm init
Swarm initialized: current node (3xaj7u1z8771pw0uh671x9thv) is now a manager.

To add a worker to this swarm, run the following command:

 docker swarm join --token SWMTKN-1-4btt2hcailtdylyf8cfvsk7l9kqf6h6ijeu9luggynij33w4-7gqzif9y4alfbc2krl8p34yp 192.168.65.3:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
PS C:\Users\anshu> docker network create --driver overlay my-overlay
c80eydygjn7Cf5ctvcttc0Gq
PS C:\Users\anshu> docker network ls
NETWORK ID NAME DRIVER SCOPE
f21e98b8e98e assignment-1_synnetwork bridge local
2e9763b03763 bridge bridge local
d1b51c225633 docker_gwbridge bridge local
16988c23f4ff host host local
32a8b11povax ingress overlay swarm
2a833fed5b8e my-network bridge local
c80eydygjn7t my-overlay overlay swarm
b0da7ad74c4d mynetwork macvlan local
80f669ff719c none null local
PS C:\Users\anshu> docker service create --name web1 --network my-overlay nginx
4fg29yuhun8umtfl4nlyxy
overall progress: 1 out of 1 tasks
1/1: running
verify: Service 4fg29yuhun8umtfl4nlyxy converged
PS C:\Users\anshu> docker service create --name web2 --network my-overlay nginx
d0bedc0eae9ayllt8v9gdr
overall progress: 1 out of 1 tasks
1/1: running
verify: Service d0bedc0eae9ayllt8v9gdr converged
PS C:\Users\anshu>

```

## 57.3.14 4. Overlay Network (Swarm)

# For Docker Swarm multi-host networking

docker network create --driver overlay my-overlay

```

Windows PowerShell
PS C:\Users\anshu> docker run -d --name web1 nginx
a743b5c3a8572801da2cbe1b28172e19bce9783293feb981b5a4353ca6f3362b
PS C:\Users\anshu> docker network connect app-network web1
PS C:\Users\anshu> docker inspect web1
[
 {
 "Id": "a743b5c3a8572801da2cbe1b28172e19bce9783293feb981b5a4353ca6f3362b",
 "CreatedAt": "2020-08-20T15:20:22.386726851Z",
 "Path": "/docker-entrypoint.sh",
 "Args": [
 "nginx",
 "-g",
 "daemon off;"
],
 "State": {
 "Status": "running",
 "Running": true,
 "Paused": false,
 "Restarting": false,
 "OOMKilled": false,
 "Dead": false,
 "Pid": 3855,
 "ExitCode": 0,
 "Error": "",
 "StartedAt": "2020-08-20T15:20:22.516159978Z",
 "FinishedAt": "2020-08-20T15:20:22.516159978Z"
 },
 "Image": "sha256:381b0f0c6c5778d802f6a1f8710f6e352cd28ab6d62d4086d712288",
 "ResolveConfPath": "/var/lib/docker/containers/a743b5c3a8572801da2cbe1b28172e19bce9783293feb981b5a4353ca6f3362b/resolve.conf",
 "HostnamePath": "/var/lib/docker/containers/a743b5c3a8572801da2cbe1b28172e19bce9783293feb981b5a4353ca6f3362b/hostname",
 "HostsPath": "/var/lib/docker/containers/a743b5c3a8572801da2cbe1b28172e19bce9783293feb981b5a4353ca6f3362b/hosts",
 "LogPath": "/var/lib/docker/containers/a743b5c3a8572801da2cbe1b28172e19bce9783293feb981b5a4353ca6f3362b/at743b5c3a8572801da2cbe1b28172e19bce9783293feb981b5a4353ca6f3362b-json.log",
 "Name": "/web1",
 "RestartCount": 0,
 "Driver": "overlayfs",
 "Platform": "linux",
 "MountLabel": "",
 "ProcessLabel": "",
 "AppArmorProfile": "",
 "ExecIDs": null,
 }
]

```

## 57.3.15 Lab 3: Network Management Commands

# Create network

docker network create app-network

docker network create --driver bridge --subnet 172.20.0.1/16 my-subnet

# Connect container to network

docker network connect app-network existing-container

# Disconnect container from network

docker network disconnect app-network container-name

# Remove network

docker network rm network-name

# Prune unused networks

docker network prune

```

PS C:\Users\anshu> docker network create app-network
d6f5b99485c833a06d046d01d06d699ad08e015f1fbcf87c21b99a85c0b3bd7c
PS C:\Users\anshu> docker run -d --name postgres-db --network app-network -e
POSTGRES_PASSWORD=secret -v pgdata:/var/lib/postgresql/data postgres:15
Unable to find image 'postgres:15' locally
15: Pulling from library/postgres
26803cb302a4: Pull complete
dd1cc88a000a: Pull complete
ff8482cd03b7: Pull complete
00da1877cea6: Pull complete
ac28d232c7f3: Pull complete
0892560966bb: Pull complete
be67b9cdfb60: Pull complete
58e6420a0782: Pull complete
c375454b4ff0: Pull complete
3e0c43622abe: Pull complete
9fa019823549: Pull complete
fa77b44f96d2: Pull complete
4c7f2d7f4f80: Pull complete
14c26c45f021: Download complete
ee662d4e76ff: Download complete
Digest: sha256:29342cb52157b098821961d2c14eec3c019071f56a5d559e990cf07cf541e
a9b
Status: Downloaded newer image for postgres:15
0ee017fd813e7bf53713dd1f049240b5908433b76c2e241ade4781bb632ef150

```

```

PS C:\Users\HP\Desktop> docker inspect --format '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' postgres-db
172.18.0.2
PS C:\Users\HP\Desktop> docker port web-app
8080/tcp -> 0.0.0.0:8080
8080/tcp -> :::8080

```

### 57.3.16 Lab 4: Multi-Container Application Example

Web App + Database Communication

```

Create network
docker network create app-network

Start database
docker run -d \
 --name postgres-db \
 --network app-network \
 -e POSTGRES_PASSWORD=secret \
 -v pgdata:/var/lib/postgresql/data \
 postgres:15

Start web application
docker run -d \
 --name web-app \
 --network app-network \
 -p 8080:3000 \
 -e DATABASE_URL="postgres://postgres:secret@postgres-db:5432/mydb" \
 -e DATABASE_HOST="postgres-db" \
 node-app

Web app can connect to database using "postgres-db" hostname

```

### 57.3.17 Lab 5: Network Inspection & Debugging

```

Inspect network
docker network inspect bridge

Check container IP
docker inspect --format '{{.IPAddress}}' container-name

DNS resolution test
docker exec container-name nslookup another-container

```



```
Network connectivity test
docker exec container-name ping -c 4 google.com
docker exec container-name curl -I http://another-container

View network ports
docker port container-name
```

### 57.3.18 Lab 6: Port Publishing vs Exposing

```
PORT PUBLISHING (host:container)
docker run -d -p 80:8080 --name app1 nginx
Host port 80 → Container port 8080

Dynamic port publishing
docker run -d -p 8080 --name app2 nginx
Docker assigns random host port

Multiple ports
docker run -d -p 80:80 -p 443:443 --name app3 nginx

Specific host IP
docker run -d -p 127.0.0.1:8080:80 --name app4 nginx

EXPOSE in Dockerfile (metadata only)
Dockerfile: EXPOSE 80
Still need -p to publish
```

```
PS C:\Users\anshu> docker rm -f app1 app2 app3 app4 2>$null
PS C:\Users\anshu> docker run -d -p 8080:80 --name app1 nginx
d027e66c2b9ba563ede9261b1f12feab8b6ef9031c6795ad905bc5d02f3785f2
PS C:\Users\anshu> curl http://localhost:8080

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in
the web page might be run when the page is parsed.
RECOMMENDED ACTION:
Use the -UseBasicParsing switch to avoid script code
execution.

Do you want to continue?
[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend
[?] Help(default is "N"): y

StatusCode : 200
StatusDescription: OK
Content : <!DOCTYPE html>
 <html>
 <head>
 <title>Welcome to nginx!</title>
 <style>
 html { color:scheme: light dark; }
 body { width: 35em; margin: 0 auto;
 font-family: Tahoma, Verdana, Arial, sans-serif;
 }
 </style>...
RawContent : HTTP/1.1 200 OK
 Connection: keep-alive
 Accept-Ranges: bytes
 Content-Length: 615
 Content-type: text/html
 Date: Fri, 24 Apr 2026 15:57:18 GMT
 ETag: "698361d4-267"
 Last-Modified: Wed, 04 Feb 2026 ...
```

### 57.3.19 Part 5: Complete Real-World Example

Application Architecture:

Flask Web App (port 5000)

PostgreSQL Database (port 5432)

Redis Cache (port 6379)

All connected via custom network

Implementation:

```
1. Create network
docker network create myapp-network
```

*# 2. Start database with volume*

```
docker run -d \
 --name postgres \
 --network myapp-network \
 -e POSTGRES_PASSWORD=mysecretpassword \
 -e POSTGRES_DB=mydatabase \
 -v postgres-data:/var/lib/postgresql/data \
 postgres:15
```

*# 3. Start Redis*

```
docker run -d \
 --name redis \
 --network myapp-network \
 -v redis-data:/data \
 redis:7-alpine
```

*# 4. Start Flask app with all configurations*

```
docker run -d \
 --name flask-app \
 --network myapp-network \
 -p 5000:5000 \
 -v $(pwd)/app:/app \
 -v app-logs:/var/log/app \
 -e
```

```
DATABASE_URL="postgresql://postgres:mysecretpassword@postgres:5432/mydatabase" \
 \
```

```
 -e REDIS_URL="redis://redis:6379" \
 -e DEBUG="false" \
 -e LOG_LEVEL="INFO" \
 --env-file .env.production \
 flask-app:latest
```

Monitoring commands

*# Check all components*

```
docker ps
```

*# Monitor resources*

```
docker stats postgres redis flask-app
```

*# Check logs*

```
docker logs -f flask-app
```

*# Network connectivity test*

```
docker exec flask-app ping -c 2 postgres
```

```
docker exec flask-app ping -c 2 redis
```

*# View network details*

```
docker network inspect myapp-network
```



To understand the relationship between `docker run` and Docker Compose, and to compare their configuration syntax and use cases by deploying single-container and multi-container applications.

---

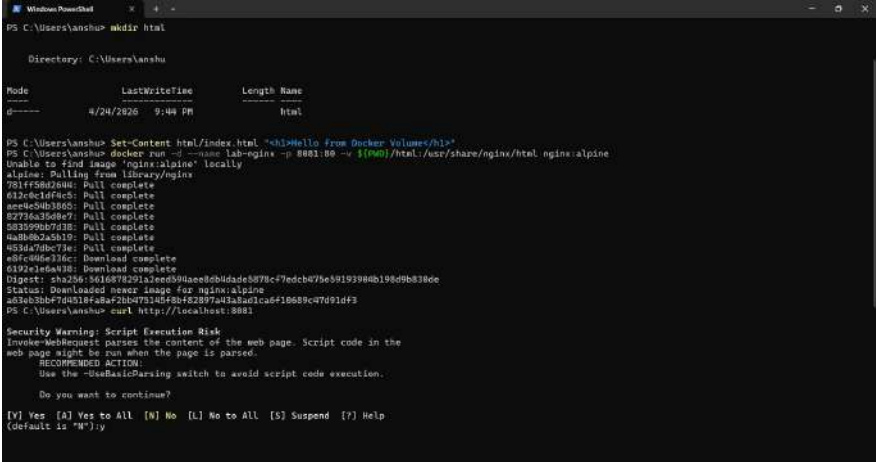
## 59.2 PART A - Theory

### 59.2.1 1. Docker Run (Imperative Approach)

The `docker run` command creates and starts a container from an image. It requires explicit flags for port mapping (`-p`), volume mounting (`-v`), environment variables (`-e`), network configuration (`--network`), restart policies (`--restart`), resource limits (`--memory`, `--cpus`), and container name (`--name`).

#### Example:

```
docker run -d \
 --name my-nginx \
 -p 8080:80 \
 -v ./html:/usr/share/nginx/html \
 -e NGINX_HOST=localhost \
 nginx:alpine
```



```
PS C:\Users\anshu> mkdir html

Directory: C:\Users\anshu

Mode LastWriteTime Length Name
---- -
d----- 4/24/2026 9:44 PM html

PS C:\Users\anshu> Set-Content html/index.html "<h1>Hello from Docker Volume</h1>"
PS C:\Users\anshu> docker run -d --name my-nginx -p 8080:80 -v i[900]/html:/usr/share/nginx/html nginx:alpine
Unable to find image 'nginx:alpine' locally
alpine: Pulling from library/nginx
281f50b2d80d: Pull complete
612c8c1d4c5: Pull complete
aee4e5b3b65: Pull complete
82726c3d497: Pull complete
583d99b7d38: Pull complete
4a8b862a5b19: Pull complete
453d47bc7fe: Pull complete
a8fc0d4e116c: Download complete
6192e1ed418: Download complete
Digest: sha256:161d8772912eed90aee8dbfdadeb778cf7edcb475e5919394db19d9b838ee
Status: Downloaded newer image for nginx:alpine
a63db3bbf7d4518fa8af2bb473145f8bfe82897a43a8ad1ca6f10689c47d91df3
PS C:\Users\anshu> curl http://localhost:8080

Security Warning: Script Execution Risk
Invoke-WebRequest: The content of the web page. Script code in the
web page might be run when the page is parsed.
RECOMMENDED ACTION:
Use the -UseBasicParsing switch to avoid script code execution.
Do you want to continue?
[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help
(default is "N").y
```



```
Status: 200
StatusDescription: OK
Content: <h1>Hello from Docker Volume</h1>
RawContent: HTTP/1.1 200 OK
Connection: keep-alive
Accept-Ranges: bytes
Content-Length: 35
Content-Type: text/html
Date: Fri, 24 Apr 2026 16:14:53 GMT
Etag: "60b9feb-25"
Last-Modified: Fri, 24 Apr 2026 16:14:53 GMT
Forms: {}
Headers: {[Connection, keep-alive], [Accept-Ranges, bytes], [Content-Length, 35], [Content-Type, text/html]...}
Images: {}
InputFields: {}
Links: {}
ParsedHtml: mhtml:HTMLDocumentClass
RawContentLength: 35

PS C:\Users\anshu>
```

### 59.2.2 2. Docker Compose (Declarative Approach)

Docker Compose uses a YAML file (`docker-compose.yml`) to define services, networks, and volumes in a structured format. Instead of multiple commands, a single command is used: `docker compose up -d`

#### Equivalent Compose file:

```

version: '3.8'
services:
 nginx:
 image: nginx:alpine
 container_name: my-nginx
 ports:
 - "8080:80"
 volumes:
 - ./html:/usr/share/nginx/html
 environment:
 NGINX_HOST: localhost
 restart: unless-stopped

```

```

lab > exp6 > docker-compose.yml
1 version: '3.8'
2 services:
3 nginx:
4 image: nginx:alpine
5 container_name: my-nginx
6 ports:
7 - "8080:80"
8 volumes:
9 - ./html:/usr/share/nginx/html
10 environment:
11 NGINX_HOST: localhost
12 restart: unless-stopped

```

### 59.2.3 3. Mapping: Docker Run vs Docker Compose

| Docker Run Flag   | Docker Compose Equivalent      |
|-------------------|--------------------------------|
| -p 8080:80        | ports:                         |
| -v host:container | volumes:                       |
| -e KEY=value      | environment:                   |
| --name            | container_name:                |
| --network         | networks:                      |
| --restart         | restart:                       |
| --memory          | deploy.resources.limits.memory |
| --cpus            | deploy.resources.limits.cpus   |
| -d                | docker compose up -d           |

### 59.2.4 4. Advantages of Docker Compose

1. Simplifies multi-container applications
2. Provides reproducibility
3. Version controllable configuration
4. Unified lifecycle management
5. Supports service scaling: `docker compose up --scale web=3`

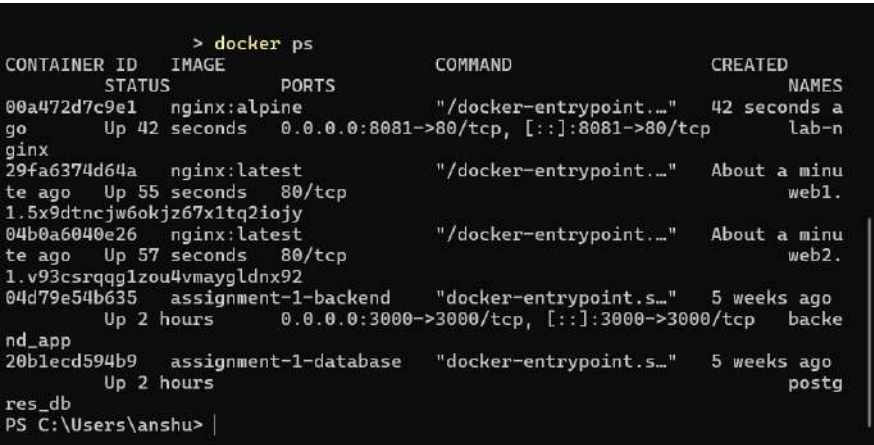
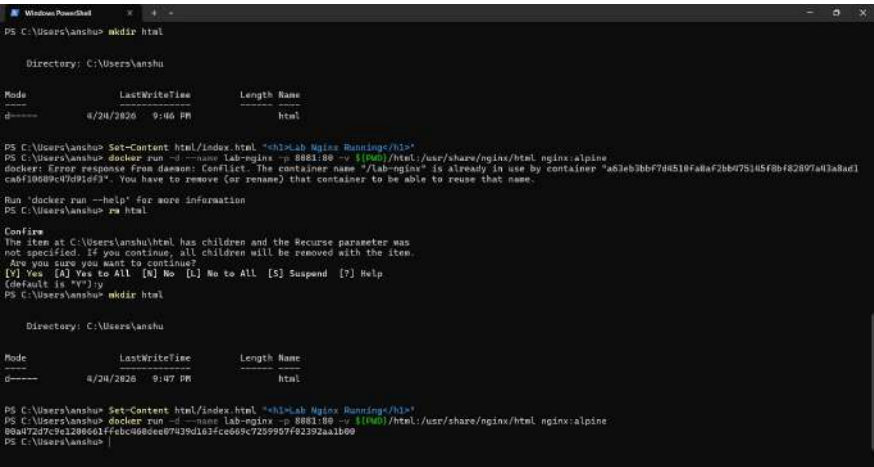
## 59.3 PART B - Practical Implementation

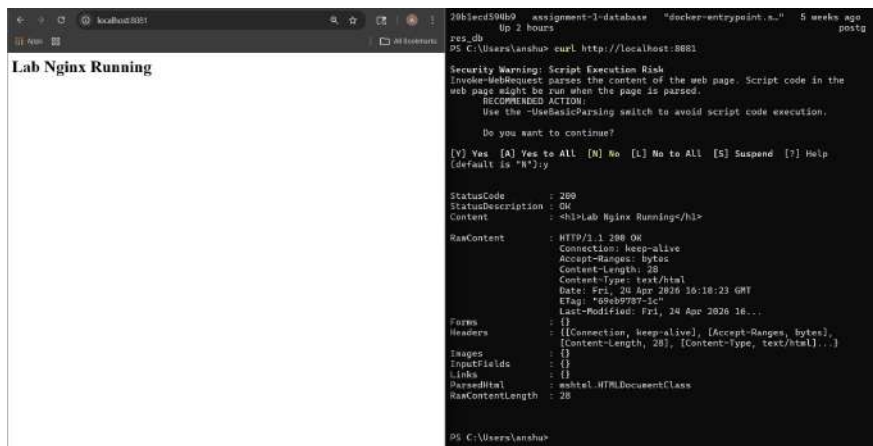
### 59.3.1 Task 1: Single Container Comparison

#### 59.3.1.1 Step 1: Run Nginx Using Docker Run

**Command:**

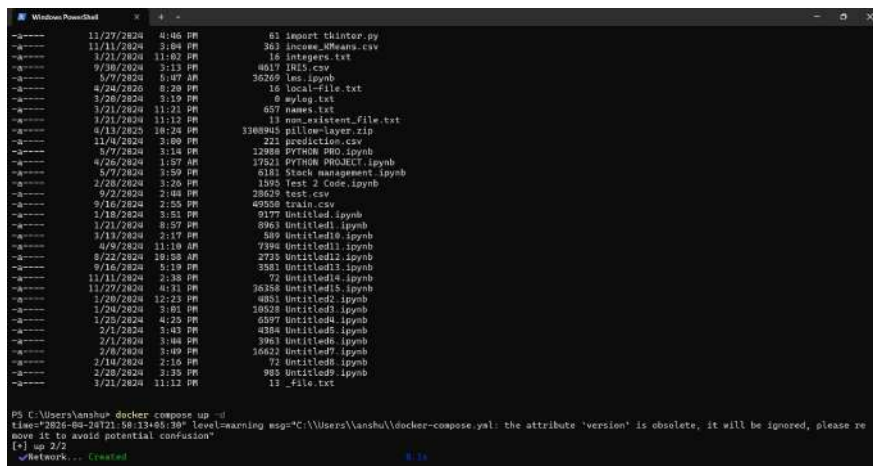
```
docker run -d \
 --name lab-nginx \
 -p 8081:80 \
 -v $(pwd)/html:/usr/share/nginx/html \
 nginx:alpine
```





Verify container is running:

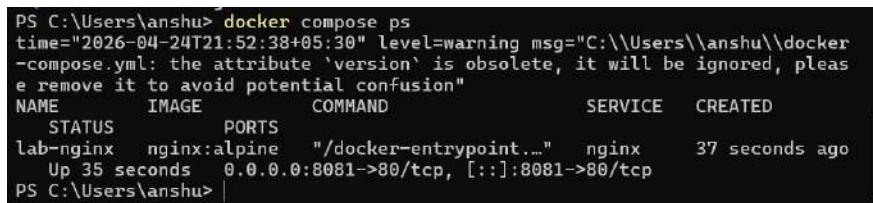
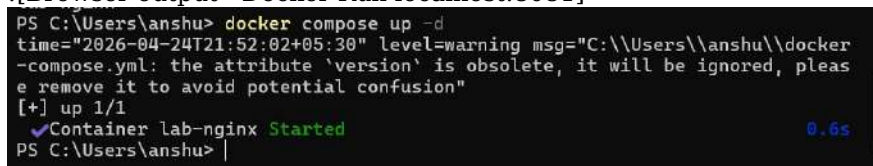
docker ps



Access in browser: <http://localhost:8081>

📄 Screenshot - Browser output (Docker Run):

![Browser output - Docker Run localhost:8081]



Stop and remove container:

docker stop lab-nginx

docker rm lab-nginx



---

### 59.3.1.2 Step 2: Run Same Setup Using Docker Compose

Create docker-compose.yml:

```
version: '3.8'
services:
 nginx:
 image: nginx:alpine
 container_name: lab-nginx
 ports:
 - "8081:80"
 volumes:
 - ./html:/usr/share/nginx/html
```

Run container:

docker compose up -d

```
PS C:\Users\anshu> docker network create wp-net
03e345c7c396e285d07764c57c8e2176b14c1d38d45537a8ef89917bala943a7
PS C:\Users\anshu> |
```

Verify:

docker compose ps



Stop containers:

docker compose down

**59.3.1.2.1 Command Explanation: Docker compose down: Stop and remove all services, networks (but preserves volumes)**

---

## 59.3.2 Task 2: Multi-Container Application – WordPress + MySQL

---

### 59.3.2.1 Part A: Using Docker Run (Manual Method)

Step 1: Create network:

docker network create wp-net

```
PS C:\Users\anshu> docker network create wp-net
fd17f6c7f0f6ac727cdcc7ad22ba5d0715e5da0bcb0e0f72ec1c0401f8fb
PS C:\Users\anshu> docker run -d --name mysql --network wp-net -- MYSQL_ROOT_PASSWORD=secret -- MYSQL_DATABASE=wordpress -- MYSQL_USER=wpuser -- MYSQL_PASSWORD=wpuser -- mysql_data:/var/lib/mysql mysql:5.7
Unable to find image 'mysql:5.7' locally
5.7 Pulling from library/mysql
df9add85549b: Pull complete
f7c8e90f488: Pull complete
c56c3b0c979: Pull complete
43d85e928198: Pull complete
664b3d298fba: Pull complete
e9193a1c24cc: Pull complete
90986b0b0dcd: Pull complete
ae73130c9799: Pull complete
6b95a04007bd: Pull complete
68c3898c3815: Pull complete
2b0dcac0e69: Pull complete
90889c07a44a: Download complete
9eb6a52b8441: Download complete
Digest: sha256:40c0b0c3036d604333676cae56576f4b0156d71bae83c8145cbe47c2aad73bb
Status: Downloaded newer image for mysql:5.7
ec57f5ee7c2b28cb131af071f8c97bf825a95a8748d00c86a51a83918dc9ate0
PS C:\Users\anshu> |
```

Step 2: Run MySQL container:

```
docker run -d \
--name mysql \
```



```
--network wp-net \
-e MYSQL_ROOT_PASSWORD=secret \
-e MYSQL_DATABASE=wordpress \
mysql:5.7
```

```
PS C:\Users\anshu> docker run -d --name wordpress --network wp-net -p 8082:80 -e WORDPRESS_DB_HOST=mysql:3306 -e WORDPRESS_DB_USER=root -e WORDPRESS_DB_PASSWORD=root --wp-content=/var/www/html/wp-content wordpress:latest
Unable to find image 'wordpress:latest' locally
latest: Pulling from library/wordpress
a5f587a05427: Pull complete
2a08301d771e: Pull complete
6d6c52a8f6c3b: Pull complete
2079a636ef2d: Pull complete
79b094a5a7d2: Pull complete
caca5d0c0a6f: Pull complete
23baa80ca7f: Pull complete
f7417fc09aeb: Pull complete
c70d51c9e3e9: Pull complete
9f7cb9b3b5f2e: Pull complete
a529d4d615a2: Pull complete
0ba92b6179b: Pull complete
bd5f9b09a043: Pull complete
c13d3816906f: Pull complete
1394fc79f3a1: Pull complete
08d8ea2c394d: Pull complete
099b0c4f0b05: Pull complete
a8d1ead0bb32: Pull complete
9ebba276089f: Pull complete
a923b3e0a221: Pull complete
27b0f9ec9d4b: Pull complete
0f4b78baf5d: Pull complete
072aee0e356c: Pull complete
777e77f17b06: Download complete
07e7dd823dd: Download complete
Digest: sha256:73af0310a9938833ca1a8c29b2b853c200bba8e71cb0a32372bae32aff2f29
Status: Downloaded newer image for wordpress:latest
0b09d6b2d482b9670d95d0962a29823c470b0e90e2ad37a7b588c17f84a47
PS C:\Users\anshu>
```

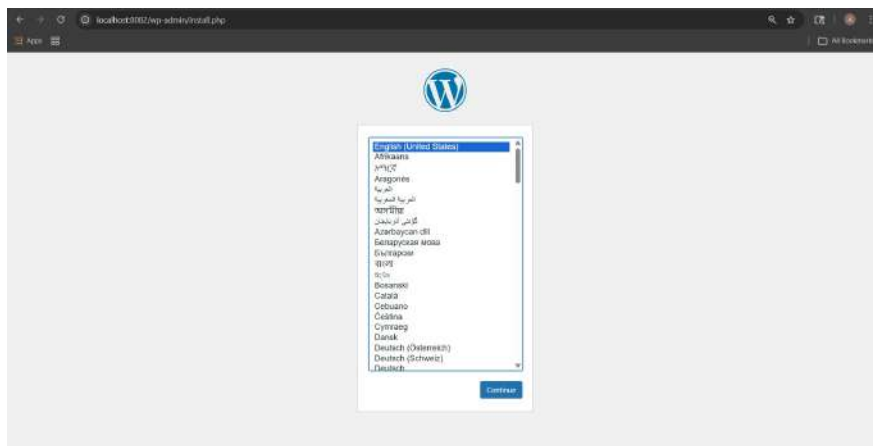
### Step 3: Run WordPress container:

```
docker run -d \
--name wordpress \
--network wp-net \
-p 8082:80 \
-e WORDPRESS_DB_HOST=mysql \
-e WORDPRESS_DB_PASSWORD=secret \
wordpress:latest
```

```
PS C:\Users\anshu> docker ps
CONTAINER ID IMAGE PORTS COMMAND CREATED
STATUS NAMES
4bb9d64b2640 wordpress 0.0.0.0:8082->80/tcp, [::]:8082->80/tcp 43 seconds ago
go Up 42 seconds
ec67f5ee7c2b mysql:5.7 3306/tcp, 33060/tcp "docker-entrypoint.s..." 2 minutes ago
o Up 2 minutes
3131652f18f1 nginx:alpine "/docker-entrypoint..." 6 minutes ago
o Up 6 minutes
29fa6374d64a nginx:latest "/docker-entrypoint..." 11 minutes ago
go Up 11 minutes
9dtncjw6okjz67x1tq2iojy
04b0a6040e26 nginx:latest "/docker-entrypoint..." 11 minutes ago
go Up 11 minutes
3csrqqglzou4vmaygldnx92
04d79e54b635 assignment-1-backend "docker-entrypoint.s..." 5 weeks ago
Up 2 hours
0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
pp Up 2 hours
20b1ecd594b9 assignment-1-database "docker-entrypoint.s..." 5 weeks ago
Up 2 hours
db Up 2 hours
postgres_
PS C:\Users\anshu>
```

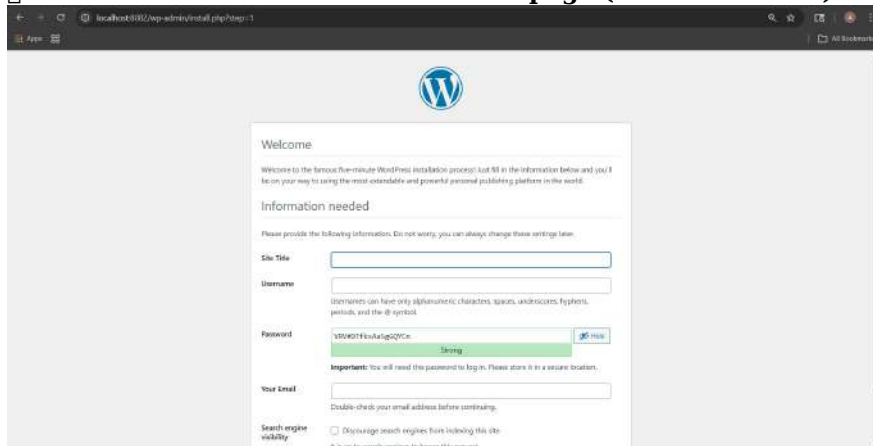
### Verify both containers running:

```
docker ps
```



**Access in browser:** <http://localhost:8082>

### 📄 Screenshot - WordPress installation page (via Docker Run):



```
docker-compose.yml M • Experiment6_Docker_Compo
lab > exp6 > docker-compose.yml
1 version: '3.8'
2
3 services:
4 mysql:
5 image: mysql:5.7
6 container_name: wordpress_db
7 environment:
8 MYSQL_ROOT_PASSWORD: secret
9 MYSQL_DATABASE: wordpress
10 volumes:
11 - mysql_data:/var/lib/mysql
12
13 wordpress:
14 image: wordpress:latest
15 container_name: wordpress_app
16 depends_on:
17 - mysql
18 ports:
19 - "8082:80"
20 environment:
21 WORDPRESS_DB_HOST: mysql
22 WORDPRESS_DB_PASSWORD: secret
23 WORDPRESS_DB_NAME: wordpress
24 volumes:
25 - wp_data:/var/www/html
26
27 volumes:
28 mysql_data:
29 wp_data:
```

*The WordPress installation page loads at <http://localhost:8082>, confirming both the MySQL and WordPress containers are running and communicating via the wp-net Docker network.*

### 59.3.2.2 Part B: Using Docker Compose (Structured Method)

#### Create docker-compose.yml:

```
version: '3.8'
services:
 mysql:
 image: mysql:5.7
 environment:
 MYSQL_ROOT_PASSWORD: secret
 MYSQL_DATABASE: wordpress
 volumes:
 - mysql_data:/var/lib/mysql

 wordpress:
 image: wordpress:latest
```

```
PS C:\Users\anshu> notepad docker-compose.yml
PS C:\Users\anshu> docker compose up -d
[+] up 1/1
✔ Image node:18-alpine Pulled 7.3s
time="2026-04-24T22:02:42+05:30" level=warning msg="Found orphan containers ([lab-nginx]) for this project. If you removed or renamed this service in yo
[+] up 2/2 file, you can run this command with the --remove-orphans flag to
✔ Image node:18-alpine Pulled 7.3s
✔ Container webapp Started 0.7s
PS C:\Users\anshu> |
```

```
docker compose down -v
```

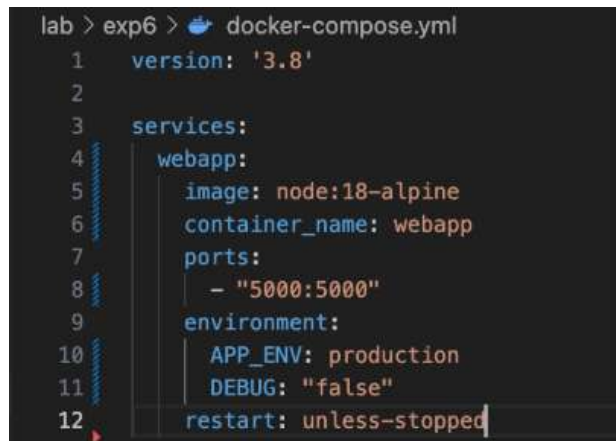
### 59.4.1 Task 3: Convert Docker Run to Docker Compose

---

### 59.4.1.1 Problem 1: Basic Web Application

#### Given Docker Run command:

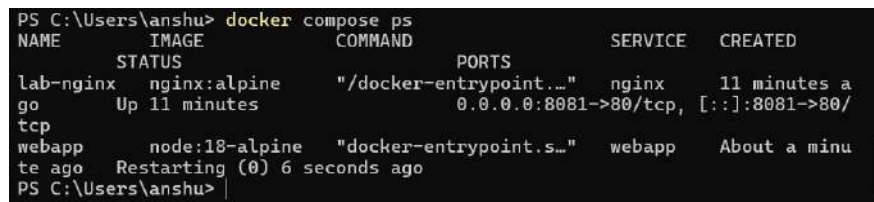
```
docker run -d \
 --name webapp \
 -p 5000:5000 \
 -e APP_ENV=production \
 -e DEBUG=false \
 --restart unless-stopped \
 node:18-alpine
```



```
lab > exp6 > docker-compose.yml
1 version: '3.8'
2
3 services:
4 webapp:
5 image: node:18-alpine
6 container_name: webapp
7 ports:
8 - "5000:5000"
9 environment:
10 APP_ENV: production
11 DEBUG: "false"
12 restart: unless-stopped
```

#### Equivalent docker-compose.yml:

```
version: '3.8'
services:
 webapp:
 image: node:18-alpine
 container_name: webapp
 ports:
 - "5000:5000"
 environment:
 APP_ENV: production
 DEBUG: "false"
 restart: unless-stopped
```



```
PS C:\Users\anshu> docker compose ps
NAME IMAGE COMMAND SERVICE CREATED
lab-nginx nginx:alpine "/docker-entrypoint..." nginx 11 minutes a
go Up 11 minutes 0.0.0.0:8081->80/tcp, [::]:8081->80/
tcp
webapp node:18-alpine "docker-entrypoint.s..." webapp About a minu
te ago Restarting (0) 6 seconds ago
PS C:\Users\anshu>
```

#### Run:

```
docker compose up -d
```

#### Verify:

```
docker compose ps
```

---

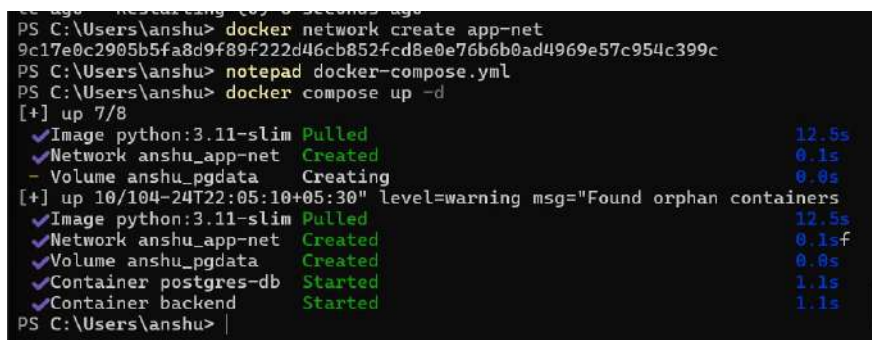
### 59.4.1.2 Problem 2: Volume + Network Configuration

### Given Docker Run commands:

```
docker network create app-net
```

```
docker run -d \
 --name postgres-db \
 --network app-net \
 -e POSTGRES_USER=admin \
 -e POSTGRES_PASSWORD=secret \
 -v pgdata:/var/lib/postgresql/data \
 postgres:15
```

```
docker run -d \
 --name backend \
 --network app-net \
 -p 8000:8000 \
 -e DB_HOST=postgres-db \
 -e DB_USER=admin \
 -e DB_PASS=secret \
 python:3.11-slim
```

A terminal window showing a series of Docker commands and their outputs. The commands are: 'docker network create app-net', 'notepad docker-compose.yml', and 'docker compose up -d'. The output for 'docker compose up -d' shows the progress of pulling images, creating networks and volumes, and starting containers. The output is color-coded: green for success, red for error, and yellow for warning. The output for 'docker compose up -d' is as follows:

```
PS C:\Users\anshu> docker network create app-net
9c17e0c2905b5fa8d9f89f222d46cb852fcd8e0e76b6b0ad4969e57c954c399c
PS C:\Users\anshu> notepad docker-compose.yml
PS C:\Users\anshu> docker compose up -d
[+] up 7/8
✓Image python:3.11-slim Pulled 12.5s
✓Network anshu_app-net Created 0.1s
- Volume anshu_pgdata Creating 0.0s
[+] up 10/104-24T22:05:10+05:30" level=warning msg="Found orphan containers
✓Image python:3.11-slim Pulled 12.5s
✓Network anshu_app-net Created 0.1sf
✓Volume anshu_pgdata Created 0.8s
✓Container postgres-db Started 1.1s
✓Container backend Started 1.1s
PS C:\Users\anshu> |
```

### Equivalent docker-compose.yml:

```
version: '3.8'
services:
 postgres-db:
 image: postgres:15
 container_name: postgres-db
 environment:
 POSTGRES_USER: admin
 POSTGRES_PASSWORD: secret
 volumes:
 - pgdata:/var/lib/postgresql/data
 networks:
 - app-net

 backend:
 image: python:3.11-slim
 container_name: backend
 ports:
 - "8000:8000"
 environment:
 DB_HOST: postgres-db
 DB_USER: admin
 DB_PASS: secret
 depends_on:
 - postgres-db
 networks:
 - app-net
```

volumes:  
pgdata:

networks:  
app-net:

```
lab > exp6 > docker-compose.yml
1 services:
2 mysql:
3 image: mysql:8.0
4 container_name: mysql
5 environment:
6 MYSQL_ROOT_PASSWORD: secret
7 MYSQL_DATABASE: wordpress
8 MYSQL_USER: wpuser
9 MYSQL_PASSWORD: wppass
10 volumes:
11 - mysql_data:/var/lib/mysql
12 networks:
13 - wordpress-network
14
15 wordpress:
16 image: wordpress:latest
17 container_name: wordpress
18 ports:
19 - "8080:80"
20 environment:
21 WORDPRESS_DB_HOST: mysql
22 WORDPRESS_DB_USER: wpuser
23 WORDPRESS_DB_PASSWORD: wppass
24 WORDPRESS_DB_NAME: wordpress
25 depends_on:
26 - mysql
27 volumes:
28 - wp_content:/var/www/html/wp-content
29 networks:
30 - wordpress-network
31
32 volumes:
33 mysql_data:
34 wp_content:
35
36 networks:
37 wordpress-network:
```

**Run:**

docker compose up -d

**Stop and remove:**

docker compose down -v

---

## 59.5 □ Comparison: Docker Run vs Docker Compose

| Feature                 | Docker Run         | Docker Compose     |
|-------------------------|--------------------|--------------------|
| Approach                | Imperative         | Declarative        |
| Configuration           | Command line flags | YAML file          |
| Multi-container support | Complex (manual)   | Easy (depends_on)  |
| Reusability             | Low                | High               |
| Version control         | Difficult          | Easy (commit YAML) |

|            |                    |                 |
|------------|--------------------|-----------------|
| Networking | Manual (--network) | Auto-created    |
| Volumes    | Manual (-v)        | Defined in YAML |
| Scaling    | Not supported      | --scale flag    |

---

## 59.6 Result

Successfully completed:

- Nginx container using Docker Run — verified at <http://localhost:8081>
  - Nginx container using Docker Compose — verified at <http://localhost:8081>
  - WordPress + MySQL using Docker Run — verified at <http://localhost:8082>
  - WordPress + MySQL using Docker Compose — verified at <http://localhost:8082>
  - Docker Run to Compose conversion (Problems 1 & 2)
  - Resource limits conversion (Task 4)
  - Custom Dockerfile with Compose (Task 5)
  - Multi-stage Dockerfile with Compose (Task 6)
- 

## 59.7 Viva Questions

**Q1. What is Docker Compose?** Docker Compose is a tool used to define and run multi-container Docker applications using a YAML configuration file (docker-compose.yml).

**Q2. What is the difference between Docker Run and Docker Compose?** Docker Run executes containers manually using CLI commands (imperative), while Docker Compose manages multiple containers using a declarative YAML configuration file.

**Q3. What does depends\_on do?** It ensures that one service starts before another. For example, WordPress waits for MySQL to start before launching.

**Q4. Why are Docker networks used?** Docker networks allow containers to communicate with each other using container names as hostnames, without exposing ports to the host machine.

**Q5. What is the difference between image: and build: in Compose?** image: pulls a prebuilt image from Docker Hub, while build: builds a custom image from a local Dockerfile.

**Q6. What does docker compose down -v do?** It stops and removes containers, networks, AND named volumes defined in the Compose file.

---

## 59.8 Conclusion

Docker Compose provides a structured and efficient way to manage multi-container applications compared to docker run. It simplifies deployment, improves maintainability, enables version control of



infrastructure, and allows easy scaling of services using a single declarative YAML file.

---

|                  |                                    |                   |
|------------------|------------------------------------|-------------------|
| *Experiment<br>6 | Containerization and DevOps<br>Lab | UPES<br>Dehradun* |
|------------------|------------------------------------|-------------------|

---

## **60 Containerization-and-Devops-Lab**

### **61 Lab Experiment 7: CI/CD Pipeline using Jenkins, GitHub and Docker Hub**

**Name:** Anshuman Mohapatra **Subject:** Containerization and DevOps

---

#### **61.1 1. Aim**

To design and implement a complete CI/CD pipeline using Jenkins, integrating source code from GitHub, and building & pushing Docker images to Docker Hub.

---

#### **61.2 2. Objectives**

- Understand CI/CD workflow using Jenkins (GUI-based tool)
  - Create a structured GitHub repository with application + Jenkinsfile
  - Build Docker images from source code
  - Securely store Docker Hub credentials in Jenkins
  - Automate build & push process using webhook triggers
  - Use same host (Docker) as Jenkins agent
- 

#### **61.3 3. Theory**

##### **61.3.1 What is Jenkins?**

Jenkins is a web-based GUI automation server used to:

- Build applications
- Test code
- Deploy software

It provides:

- Dashboard (browser-based UI)
- Plugin ecosystem (GitHub, Docker, etc.)
- Pipeline as Code using Jenkinsfile

### 61.3.2 What is CI/CD?

**Continuous Integration (CI):** Code is automatically built and tested after each commit.

**Continuous Deployment (CD):** Built artifacts (Docker images) are automatically delivered/deployed.

### 61.3.3 Workflow Overview

Developer → GitHub → Webhook → Jenkins → Build → Docker Hub

---

## 61.4 4. Prerequisites

- Docker & Docker Compose installed
  - GitHub account
  - Docker Hub account
  - Basic Linux command knowledge
- 

## 61.5 5. Part A: GitHub Repository Setup

### 61.5.1 5.1 Project Structure

```
my-app/
├── app.py
├── requirements.txt
├── Dockerfile
└── Jenkinsfile
```

### 61.5.2 5.2 Application Code — app.py

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
 return "Hello from CI/CD Pipeline!"

app.run(host="0.0.0.0", port=80)
```

### 61.5.3 5.3 requirements.txt

```
flask
```

### 61.5.4 5.4 Dockerfile

```
FROM python:3.10-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 80
CMD ["python", "app.py"]
```

### 61.5.5 5.5 Jenkinsfile

```
pipeline {
 agent any
 environment {
 IMAGE_NAME = "your-dockerhub-username/myapp"
 }
 stages {
 stage('Clone Source') {
 steps {
 git 'https://github.com/your-username/my-app.git'
 }
 }
 stage('Build Docker Image') {
 steps {
 sh 'docker build -t $IMAGE_NAME:latest .'
 }
 }
 stage('Login to Docker Hub') {
 steps {
 withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]) {
 sh 'echo $DOCKER_TOKEN | docker login -u your-dockerhub-username --password-stdin'
 }
 }
 }
 stage('Push to Docker Hub') {
 steps {
 sh 'docker push $IMAGE_NAME:latest'
 }
 }
 }
}
```

---

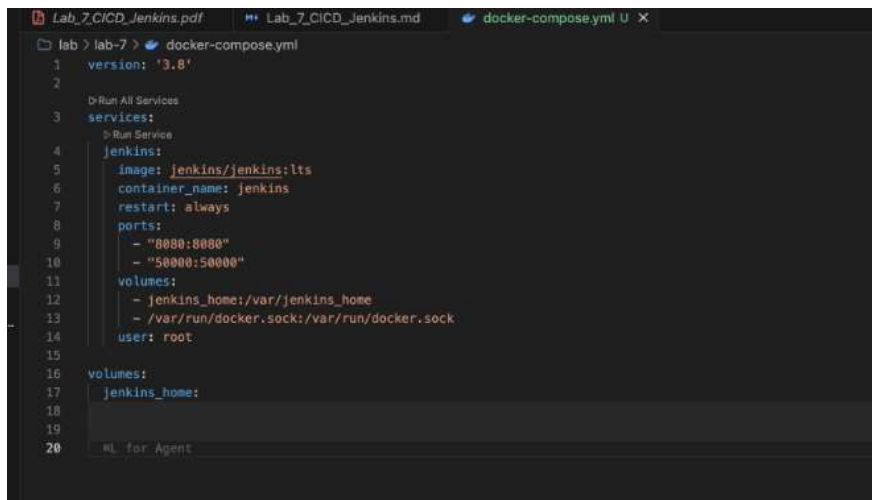
## 61.6 6. Part B: Jenkins Setup using Docker

### 61.6.1 6.1 docker-compose.yml

```
version: '3.8'

services:
 jenkins:
 image: jenkins/jenkins:lts
 container_name: jenkins
 restart: always
 ports:
 - "8080:8080"
 - "50000:50000"
 volumes:
 - jenkins_home:/var/jenkins_home
 - /var/run/docker.sock:/var/run/docker.sock
 user: root

volumes:
 jenkins_home:
```

A screenshot of a code editor with three tabs: 'Lab\_7\_CICD\_Jenkins.pdf', 'Lab\_7\_CICD\_Jenkins.md', and 'docker-compose.yml'. The 'docker-compose.yml' tab is active, showing a configuration for a Jenkins service. The file content is as follows:

```
1 version: '3.8'
2
3 > Run All Services
4 services:
5 > Run Service
6 jenkins:
7 image: jenkins/jenkins:lts
8 container_name: jenkins
9 restart: always
10 ports:
11 - "8080:8080"
12 - "50000:50000"
13 volumes:
14 - jenkins_home:/var/jenkins_home
15 - /var/run/docker.sock:/var/run/docker.sock
16 user: root
17
18 volumes:
19 jenkins_home:
20 #L for Agent
```

### 61.6.2 6.2 Start Jenkins

`docker-compose up -d`

Access Jenkins at: `http://localhost:8080`

### 61.6.3 6.3 Unlock Jenkins

`docker exec -it jenkins cat`  
`/var/jenkins_home/secrets/initialAdminPassword`

### 61.6.4 6.4 Initial Setup

- Install suggested plugins
- Create admin user (Anshuman\_M)

---

## 61.7 7. Part C: Jenkins Configuration

### 61.7.1 7.1 Add Docker Hub Credentials

Path: Manage Jenkins → Credentials → Add Credentials

- **Type:** Secret Text
- **ID:** dockerhub-token
- **Value:** Docker Hub Access Token

### 61.7.2 7.2 Create Pipeline Job

1. New Item → Pipeline
2. **Name:** ci-cd-pipeline
3. Configure:
  - Pipeline script from SCM
  - SCM: Git
  - Repo URL: your GitHub repo
  - Script Path: Jenkinsfile

## 61.8 8. Part D: GitHub Webhook Integration

### 61.8.1 8.1 Configure Webhook

In GitHub: Settings → Webhooks → Add Webhook

- **Payload URL:** `http://<your-server-ip>:8080/github-webhook/`
- **Events:** Push events

### 61.8.2 8.2 Exposing Jenkins via LocalTunnel

Since Jenkins runs locally, we used localtunnel to expose it publicly for GitHub webhook integration:

```
npm install -g localtunnel
npx localtunnel --port 8080
```

This generated a public URL (e.g., (`https://eight-ducks-open.loca.lt/`)) which was used as the webhook payload URL.

---

## 61.9 9. Part E: Execution Flow

| Stage                  | Action                                          |
|------------------------|-------------------------------------------------|
| <b>Code Push</b>       | Developer pushes code to GitHub                 |
| <b>Webhook Trigger</b> | GitHub sends event to Jenkins                   |
| <b>Clone</b>           | Jenkins pulls latest code from GitHub           |
| <b>Build</b>           | Docker builds image using Dockerfile            |
| <b>Auth</b>            | Jenkins logs into Docker Hub using stored token |
| <b>Push</b>            | Image pushed to Docker Hub                      |
| <b>Done</b>            | Docker image available globally                 |

---

## 61.10 10. Role of Same Host Agent

Jenkins runs inside Docker with the Docker socket mounted:

```
/var/run/docker.sock
```

This allows Jenkins to directly control the host's Docker daemon — building and pushing images without needing a separate agent node.

A custom **Permanent Agent** node (Anshuman Mohapatra node macos) was also configured in Jenkins under Manage Jenkins → Nodes → New Node to demonstrate multi-node agent setup.

---

## 61.11 11. Observations

- Jenkins GUI significantly simplifies CI/CD pipeline management

- GitHub acts as both source repository and pipeline definition store (via Jenkinsfile)
  - Docker ensures consistent, reproducible builds across environments
  - Webhook integration enables fully automated, event-driven pipelines
  - localtunnel provides a quick way to expose local Jenkins to the internet for webhook testing
  - The Docker socket mount allows Jenkins to control the host Docker engine directly
- 

## 61.12 12. Result

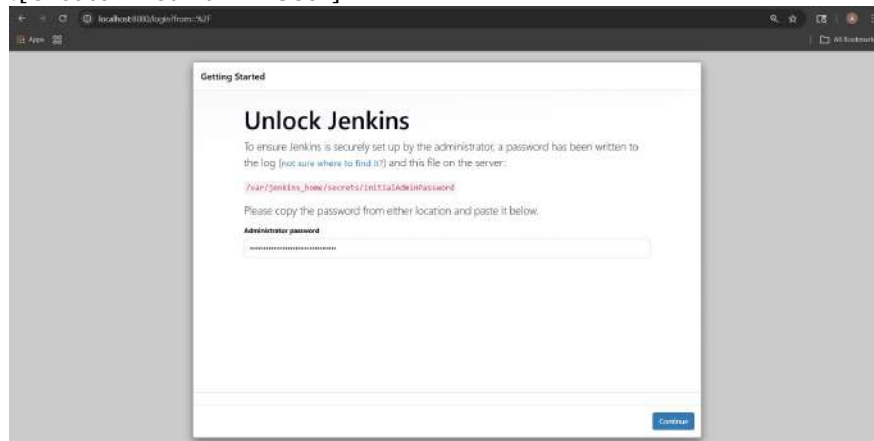
Successfully implemented a complete CI/CD pipeline where:

- Source code and pipeline definition are maintained in GitHub
  - Jenkins automatically detects code changes via GitHub webhook
  - Docker image is built on the host agent
  - Image is securely pushed to Docker Hub using stored credentials
- 

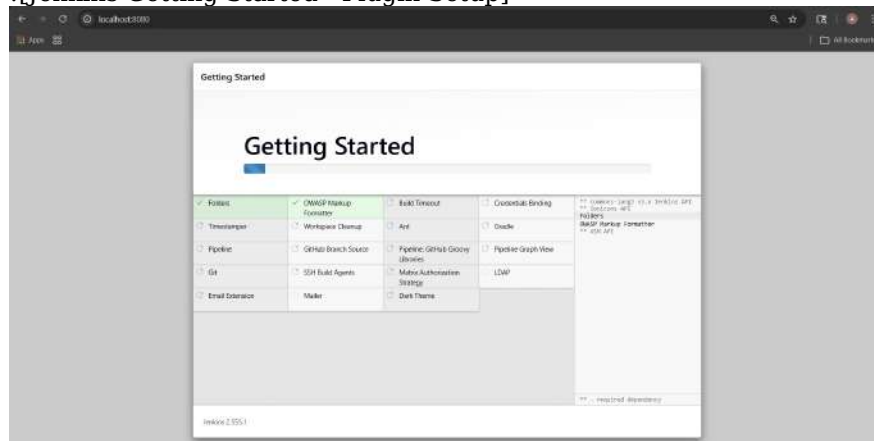
## 61.13 13. Screenshots

### 61.13.1 Screenshot 1 — Jenkins Plugin Installation (Getting Started)

![[Create First Admin User]]



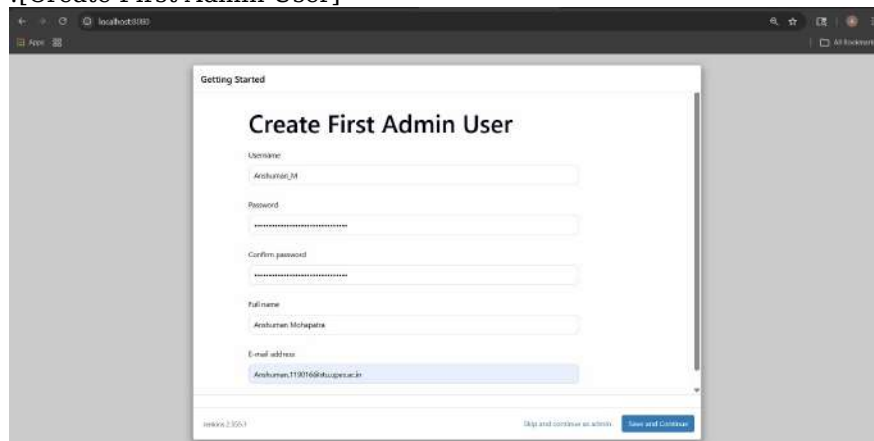
### !![Jenkins Getting Started - Plugin Setup]



Jenkins loading suggested plugins during the initial setup wizard at localhost:8080.

### 61.13.2 Screenshot 2 — Create First Admin User

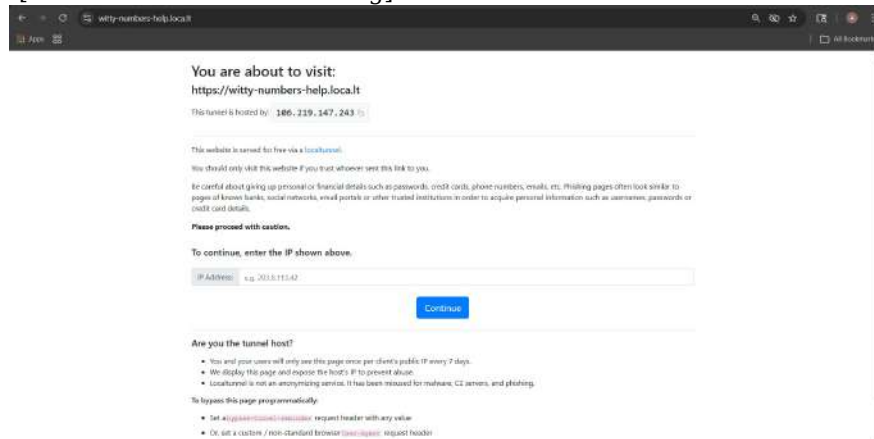
#### !![Create First Admin User]



Creating the admin user Anshuman Mohapatra during Jenkins initial configuration.

### 61.13.3 Screenshot 3 — LocalTunnel Warning Page

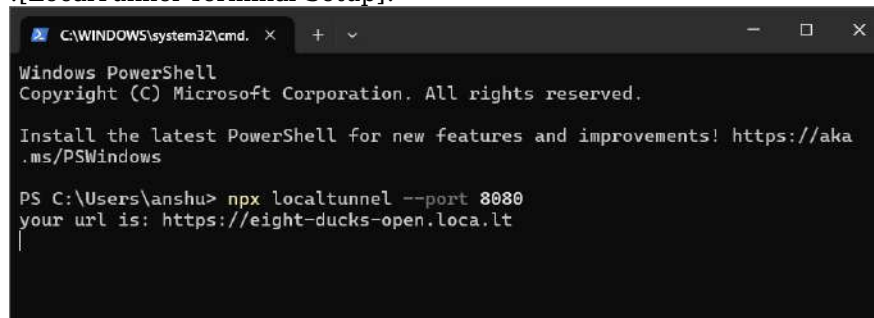
### ![LocalTunnel Tunnel Warning]



The localtunnel gateway page for (<https://eight-ducks-open.local.lt/>) — requiring IP confirmation before accessing Jenkins publicly.

## 61.13.4 Screenshot 4 — LocalTunnel Setup in Terminal

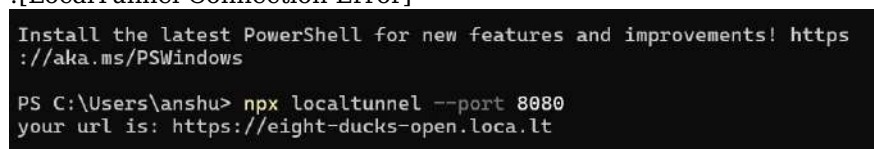
### ![LocalTunnel Terminal Setup]!



Terminal showing `npm install -g localtunnel, docker ps` confirming Jenkins container is running, and `npm localtunnel --port 8080` generating the public tunnel URL.

## 61.13.5 Screenshot 5 — LocalTunnel Connection Error (Firewall)

### ![LocalTunnel Connection Error]

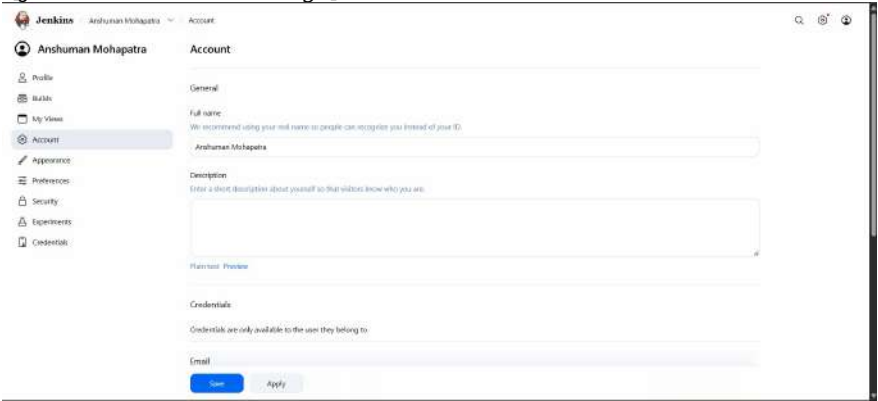


Localtunnel connection refused error due to firewall settings — resolved by switching to `npm localtunnel` instead of the global `lt` command.

## 61.13.6 Screenshot 6 — Jenkins Account Settings



![[Jenkins Account Settings]]



Jenkins user account page for Anshuman Mohapatra showing profile, credentials, and email settings via the localtunnel public URL.

61.13.7 Screenshot 7 — Adding New Node (Agent)

![[Jenkins New Node Creation]]



Creating a new Permanent Agent node named Anshuman Mohapatra in Jenkins under Manage Jenkins → Nodes → New Node.

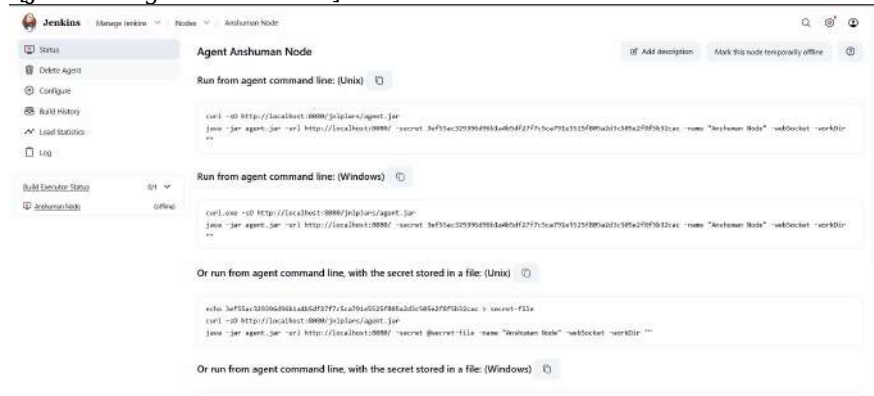
61.13.8 Screenshot 8 — Nodes Dashboard



Jenkins Nodes page showing the Built-In Node (Linux aarch64) and the newly added Anshuman Mohapatra node macos agent.

61.13.9 Screenshot 9 — Agent Connection Command

## ![[Jenkins Agent Command]



Jenkins displaying the JNLP agent connection commands (Unix/Windows) with a secret token to connect Anshuman Mohapatra node macos as a remote agent.

## 61.14 14. Viva Questions

**Q1. What is the role of Jenkinsfile?** It defines the CI/CD pipeline as code, stored in the repository alongside the application source.

**Q2. How does Jenkins integrate with GitHub?** Through GitHub Webhooks — GitHub sends a POST request to Jenkins on every push event, triggering the pipeline automatically.

**Q3. Why is Docker used in CI/CD?** Docker ensures consistent, reproducible builds across different environments by packaging the application and its dependencies into an image.

**Q4. What is a webhook?** A webhook is an HTTP callback that allows GitHub to notify Jenkins of events (like a push) in real time.

**Q5. Why store Docker Hub token in Jenkins credentials?** To avoid hardcoding secrets in the Jenkinsfile, which is stored in a public/shared repository. Jenkins credentials store is encrypted and secure.

**Q6. What is the benefit of using the same host as agent?** Jenkins can directly invoke Docker commands on the host, eliminating the need for a separate build agent and simplifying the setup.

## 61.15 15. Key Takeaways

- Jenkins is GUI-based but pipelines are fully code-driven via Jenkinsfile
- Always use the credentials store — **never hardcode secrets**
- Webhooks make CI/CD fully automatic and event-driven
- Docker socket mounting (/var/run/docker.sock) allows Jenkins to act as its own Docker agent
- localtunnel is a quick solution for exposing local services during development/testing

*Experiment completed successfully on April 1, 2026.*

## 62 Containerization-and-Devops-Lab

### 63 Experiment 9 — Ansible: Configuration Management and Automation

#### 63.1 Theory

##### 63.1.1 Problem Statement

Managing infrastructure manually across multiple servers leads to configuration drift, inconsistent environments, and time-consuming repetitive tasks. Scaling from one server to hundreds becomes nearly impossible with manual SSH-based administration.

##### 63.1.2 What is Ansible?

Ansible is an open-source automation tool for **configuration management**, **application deployment**, and **orchestration**. It follows an **agentless architecture** — using SSH for Linux and WinRM for Windows — and uses YAML-based **playbooks** to define automation tasks.

Ansible has become the standard choice among enterprise automation solutions because it is simple yet powerful, agentless, community-powered, predictable, and secure.

##### 63.1.3 How Ansible Solves the Problem

| Problem                               | Ansible Solution                                     |
|---------------------------------------|------------------------------------------------------|
| Agent installation on every server    | Agentless — uses SSH only                            |
| Running playbooks twice breaks things | Idempotency — same result every time                 |
| Imperative scripts hard to read       | Declarative YAML — describe desired state            |
| Waiting for changes to propagate      | Push-based — initiates from control node immediately |

#### 63.2 Key Concepts

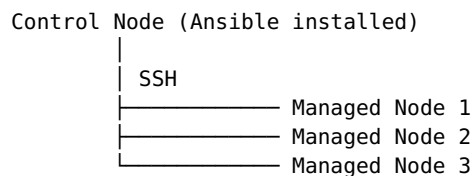
| Component            | Description                                             |
|----------------------|---------------------------------------------------------|
| <b>Control Node</b>  | Machine with Ansible installed — where you run commands |
| <b>Managed Nodes</b> | Target servers — no Ansible agent needed                |

|                  |                                                                   |
|------------------|-------------------------------------------------------------------|
| <b>Inventory</b> | File listing all managed nodes (EC2 instances, servers, etc.)     |
| <b>Playbooks</b> | YAML files containing a sequence of automation steps              |
| <b>Tasks</b>     | Individual actions in playbooks (e.g., installing a package)      |
| <b>Modules</b>   | Built-in functionality to perform tasks (e.g., apt, yum, service) |
| <b>Roles</b>     | Pre-defined reusable automation scripts                           |

---

### 63.2.1 How Ansible Works

Ansible connects from the **control node** to the **managed nodes** via SSH, sending commands and instructions. The units of code it executes are called **modules**. Each module is invoked by a **task**, and an ordered list of tasks forms a **playbook**. The managed machines are listed in an **inventory file** grouped into categories.



No extra agents required on managed nodes — just a terminal and a text editor to get started.

---

## 63.3 Part A — Hands-On Lab

### 63.3.1 Step 1: Install Ansible

**Via pip (recommended for macOS/Linux):**

```

pip install ansible
ansible --version

```

**Via apt (Ubuntu/Debian):**

```

sudo apt update -y
sudo apt install ansible -y
ansible --version

```

**Post-installation check:**

```

ansible localhost -m ping

```

**Expected output:**

```

localhost | SUCCESS => {
 "changed": false,
 "ping": "pong"
}

```

---

### 63.3.2 Step 2: Create SSH Key Pair

Ansible uses SSH key-based authentication to connect to managed nodes without passwords.

```
ssh-keygen -t rsa -b 4096
Accept all defaults – keys saved to ~/.ssh/id_rsa and
~/.ssh/id_rsa.pub
```

Copy keys to the current working directory (needed for building the Docker image):

```
cp ~/.ssh/id_rsa.pub .
cp ~/.ssh/id_rsa .
```

**Key placement explained:**

| File                       | Location                                | Purpose                                                        |
|----------------------------|-----------------------------------------|----------------------------------------------------------------|
| id_rsa<br>(Private Key)    | Control node only                       | Used to authenticate when connecting — <b>never share this</b> |
| id_rsa.pub<br>(Public Key) | Remote server<br>~/.ssh/authorized_keys | Grants access to anyone with the matching private key          |

### 63.3.3 Step 3: Create the Docker Image (Ubuntu SSH Server)

Create a Dockerfile that builds a custom Ubuntu image with OpenSSH pre-configured and our public key baked in:

```
FROM ubuntu

RUN apt update -y
RUN apt install -y python3 python3-pip openssh-server
RUN mkdir -p /var/run/ssh

Configure SSH
RUN mkdir -p /run/sshd && \
 echo 'root:password' | chpasswd && \
 sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config && \
 sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config && \
 sed -i 's/#PubkeyAuthentication yes/PubkeyAuthentication yes/' /etc/ssh/sshd_config

Create .ssh directory and set proper permissions
RUN mkdir -p /root/.ssh && \
 chmod 700 /root/.ssh

Copy SSH keys into the image
COPY id_rsa /root/.ssh/id_rsa
COPY id_rsa.pub /root/.ssh/authorized_keys

Set proper permissions
RUN chmod 600 /root/.ssh/id_rsa && \
 chmod 644 /root/.ssh/authorized_keys

Fix for SSH login
RUN sed -i 's@session\s*required\s*pam_loginuid.so@session optional'
```

```
pam_loginuid.so@g' /etc/pam.d/sshd
```

```
Expose SSH port
EXPOSE 22
```

```
Start SSH service
CMD ["/usr/sbin/sshd", "-D"]
```

Build the image:

```
docker build -t ubuntu-server .
```

---

### 63.3.4 Step 4: Launch 4 Server Containers

```
for i in {1..4}; do
 echo -e "\n Creating server${i}\n"
 docker run -d --rm -p 220${i}:22 --name server${i} ubuntu-server
 echo -e "IP of server${i} is $(docker inspect -f
'{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}'
server${i})"
done
```

#### Expected output:

```
Creating server1
IP of server1 is 172.17.0.2
```

```
Creating server2
IP of server2 is 172.17.0.3
```

```
Creating server3
IP of server3 is 172.17.0.4
```

```
Creating server4
IP of server4 is 172.17.0.5
```

---

### 63.3.5 Step 5: Create Ansible Inventory

This script auto-generates inventory.ini with the real container IPs:

```
echo "[servers]" > inventory.ini
for i in {1..4}; do
 docker inspect -f '{{range.NetworkSettings.Networks}}
{{.IPAddress}}{{end}}' server${i} >> inventory.ini
done
```

```
cat << EOF >> inventory.ini
```

```
[servers:vars]
ansible_user=root
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
EOF
```

Review the generated file:

```
cat inventory.ini
```

#### Expected inventory.ini content:

```

Dockerfile > ...
1 FROM ubuntu
2
3 RUN apt update -y
4 RUN apt install -y python3 python3-pip openssh-server sudo
5
6 RUN mkdir -p /var/run/sshd
7
8 # Enable root login + SSH config
9 RUN echo 'root:password' | chpasswd && \
10 sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config && \
11 sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config && \
12 sed -i 's/#PubkeyAuthentication yes/PubkeyAuthentication yes/' /etc/ssh/sshd_config
13
14 # SSH folder setup
15 RUN mkdir -p /root/.ssh
16 RUN chmod 700 /root/.ssh
17 # ML for Agent
18 # Copy keys (IMPORTANT)
19 COPY id_rsa.pub /root/.ssh/authorized_keys
20
21 RUN chmod 600 /root/.ssh/authorized_keys
22
23 EXPOSE 22
24
25 CMD ["usr/sbin/sshd", "-D"]
26

```

### 63.3.6 Step 6: Test Connectivity

Manual SSH test to confirm keys work:

```
ssh -i ~/.ssh/id_rsa root@172.17.0.2
```

Ansible ping test across all servers:

```
ansible all -i inventory.ini -m ping
```

#### Expected output:

```

172.17.0.2 | SUCCESS => {
 "changed": false,
 "ping": "pong"
}
172.17.0.3 | SUCCESS => {
 "changed": false,
 "ping": "pong"
}
172.17.0.4 | SUCCESS => {
 "changed": false,
 "ping": "pong"
}
172.17.0.5 | SUCCESS => {
 "changed": false,
 "ping": "pong"
}

```

```

inventory.ini
1 [servers]
2 server1 ansible_host=localhost ansible_port=2201
3 server2 ansible_host=localhost ansible_port=2202
4 server3 ansible_host=localhost ansible_port=2203
5 server4 ansible_host=localhost ansible_port=2204
6
7 [servers:vars]
8 ansible_user=root
9 ansible_ssh_private_key_file=~/.ssh/id_rsa
10 ansible_python_interpreter=/usr/bin/python3
11 ansible_ssh_common_args='-o StrictHostKeyChecking=no'
12

```

For verbose output (useful for debugging):

### 63.3.7 Step 7: Create and Run Playbook (update.yml)

```

- name: Update and configure servers
 hosts: all
 become: yes
 tasks:

 - name: Update apt packages
 apt:
 update_cache: yes
 upgrade: dist

 - name: Install required packages
 apt:
 name: ["vim", "htop", "wget"]
 state: present

 - name: Create test file
 copy:
 dest: /root/ansible_test.txt
 content: "Configured by Ansible on {{ inventory_hostname }}"

```



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\lanhuo> wsl
anishu@Pavilion:~$ sudo apt update
anishu@Pavilion:~$ sudo apt install ansible
[sudo] password for anishu:
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
68 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
 ansible-core ieee-data python3-argcomplete python3-dnspython
 python3-jmespath python3-kerberos python3-libcloud python3-lockfile
 python3-netaddr python3-ntlm-auth python3-packaging python3-passlib
 python3-requests-ntlm python3-resolveip python3-selinux
 python3-simplesync python3-winnn python3-yaml
Suggested packages:
 cifs-utils python3-trio python3-aiomqtt python3-b2 python3-httplib2
 python3-httplib3 python-lockfile-doc python3-netaddr-docs
The following NEW packages will be installed:
 ansible ansible-core ieee-data python3-argcomplete python3-dnspython
 python3-jmespath python3-kerberos python3-libcloud python3-lockfile
 python3-netaddr python3-ntlm-auth python3-packaging python3-passlib
 python3-requests-ntlm python3-resolveip python3-selinux
 python3-simplesync python3-winnn python3-yaml
0 upgraded, 19 newly installed, 0 to remove and 68 not upgraded.
Need to get 22.9 MB of archives.
After this operation, 228 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/main amd64 python3-packaging all 24.0-1 [41.1 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble/universe amd64 python3-resolveip all 1.0.1-1 [25.7 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble/main amd64 python3-dnspython all 2.6.1-1ubuntu1 [161 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble/main amd64 python3-yaml all 6.0.1-1 [139 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble/main amd64 python3-argcomplete all 3.1.4-1 [24.8 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble/main amd64 python3-jmespath all 1.0.2-1 [22.1 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble/main amd64 python3-lockfile all 0.12.2-1 [12.1 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble/main amd64 python3-ntlm-auth all 1.5.0-1 [24.8 kB]
Get:9 http://archive.ubuntu.com/ubuntu noble/main amd64 python3-passlib all 1.7.4-1 [24.8 kB]
Get:10 http://archive.ubuntu.com/ubuntu noble/main amd64 python3-selinux all 3.10-1 [24.8 kB]
Get:11 http://archive.ubuntu.com/ubuntu noble/main amd64 python3-simplesync all 0.1.0-1 [12.1 kB]
Get:12 http://archive.ubuntu.com/ubuntu noble/main amd64 python3-winnn all 0.1.0-1 [12.1 kB]
Get:13 http://archive.ubuntu.com/ubuntu noble/main amd64 python3-kerberos all 1.3.0-1 [24.8 kB]
Get:14 http://archive.ubuntu.com/ubuntu noble/main amd64 python3-libcloud all 3.14.0-1 [24.8 kB]
Get:15 http://archive.ubuntu.com/ubuntu noble/main amd64 python3-ieee-data all 2024.10.1-1 [24.8 kB]
Get:16 http://archive.ubuntu.com/ubuntu noble/main amd64 python3-netaddr all 0.8.1-1 [24.8 kB]
Get:17 http://archive.ubuntu.com/ubuntu noble/main amd64 python3-ansible-core all 2.16.0-1 [24.8 kB]
Get:18 http://archive.ubuntu.com/ubuntu noble/main amd64 python3-ansible all 2.16.0-1 [24.8 kB]
Get:19 http://archive.ubuntu.com/ubuntu noble/main amd64 python3-ansible-core all 2.16.0-1 [24.8 kB]
Fetched 22.9 MB in 1s (22.9 MB/s)
Selecting previously unselected package python3-packaging.
(Reading database ... 123456 files and directories currently installed.)
Preparing to unpack .../python3-packaging_24.0-1_amd64.deb ...
Unpacking python3-packaging (24.0-1) ...
Selecting previously unselected package python3-resolveip.
Preparing to unpack .../python3-resolveip_1.0.1-1_amd64.deb ...
Unpacking python3-resolveip (1.0.1-1) ...
Selecting previously unselected package python3-dnspython.
Preparing to unpack .../python3-dnspython_2.6.1-1ubuntu1_amd64.deb ...
Unpacking python3-dnspython (2.6.1-1ubuntu1) ...
Selecting previously unselected package python3-yaml.
Preparing to unpack .../python3-yaml_6.0.1-1_amd64.deb ...
Unpacking python3-yaml (6.0.1-1) ...
Selecting previously unselected package python3-argcomplete.
Preparing to unpack .../python3-argcomplete_3.1.4-1_amd64.deb ...
Unpacking python3-argcomplete (3.1.4-1) ...
Selecting previously unselected package python3-jmespath.
Preparing to unpack .../python3-jmespath_1.0.2-1_amd64.deb ...
Unpacking python3-jmespath (1.0.2-1) ...
Selecting previously unselected package python3-lockfile.
Preparing to unpack .../python3-lockfile_0.12.2-1_amd64.deb ...
Unpacking python3-lockfile (0.12.2-1) ...
Selecting previously unselected package python3-ntlm-auth.
Preparing to unpack .../python3-ntlm-auth_1.5.0-1_amd64.deb ...
Unpacking python3-ntlm-auth (1.5.0-1) ...
Selecting previously unselected package python3-passlib.
Preparing to unpack .../python3-passlib_1.7.4-1_amd64.deb ...
Unpacking python3-passlib (1.7.4-1) ...
Selecting previously unselected package python3-selinux.
Preparing to unpack .../python3-selinux_3.10-1_amd64.deb ...
Unpacking python3-selinux (3.10-1) ...
Selecting previously unselected package python3-simplesync.
Preparing to unpack .../python3-simplesync_0.1.0-1_amd64.deb ...
Unpacking python3-simplesync (0.1.0-1) ...
Selecting previously unselected package python3-winnn.
Preparing to unpack .../python3-winnn_0.1.0-1_amd64.deb ...
Unpacking python3-winnn (0.1.0-1) ...
Selecting previously unselected package python3-kerberos.
Preparing to unpack .../python3-kerberos_1.3.0-1_amd64.deb ...
Unpacking python3-kerberos (1.3.0-1) ...
Selecting previously unselected package python3-libcloud.
Preparing to unpack .../python3-libcloud_3.14.0-1_amd64.deb ...
Unpacking python3-libcloud (3.14.0-1) ...
Selecting previously unselected package python3-ieee-data.
Preparing to unpack .../python3-ieee-data_2024.10.1-1_amd64.deb ...
Unpacking python3-ieee-data (2024.10.1-1) ...
Selecting previously unselected package python3-netaddr.
Preparing to unpack .../python3-netaddr_0.8.1-1_amd64.deb ...
Unpacking python3-netaddr (0.8.1-1) ...
Selecting previously unselected package python3-ansible-core.
Preparing to unpack .../python3-ansible-core_2.16.0-1_amd64.deb ...
Unpacking python3-ansible-core (2.16.0-1) ...
Selecting previously unselected package python3-ansible.
Preparing to unpack .../python3-ansible_2.16.0-1_amd64.deb ...
Unpacking python3-ansible (2.16.0-1) ...
Selecting previously unselected package python3-ansible-core.
Preparing to unpack .../python3-ansible-core_2.16.0-1_amd64.deb ...
Unpacking python3-ansible-core (2.16.0-1) ...
Setting up python3-packaging (24.0-1) ...
Setting up python3-resolveip (1.0.1-1) ...
Setting up python3-dnspython (2.6.1-1ubuntu1) ...
Setting up python3-yaml (6.0.1-1) ...
Setting up python3-argcomplete (3.1.4-1) ...
Setting up python3-jmespath (1.0.2-1) ...
Setting up python3-lockfile (0.12.2-1) ...
Setting up python3-ntlm-auth (1.5.0-1) ...
Setting up python3-passlib (1.7.4-1) ...
Setting up python3-selinux (3.10-1) ...
Setting up python3-simplesync (0.1.0-1) ...
Setting up python3-winnn (0.1.0-1) ...
Setting up python3-kerberos (1.3.0-1) ...
Setting up python3-libcloud (3.14.0-1) ...
Setting up python3-ieee-data (2024.10.1-1) ...
Setting up python3-netaddr (0.8.1-1) ...
Setting up python3-ansible-core (2.16.0-1) ...
Setting up python3-ansible (2.16.0-1) ...
Setting up python3-ansible-core (2.16.0-1) ...

anishu@Pavilion:~$ ssh-keygen -t rsa -b 2048
Generating public/private rsa key pair.
Enter file in which to save the key (/home/anishu/.ssh/id_rsa):
Created directory /home/anishu/.ssh.
Enter passphrase (empty for no passphrase):
Your identification has been saved in /home/anishu/.ssh/id_rsa
Your public key has been saved in /home/anishu/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:707e9e971b011c1f05f0590182328811f0a8 anishu@Pavilion
The key's randomart image is:
[RSA 2048]
+-----+
| .. . + |
| oo . o |
| .oo . o E |
| .. B o + B . |
| . + o . . |
| . + o . . |
| o o . o + + |
| o o . o + + |
+-----+
[SHA256]

anishu@Pavilion:~$ cd /home/anishu/.ssh
anishu@Pavilion:~/ssh$ cd ansible-lab

cp ~/ssh/id_rsa.pub
anishu@Pavilion:~/ssh$ cd /home/anishu/ansible-lab
anishu@Pavilion:~/ansible-lab$ nano Dockerfile
anishu@Pavilion:~/ansible-lab$ docker build -t ubuntu-ssh .
[*] Building 72.6s (12/12) FINISHED
Dockerfile
FROM ubuntu:22.04
COPY ssh/id_rsa.pub /home/ubuntu/.ssh/authorized_keys
RUN apt-get update && apt-get install -y python3 python3-pip python3-ansible
anishu@Pavilion:~/ansible-lab$ docker run -d --name server1 --p 2201:22 ubuntu-ssh
anishu@Pavilion:~/ansible-lab$ docker run -d --name server2 --p 2202:22 ubuntu-ssh
anishu@Pavilion:~/ansible-lab$ docker run -d --name server3 --p 2203:22 ubuntu-ssh
anishu@Pavilion:~/ansible-lab$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
server1 ubuntu-ssh python3 ansible-playbo... 12m ago Up 2201:22->22
server2 ubuntu-ssh python3 ansible-playbo... 12m ago Up 2202:22->22
server3 ubuntu-ssh python3 ansible-playbo... 12m ago Up 2203:22->22
anishu@Pavilion:~/ansible-lab$ cat inventory.ini
[server1]
ansible_host=172.17.0.2
ansible_user=root
ansible_port=2201
[server2]
ansible_host=172.17.0.3
ansible_user=root
ansible_port=2202
[server3]
ansible_host=172.17.0.4
ansible_user=root
ansible_port=2203
[all:vars]
ansible_port=22
```

Run the playbook:

ansible-playbook -i inventory.ini update.yml

## Expected output:

PLAY [Update and configure servers]

\*\*\*\*\*

TASK [Gathering Facts]

\*\*\*\*\*

ok: [172.17.0.2]

```
ok: [172.17.0.3]
ok: [172.17.0.4]
ok: [172.17.0.5]
```

```
TASK [Update apt packages]
```

```

```

```
changed: [172.17.0.2]
changed: [172.17.0.3]
...
```

```
TASK [Install required packages]
```

```

```

```
changed: [172.17.0.2]
...
```

```
TASK [Create test file]
```

```

```

```
changed: [172.17.0.2]
...
```

```
PLAY RECAP
```

```

```

```
172.17.0.2 : ok=4 changed=3 unreachable=0 failed=0
172.17.0.3 : ok=4 changed=3 unreachable=0 failed=0
172.17.0.4 : ok=4 changed=3 unreachable=0 failed=0
172.17.0.5 : ok=4 changed=3 unreachable=0 failed=0
```

◀ ▶

```
Y update.yml
1 ---
2 - name: Update and configure servers
3 hosts: all
4 become: yes
5
6 tasks:
7 - name: Update apt packages
8 apt:
9 update_cache: yes
10 upgrade: dist
11
12 - name: Install required packages
13 apt:
14 name: ["vim", "htop", "wget"]
15 state: present
16
17 - name: Create test file
18 copy:
19 dest: /root/ansible_test.txt
20 content: "Configured by Ansible on {{ inventory_hostname }}"
21 *L for Agent
```

### 63.3.8 Step 8: Create Advanced Playbook (playbook1.yml)

```

- name: Configure multiple servers
```

```

hosts: servers
become: yes
tasks:

 - name: Update apt package index
 apt:
 update_cache: yes

 - name: Install Python 3 (latest available)
 apt:
 name: python3
 state: latest

 - name: Create test file with content
 copy:
 dest: /root/test_file.txt
 content: |
 This is a test file created by Ansible
 Server name: {{ inventory_hostname }}
 Current date: {{ ansible_date_time.date }}

 - name: Display system information
 command: uname -a
 register: uname_output

 - name: Show disk space
 command: df -h
 register: disk_space

 - name: Print results
 debug:
 msg:
 - "System info: {{ uname_output.stdout }}"
 - "Disk space: {{ disk_space.stdout_lines }}"

```

Run it:

```
ansible-playbook -i inventory.ini playbook1.yml
```

---

### 63.3.9 Step 9: Verify Changes

Using Ansible to read the created file across all servers:

```
ansible all -i inventory.ini -m command -a "cat /root/ansible_test.txt"
```



Using Docker exec directly:

```
for i in {1..4}; do
 docker exec server${i} cat /root/ansible_test.txt
done
```

```

PLAY RECAP *****
server1 : ok=1 changed=3 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
server2 : ok=1 changed=3 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
server3 : ok=1 changed=3 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0

```

**Expected output on each server:**

```

Configured by Ansible on 172.17.0.2
Configured by Ansible on 172.17.0.3
Configured by Ansible on 172.17.0.4

```

### 63.3.10 Step 10: Cleanup

Stop all server containers:

```
for i in {1..4}; do
 docker rm -f server${i}
done
```

---

## 63.4 Complete Workflow Summary

1. Setup SSH keys  
↓
  2. Build ubuntu-server Docker image  
↓
  3. Launch 4 containers (server1-server4)  
↓
  4. Generate inventory.ini with container IPs  
↓
  5. Test connectivity (ansible -m ping)  
↓
  6. Run playbook (ansible-playbook)  
↓
  7. Verify changes  
↓
  8. Cleanup (docker rm -f)
- 

## 63.5 Key Takeaways

1. **Agentless** — Ansible only needs SSH on managed nodes, no extra software to install
  2. **Idempotent** — running the same playbook twice produces the same result, no unintended side effects
  3. **Declarative** — you describe *what* you want (nginx installed, file present), not *how* to do it step by step
  4. **Inventory** — the single source of truth for all managed nodes, supports groups and variables
  5. **Modules** — 3000+ built-in modules cover everything from apt/yum to AWS/Azure/GCP resources
  6. **register + debug** — capture command output and print it back, useful for auditing and troubleshooting
  7. **Playbooks scale** — the same playbook that ran on 4 Docker containers runs identically on 400 EC2 instances
- 

## 63.6 Screenshots

| All screenshots are stored in the screenshots/ folder.

## 63.7 References

- [Ansible Official Documentation](#)
- [Ansible Tutorial — Spacelift](#)
- [Ansible Official Website](#)
- [Ansible Tower GUI](#)
- [Ansible Tower Tutorial — GeeksforGeeks](#)

---

|                  |                                    |                   |
|------------------|------------------------------------|-------------------|
| *Experiment<br>9 | Containerization and DevOps<br>Lab | UPES<br>Dehradun* |
|------------------|------------------------------------|-------------------|

---

## **64 Containerization-and-Devops-Lab**

### **65 Experiment 10: SonarQube — Static Code Analysis**

**Name:** Anshuman Mohapatra **Roll No:** R2142230609 **Course:** Containerization and DevOps

---

#### **65.1 Objective**

Perform static code analysis on a Java application using SonarQube to automatically detect bugs, security vulnerabilities, and code smells — and understand how it integrates into a CI/CD pipeline.

---

#### **65.2 Theory**

##### **65.2.1 Problem Statement**

Code bugs and security issues are often found too late — during testing or even after deployment. Manual code reviews are slow, inconsistent, and don't scale as teams grow.

##### **65.2.2 What is SonarQube?**

SonarQube is an open-source platform that automatically scans source code for bugs, security vulnerabilities, and maintainability issues — **without running the code**. This is called **static analysis**.

##### **65.2.3 Key Terms**

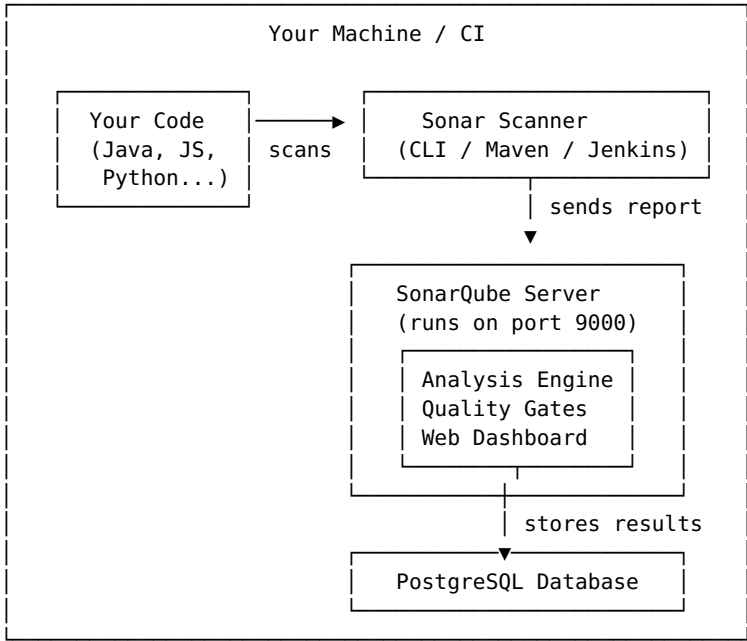
---

| Term                 | Meaning                                                   |
|----------------------|-----------------------------------------------------------|
| <b>Quality Gate</b>  | A set of rules; code must pass before deployment          |
| <b>Bug</b>           | Code that will likely break or behave incorrectly         |
| <b>Vulnerability</b> | A security weakness in the code                           |
| <b>Code Smell</b>    | Code that works but is poorly written or hard to maintain |

|                       |                                         |
|-----------------------|-----------------------------------------|
| <b>Technical Debt</b> | Estimated time to fix all issues        |
| <b>Coverage</b>       | Percentage of code tested by unit tests |
| <b>Duplication</b>    | Repeated code blocks (copy-paste)       |

---

### 65.2.4 Lab Architecture



## 65.3 Prerequisites

- Docker Desktop installed and running
- Maven (mvn) installed (or use the Docker-based scanner)
- Terminal / command line

Check Docker is running:

```
docker --version
docker-compose --version
```

---

## 65.4 Step 1: Start the SonarQube Server

Navigate to the experiment folder and start all containers:

```
cd ~/Desktop/exp10
docker-compose up -d
```

```
PS C:\Users\anshu> mkdir sonar-lab

Directory: C:\Users\anshu

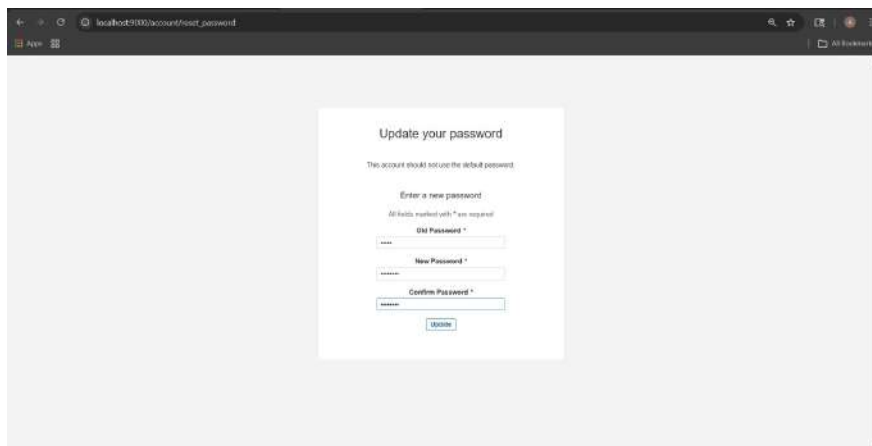
Mode LastWriteTime Length Name
---- -
d----- 4/26/2026 6:21 PM sonar-lab

PS C:\Users\anshu> cd sonar-lab
PS C:\Users\anshu\sonar-lab> notepad docker-compose.yml
PS C:\Users\anshu\sonar-lab> docker-compose up --
time="2026-04-26T18:21:04+05:30" level=warning msg="C:\Users\anshu\sonar-lab\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored
please remove it to avoid potential confusion"
[+] up 347ms
✓Image sonarqube/lts-community Pulled (20.4s)
✓Image postgres:13 Created 0.1s
✓Network sonar-lab_sonarqube-lab Created 0.1s
✓Volume sonar-lab_sonar-data Created 0.0s
✓Volume sonar-lab_sonar-extensions Created 0.0s
✓Volume sonar-lab_sonar-db-data Created 0.0s
✓Container sonar-db Started 1.1s
✓Container sonarqube Started 1.1s
PS C:\Users\anshu\sonar-lab>

Windows PowerShell
PS C:\Users\anshu> docker-compose logs -f sonarqube
time="2026-04-26T18:21:04+05:30" level=warning msg="C:\Users\anshu\sonar-lab\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored
please remove it to avoid potential confusion"
sonarqube | 2026.04.26 12:53:56 INFO app[[{"o.s.a.AppFileSystem} Cleaning up creating temp directory /opt/sonarqube/temp
sonarqube | 2026.04.26 12:53:56 INFO app[[{"o.s.a.es.Elasticsearch} Elasticsearch listening on [HTTP: 127.0.0.1:9001, TCP: 127.0.0.1:35837]
sonarqube | 2026.04.26 12:53:56 INFO app[[{"o.s.a.ProcessLauncherImpl} Launch process[ELASTICSEARCH] from [/opt/sonarqube/elasticsearch/bin/elasticsearch
sonarqube | 2026.04.26 12:53:56 INFO app[[{"o.s.a.SchedulerImpl} Waiting for Elasticsearch to be up and running
sonarqube | 2026.04.26 12:54:00 INFO es[[{"e.s.n.Node} version[7.17.15], pid[26], build[default/tar/8b8ecf947833546089cd223d4f1115f10be73ba/2023-11-10T22:
82:06-207390316], OS[Linux/6.0.07.2-microsoft-standard-WSL2/amd64], JVM[Oracle Adoptium/OpenJDK 64-Bit Server VM/17.0.16/17.0.16+0]
sonarqube | 2026.04.26 12:54:00 INFO es[[{"e.s.n.Node} JVM home [/opt/java/openjdk]
sonarqube | 2026.04.26 12:54:00 INFO es[[{"e.s.n.Node} JVM arguments [-XX:UseG1GC, -Djava.io.tmpdir=/opt/sonarqube/temp, -XX:ErrorFile=/opt/sonarqube/logs
/ps.err.%p.log, -Des.networkaddress.cache.ttl=50, -Des.networkaddress.cache.negative.ttl=50, -XX:+AlwaysPreTouch, -Xss1m, -Djava.awt.headless=true, -D
file.encoding=utf-8, -Djna.nodispatch, -Djna.tmpdir=/opt/sonarqube/temp, -XX:-OmitStackTraceInFastThrow, -Dio.netty.noUnsafe=true, -Dio.netty.noKeySetOptimi
zation=true, -Dio.netty.recycler.maxCapacityPerThread=8, -Dio.netty allocator.numDirectArena=8, -Dlog4j.shutdownHookEnabled=false, -Dlog4j.disable.jmx=true
, -Dlog4j.disable.jmx=true, -Djava.locale.providers=CF,CLIBC,POSIX,UNIX, -Des.enforce.bootstrap.check=true, -Xmx512m, -Xms512m, -XX:Max
DirectMemorySize=256m, -XX:HeapDumpOnOutOfMemoryError, -Des.path.home=/opt/sonarqube/elasticsearch, -Des.path.conf=/opt/sonarqube/temp/conf/es, -Des.distr
ibution.flavor=default, -Des.distribution.type=tar, -Des.bundled.jdk=false]
sonarqube | 2026.04.26 12:54:01 INFO es[[{"e.s.p.PluginsService} loaded module [analysis-common]
sonarqube | 2026.04.26 12:54:01 INFO es[[{"e.s.p.PluginsService} loaded module [lang-painless]
sonarqube | 2026.04.26 12:54:01 INFO es[[{"e.s.p.PluginsService} loaded module [parent-join]
sonarqube | 2026.04.26 12:54:01 INFO es[[{"e.s.p.PluginsService} loaded module [reindex]
sonarqube | 2026.04.26 12:54:01 INFO es[[{"e.s.p.PluginsService} loaded module [transport-netty4]
sonarqube | 2026.04.26 12:54:01 INFO es[[{"e.s.p.PluginsService} no plugins loaded
sonarqube | 2026.04.26 12:54:01 INFO es[[{"e.s.e.NodeEnvironment} using [1] data paths, mounts [/opt/sonarqube/data (/dev/sde)], net usable_space [945.4g
b], net total_space [1006.8gb], types [ext4]
sonarqube | 2026.04.26 12:54:01 INFO es[[{"e.s.e.NodeEnvironment} heap size [512mb], compressed ordinary object pointers [true]
sonarqube | 2026.04.26 12:54:01 INFO es[[{"e.s.e.Node} made name [sonarqube], node ID [PSVy2ba5-eiouVv8If4Q], cluster name [sonarqube], roles [data_froze
n_master, remote_cluster_client, data, data_content, data_hot, data_warm, data_cold, ingest]
sonarqube | 2026.04.26 12:54:06 INFO es[[{"e.s.e.NodeEnvironment} creating NettyAllocator with the following configs: [named_pooled, suggested_max_allocati
on_size=256mb, factories=[es.unsafe.new_unpooled_allocator=true, gipc.enabled=true, gipc.pool_size=1mb, heap_size=512mb]]
sonarqube | 2026.04.26 12:54:06 INFO es[[{"e.s.e.R.RecoverySettings} using rate limit [40mb] with [default=4mb, read=0b, write=0b, max=0b]
sonarqube | 2026.04.26 12:54:06 INFO es[[{"e.s.e.DiscoveryModule} using discovery type [sm] and seed hosts providers [settings]
sonarqube | 2026.04.26 12:54:06 INFO es[[{"e.s.e.DanglingIndicesState} gateway auto.import.dangling_indices is disabled, dangling indices will not be autom
atically detected or imported and must be managed manually
sonarqube | 2026.04.26 12:54:06 INFO es[[{"e.s.e.Node} initialized
sonarqube | 2026.04.26 12:54:06 INFO es[[{"e.s.e.Node} starting ...
sonarqube | 2026.04.26 12:54:06 INFO es[[{"e.s.t.TransportService} publish_address [127.0.0.1:35837], bound_addresses [127.0.0.1:35837]
sonarqube | 2026.04.26 12:54:07 INFO es[[{"e.s.b.BootstrapChecks} explicitly enforcing bootstrap checks
sonarqube | 2026.04.26 12:54:07 INFO es[[{"e.s.c.Coordinator} setting initial configuration to VotingConfiguration[PSVy2ba5-eiouVv8If4Q]
sonarqube | 2026.04.26 12:54:07 INFO es[[{"e.s.c.s.MasterService} elected-as-master ([1] nodes joined)[{sonarqube}[PSVy2ba5-eiouVv8If4Q][njq4ipR85-HTD88
```

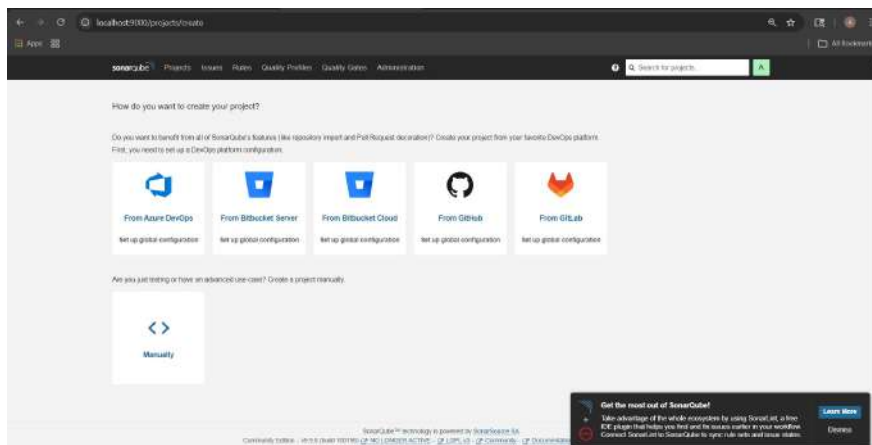
Watch logs until you see **“SonarQube is operational”**:

docker-compose logs -f sonarqube



Verify containers are running:

`docker ps`



**Expected:** Two containers — sonarqube and sonar-db — both with status Up.

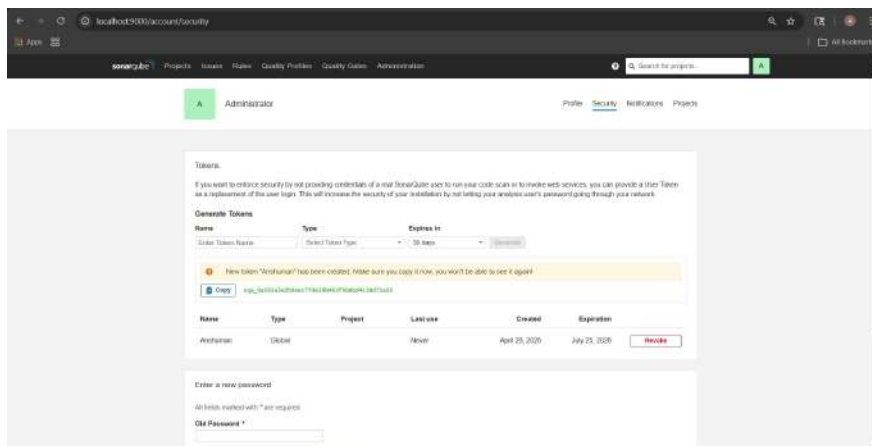
## 65.5 Step 2: Open SonarQube Dashboard

Open your browser and go to:

`http://localhost:9000`

- Default credentials: **admin / admin**
- You will be prompted to change the password on first login (set it to something like admin123)





```
PS C:\Users\anshu> $env:SONAR_TOKEN="sqa_9a592a3e2b9eec7fde38b463f9b06d4c38d71a19"
PS C:\Users\anshu>
```

## 65.6 Step 3: Generate an Authentication Token

The scanner needs a token to authenticate with the server.

1. Click your **user icon** (top right) → **My Account**
2. Click the **"Security"** tab
3. Under **"Generate Tokens"**, type a name: scanner-token
4. Click **"Generate"**
5. **Copy the token immediately** — it is shown only once!

```
PS C:\Users\anshu> $env:SONAR_TOKEN="sqa_9a592a3e2b9eec7fde38b463f9b06d4c38d71a19"
PS C:\Users\anshu> notepad pom.xml
PS C:\Users\anshu> notepad pom.xml
PS C:\Users\anshu> cd sample-java-app
```

Now export the token as an environment variable in your terminal (replace with your actual token):

```
export SONAR_TOKEN=sqp_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

## 65.7 Step 4: Create the Sample Java Application

The project files are already created under sample-java-app/. Here is what each file contains:

### 65.7.1 src/main/java/com/example/Calculator.java

A Java class intentionally containing:

- **Bug:** Division by zero (unhandled)
- **Code Smell:** Unused variable
- **Vulnerability:** SQL Injection risk

- **Code Smell:** Duplicated multiply() and multiplyAlt() methods
- **Bug:** Null pointer risk in getName()
- **Code Smell:** Empty catch block in riskyOperation()

## 65.7.2 pom.xml

Maven build file with the SonarQube Maven plugin (sonar-maven-plugin 3.9.1.2184) configured.

---

## 65.8 Step 5: Update the Token in pom.xml

Replace YOUR\_TOKEN\_HERE in pom.xml with your actual token:

```
cd ~/Desktop/exp10/sample-java-app
sed -i 's/YOUR_TOKEN_HERE/$SONAR_TOKEN/' pom.xml
```

```
PS C:\Users\anshu> docker network ls
NETWORK ID NAME DRIVER SCOPE
c72e0cccfd41 anahu_app-net bridge local
66b17726717 app-net bridge local
65f19090365 app-network bridge local
f21e08ea98a assignment-1_my-network bridge local
79b20816a81 bridge bridge local
c1b51c226c33 docker_gwbridge bridge local
b698e23fd4f host host local
22e81109a9a ingress overlay swarm
2a831fa058ba my-network bridge local
c1kayndyga2l my-overlay overlay swarm
x72897b3d4c1 my-subnet bridge local
b22897d9068f myapp-network bridge local
b6da7ad74c4d mynetwork macvlan local
35f6695f719c none null local
c81b0a2e3e1c sonar-lab_sonarqube-lab bridge local
fd17feb7c4f8 wp-net bridge local
PS C:\Users\anshu> docker run --rm '
>> --network sonarqube-lab '
>> -e SONAR_TOKEN=$env:SONAR_TOKEN '
>> -e $! /usr/bin/
>> sonarsource/sonar-scanner-cli '
>> -Dsonar.host.url=http://sonarqube:9080 '
>> -Dsonar.projectBaseDir=./src/xx
>> -Dsonar.projectKey=sample-java-app
Unable to find image 'sonarsource/sonar-scanner-cli:latest' locally
latest: Pulling from sonarsource/sonar-scanner-cli
4f4b780ef34: Pull complete
b226dad8b4c: Pull complete
69b078a3a7: Pull complete
07b33267ef4: Pull complete
f0916ef368d: Pull complete
95156e49a55: Pull complete
Digest: sha256:a429d2d22c8d97b9a10ea5585f1c5d42a54795f6ae3da86c7ala40ee78e6c2
Status: Downloaded newer image for sonarsource/sonar-scanner-cli:latest
docker: Error response from daemon: failed to set up container networking: Could not attach to network sonarqube-lab: rpc error: code = NotFound desc = netw
ork sonarqube-lab not found

Run 'docker run --help' for more information
PS C:\Users\anshu>
```

Verify the replacement:

```
grep "sonar.login" pom.xml
```

---

## 65.9 Step 6: Run the SonarQube Scan

### 65.9.1 Option A — Maven Plugin (Recommended if Maven is installed)

```
cd ~/Desktop/exp10/sample-java-app
mvn sonar:sonar -Dsonar.login=$SONAR_TOKEN
```

Maven will compile the code and send the analysis report to the SonarQube server.

### 65.9.2 Option B — Docker-based Scanner CLI (No Maven needed)

First, find the exact Docker network name:

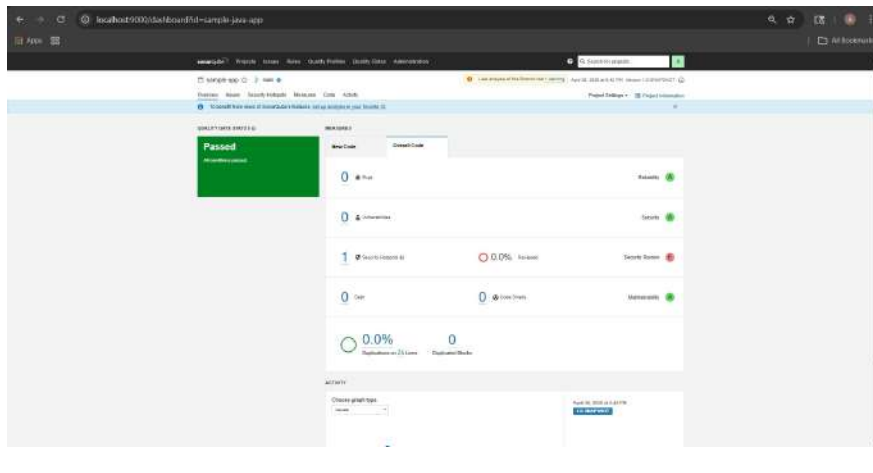
```
docker network ls | grep sonarqube
```

Then run the scanner container:

```
cd ~/Desktop/exp10/sample-java-app
```

```
docker run --rm \
 --network exp10_sonarqube-lab \
 -e SONAR_TOKEN="$SONAR_TOKEN" \
 -v "$(pwd):/usr/src" \
 sonarsource/sonar-scanner-cli \
 -Dsonar.host.url=http://sonarqube:9000 \
 -Dsonar.projectBaseDir=/usr/src \
 -Dsonar.projectKey=sample-java-app
```

**Note:** Use `http://sonarqube:9000` (container name), **not** `localhost`, because the scanner container is on the same Docker network as the server.



## 65.10 Step 7: View Results in the Dashboard

After the scan finishes, open:

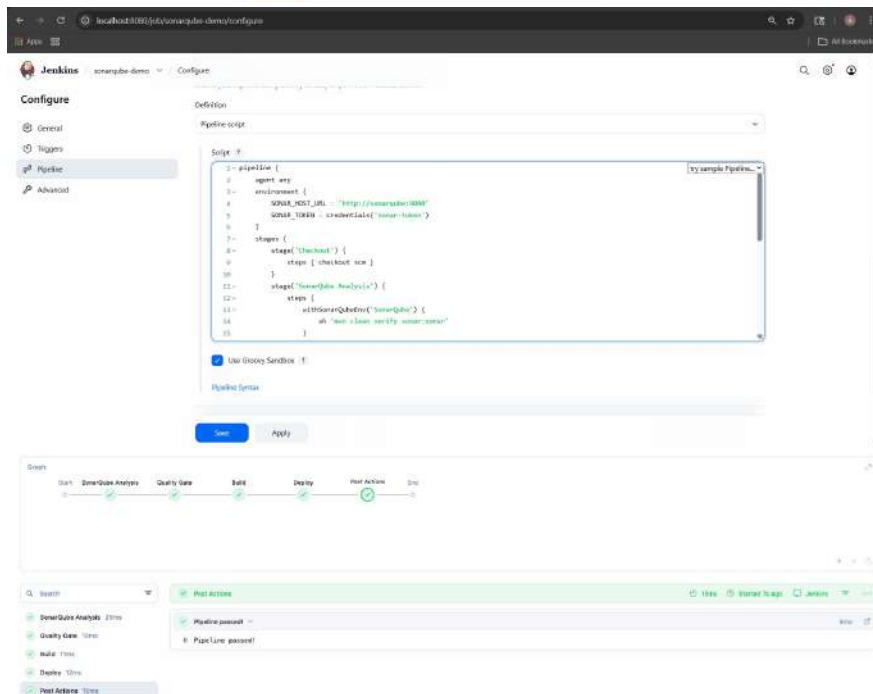
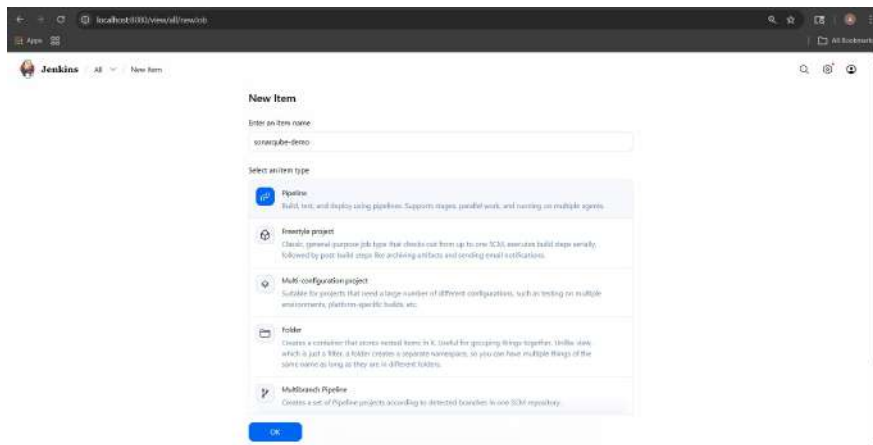
`http://localhost:9000/dashboard?id=sample-java-app`



```

 waitForQualityGate abortPipeline: true
 }
}
stage('Build') {
 steps { sh 'mvn package' }
}
stage('Deploy') {
 steps {
 sh 'docker build -t sample-app .'
 sh 'docker run -d -p 8080:8080 sample-app'
 }
}
}
}

```



 **Pipeline flow:**

## 65.12

### 65.13 Tool Comparison Matrix

| Feature          | Jenkins          | Ansible           | Chef              | SonarQube     |
|------------------|------------------|-------------------|-------------------|---------------|
| Primary Purpose  | CI/CD Automation | Config Management | Config Management | Code Quality  |
| Architecture     | Master-Agent     | Agentless         | Client-Server     | Client-Server |
| Language         | Java / Groovy    | YAML              | Ruby              | Java          |
| Learning Curve   | Moderate         | Low               | High              | Low           |
| Setup Complexity | Moderate         | Simple            | Complex           | Simple        |

### 65.14 Key Concepts Summary

- **SonarQube Server** = The brain — receives, stores, and displays analysis results
- **Sonar Scanner** = The worker — reads your code and sends the report to the server
- **Both are required.** The Scanner needs a Token to talk to the Server
- **Quality Gates** automatically block bad code from being deployed

### 65.15 Best Practices

- **Security:** Never hardcode tokens — use environment variables or a secrets manager
- **Code Quality:** Set Quality Gates to block merges when coverage drops below 80%
- **Scan on every pull request**, not just nightly builds
- **Fix issues as they appear** — do not let technical debt accumulate
- **Version all config files in Git**

## 66 Containerization-and-Devops-Lab

### 67 Experiment 11: Orchestration using Docker Compose & Docker Swarm

**Name:** Anshuman Mohapatra **Roll No:** R2142230609 **Course:** Containerization and DevOps

## 67.1 Objective

Understand container orchestration by moving from Docker Compose to Docker Swarm. Learn how to deploy a stack, scale services, and observe self-healing in a Docker Swarm cluster using WordPress and MySQL setup.

## 67.2 Prerequisites

- Docker installed with Swarm mode enabled
- The docker-compose.yml file from Experiment 6 (WordPress + MySQL)

## 67.3 PART B - PRACTICAL (EXTENSION OF EXPERIMENT 6)

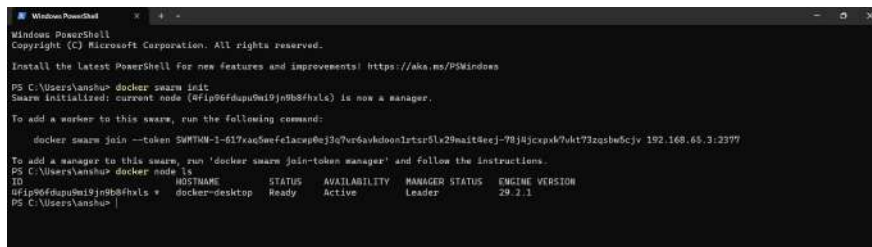
### 67.3.1 Task 1: Initialize Docker Swarm

Swarm mode turns your current machine into a manager node of a cluster.

```
docker swarm init
```

Verify that your node is ready as a Swarm manager:

```
docker node ls
```



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\anshu> docker swarm init
Swarm initialized: current node (4fip96fdpu0mi9jn9b8fhx1s) is now a manager.

To add a worker to this swarm, run the following command:

 docker swarm join --token SWMTKN-1-617xag5wefclacwpej3q7cr6avkdoonltsr5lx29nait4ecj-7b14jcxpsk7vht73zqswe5cjr 192.168.65.3:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

PS C:\Users\anshu> docker node ls
ID HOSTNAME STATUS AVAILABILITY MANAGER STATUS ENGINE VERSION
4fip96fdpu0mi9jn9b8fhx1s * docker-desktop Ready Active Leader 29.2.1
PS C:\Users\anshu>
```

### 67.3.2 Task 3: Deploy as a Stack (Not Just Compose)

In Swarm, we deploy a stack using the same Compose file. Swarm reads the file and creates services, which manage the containers automatically.

```
docker stack deploy -c docker-compose.yml wpstack
```

```

version: "3.8"

services:
 wordpress:
 image: wordpress:latest
 ports:
 - "8080"
 environment:
 WORDPRESS_DB_HOST: db
 WORDPRESS_DB_USER: root
 WORDPRESS_DB_PASSWORD: example
 WORDPRESS_DB_NAME: wordpress
 deploy:
 replicas: 1

 db:
 image: mysql:5.7
 environment:
 MYSQL_ROOT_PASSWORD: example
 MYSQL_DATABASE: wordpress
 deploy:
 replicas: 1

```

```

PS C:\Users\anshu> docker swarm init
Swarm initialized: current node (4fip96fdup9ai9jn9b8fhx1s) is now a manager

To add a worker to this swarm, run the following command:

 docker swarm join --token SWMTKN-1-617xaq5eeefelacp0e13q7vrd6xkdoenlrtr
5lv29maitdeej-783ujcpxk7vdt73qzsh6cjuv 192.168.65.1:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and fo
llow the instructions.
PS C:\Users\anshu> docker node ls
ID HOSTNAME STATUS AVAILABILITY MANA
GER STATUS ENGINE VERSION
4fip96fdup9ai9jn9b8fhx1s * docker-desktop Ready Active Load
er 19.2.1
PS C:\Users\anshu> notepad docker-compose.yml
PS C:\Users\anshu> docker stack deploy -c docker-compose.yml wpstack
Since --detach=false was not specified, tasks will be created in the backgro
und.
In a future release, --detach=false will become the default.
Creating network wpstack_default
Creating service wpstack_db
Creating service wpstack_wordpress
PS C:\Users\anshu>

```

### 67.3.3 Task 4: Verify the Deployment

List all services in the stack:

```
docker service ls
```

```

PS C:\Users\anshu> docker service ls
ID NAME MODE REPLICAS IMAGE
PORTS
h2qirxlsk4nt wpstack_db replicated 1/1 mysql:5.7
v4foez7006lv wpstack_wordpress replicated 1/1 wordpress:latest
*:8080->80/tcp
PS C:\Users\anshu>

```

See detailed tasks (containers) for a specific service:

```
docker service ps wpstack_wordpress
```

See all running containers (notice they are managed by Swarm):

```
docker ps
```

```

PS C:\Users\anshu> docker ps
CONTAINER ID IMAGE COMMAND CREATED
STATUS PORTS NAMES
e6072be91533 wordpress:latest "docker-entrypoint.s... About a mi
nute ago Up About a minute 80/tcp wpstack_wordpress.1.i9jmu6
2g36c1f1th2wu3g3bo1 mysql:5.7 "docker-entrypoint.s... About a mi
nute ago Up About a minute 3306/tcp, 33060/tcp wpstack_db.1.pojde3p5l2agm
60pg1h7f7la1 jenkins/jenkins:lts "/usr/bin/tini -- /u... 43 minutes
ago Up 43 minutes 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp,
0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp jenkins
5718d889c158 sonarqube:lts-commu " /opt/sonarqube/dock... 6 hours ag
o Up 6 hours 0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp sonarqube
c462b7350a1a postgres:13 "docker-entrypoint.s... 6 hours ag
o Up 6 hours 5432/tcp sonar-db
04d79e54b635 assignment-1-backend "docker-entrypoint.s... 5 weeks ag
o Up 6 hours 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp backend_app
20b1ecd594b9 assignment-1-database "docker-entrypoint.s... 5 weeks ag
o Up 6 hours postgres_db
PS C:\Users\anshu>

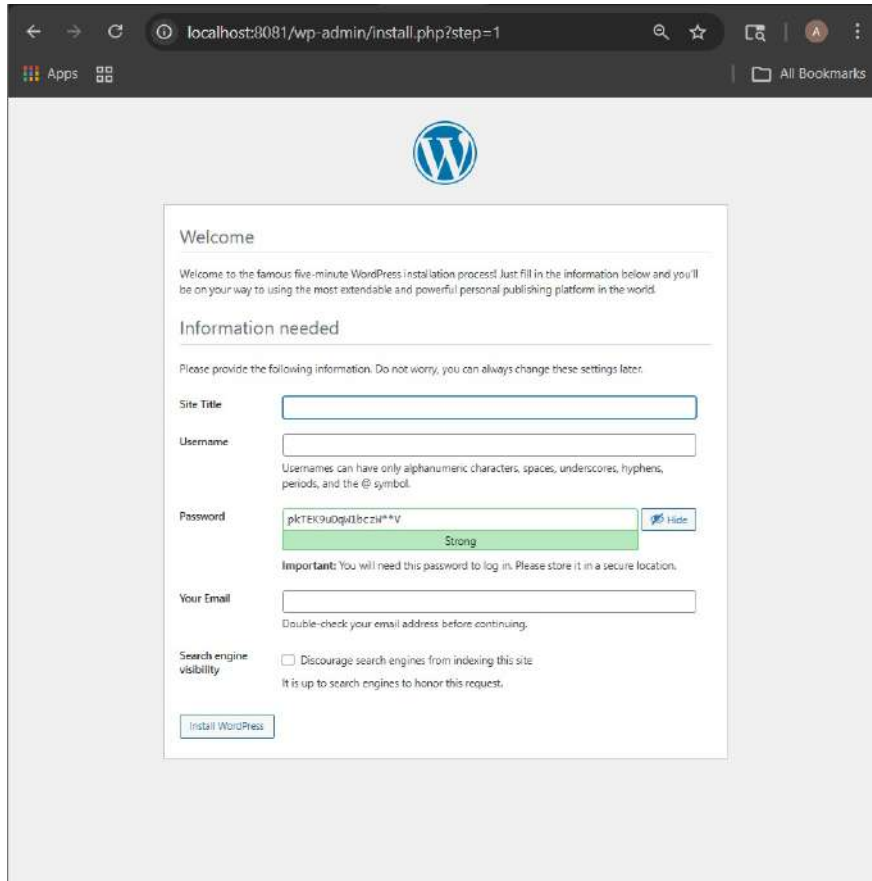
```



### 67.3.4 Task 5: Access WordPress

Open your browser and navigate to: `http://localhost:8081`

You should see the WordPress setup screen.



### 67.3.5 Task 6: Scale the Application (Swarm's Superpower)

Scale WordPress from 1 to 3 replicas using Swarm's orchestration.

```
docker service scale wpstack_wordpress=3
```

```
PS C:\Users\anshu> docker service scale wpstack_wordpress=3
wpstack_wordpress scaled to 3
overall progress: 3 out of 3 tasks
1/3: running
2/3: running
3/3: running
verify: Service wpstack_wordpress converged
PS C:\Users\anshu> |
```

Verify the scaling:

```
docker service ls
docker service ps wpstack_wordpress
docker ps | grep wordpress
```

```

PS C:\Users\anshu> docker service ls
ID NAME MODE REPLICAS IMAGE
PORTS
h2qirx1sk4nt wpstack_db replicated 1/1 mysql:5.7
v4foez7006lv wpstack_wordpress replicated 3/3 wordpress:latest
*:8081->80/tcp
PS C:\Users\anshu> docker service ps wpstack_wordpress
ID NAME IMAGE ERROR PORTS NODE
DESIRED STATE CURRENT STATE
8uf54ov70rms wpstack_wordpress.1 wordpress:latest docker-desktop
Running Running 9 minutes ago
i9jmu62g36c1 _ wpstack_wordpress.1 wordpress:latest docker-desktop
Shutdown Shutdown 9 minutes ago
ka7ew0pf3zm6 wpstack_wordpress.2 wordpress:latest docker-desktop
Running Running 7 minutes ago
6856tw4cx4xb wpstack_wordpress.3 wordpress:latest docker-desktop
Running Running 7 minutes ago
PS C:\Users\anshu> docker ps | Select-String wordpress
baceb90e421e wordpress:latest "docker-entrypoint.s0Ç*" 7
minutes ago Up 7 minutes 80/tcp

wpstack_wordpress.2.ka7ew0pf3zm6w8w4z394ftn8s
86eff685173b wordpress:latest "docker-entrypoint.s0Ç*" 7
minutes ago Up 7 minutes 80/tcp

wpstack_wordpress.3.6856tw4cx4xbk5eqem8gv0l4r
7a3d68ac03e9 wordpress:latest "docker-entrypoint.s0Ç*" 9
minutes ago Up 9 minutes 80/tcp

wpstack_wordpress.1.8uf54ov70rmshmf7lfg4xpqlly

PS C:\Users\anshu> |

```

*Note: Swarm automatically balances traffic among all 3 containers on port 8081 without port conflicts.*

### 67.3.6 Task 7: Test Self-Healing (Automatic Recovery)

Swarm automatically replaces failed containers. Let's test it:

**Step 1:** Find a WordPress container ID.

```
docker ps | grep wordpress
```

**Step 2:** Kill it to simulate a crash (replace <container-id>):

```
docker kill <container-id>
```

```

PS C:\Users\anshu> docker ps | Select-String wordpress
baceb90e421e wordpress:latest "docker-entrypoint.s0Ç*" 7
minutes ago Up 7 minutes 80/tcp

wpstack_wordpress.2.ka7ew0pf3zm6w8w4z394ftn8s
86eff685173b wordpress:latest "docker-entrypoint.s0Ç*" 7
minutes ago Up 7 minutes 80/tcp

wpstack_wordpress.3.6856tw4cx4xbk5eqem8gv0l4r
7a3d68ac03e9 wordpress:latest "docker-entrypoint.s0Ç*" 9
minutes ago Up 9 minutes 80/tcp

wpstack_wordpress.1.8uf54ov70rmshmf7lfg4xpqlly

PS C:\Users\anshu> docker kill 1b7c0beb22a3fd604d7315f3107ef8d998043513f2d7f
0f9b2b9a3ca3c207fce
1b7c0beb22a3fd604d7315f3107ef8d998043513f2d7f0f9b2b9a3ca3c207fce
PS C:\Users\anshu>

```

**Step 3:** Watch Swarm recreate it automatically:

```
docker service ps wpstack_wordpress
```

docker ps | grep wordpress

```
PS C:\Users\anshu> docker service ps wpstack_wordpress
ID NAME CURRENT STATE ERROR PORTS
8uf54ov70rms wpstack_wordpress.1 wordpress:latest docker-desktop
Running Running 14 minutes ago
i9jmu62g36c1 _ wpstack_wordpress.1 wordpress:latest docker-desktop
Shutdown Shutdown 14 minutes ago
ka7ew0pf3zm6 wpstack_wordpress.2 wordpress:latest docker-desktop
Running Running 13 minutes ago
6856tw4cx4xb wpstack_wordpress.3 wordpress:latest docker-desktop
Running Running 13 minutes ago
PS C:\Users\anshu>
```

Notice the killed container is shut down, and a new container is automatically created to maintain 3 replicas.

### 67.3.7 Task 8: Remove the Stack

Clean up the deployed stack and verify removal:

```
docker stack rm wpstack
docker service ls
docker ps
```

```
PS C:\Users\anshu> docker stack rm wpstack
Removing service wpstack_db
Removing service wpstack_wordpress
Removing network wpstack_default
PS C:\Users\anshu> docker service ls
ID NAME MODE REPLICAS IMAGE PORTS
PS C:\Users\anshu> docker ps
CONTAINER ID IMAGE COMMAND CREATED
STATUS PORTS NAMES
583c4fab77dc jenkins/jenkins:lts "/usr/bin/tini -- /u... About an h
our ago Up About an hour 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.
0.0.0:50000->50000/tcp, [::]:50000->50000/tcp jenkins
5718d889c158 sonarqube:lts-commu... "/opt/sonarqube/dock... 6 hours ag
o Up 6 hours 0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp sonarqube
c462b7350a1a postgres:13 "docker-entrypoint.s... 6 hours ag
o Up 6 hours 5432/tcp
04d79e54b635 assignment-1-backen... "docker-entrypoint.s... 5 weeks ag
o Up 7 hours 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp backend_app
20b1ecd594b9 assignment-1-databa... "docker-entrypoint.s... 5 weeks ag
o Up 7 hours postgres_db
```

## 68 Containerization-and-Devops-Lab

### 69 Experiment 12: Study and Analyse Container Orchestration using Kubernetes

**Name:** Anshuman Mohapatra **Roll No:** R2142230609 **Course:** Containerization and DevOps

---

## 69.1 Objective

Learn why Kubernetes is used, its basic concepts, and how to deploy, scale, and fix apps using Kubernetes commands.

## 69.2 Why Kubernetes over Docker Swarm?

| Reason               | Explanation                                 |
|----------------------|---------------------------------------------|
| Industry standard    | Most companies use Kubernetes               |
| Powerful scheduling  | Automatically decides where to run your app |
| Large ecosystem      | Many tools and plugins available            |
| Cloud-native support | Works on AWS, Google Cloud, Azure, etc.     |

## 69.3 Core Kubernetes Concepts (Simple Explanation)

| Docker Concept  | Kubernetes Equivalent | What it means                                                          |
|-----------------|-----------------------|------------------------------------------------------------------------|
| Container       | Pod                   | A pod is a group of one or more containers. Smallest unit in K8s.      |
| Compose service | Deployment            | Describes how your app should run (e.g., 2 copies, which image to use) |
| Load balancing  | Service               | Exposes your app to the outside world or other pods                    |
| Scaling         | ReplicaSet            | Ensures a certain number of pod copies are always running              |

## 69.4 Hands-On Lab (Using k3d or Minikube)

**Note:** We assume you already have kubectl and a cluster (k3d or Minikube) installed.

## 69.5 PART A - Core Tasks

### 69.5.1 Task 1: Create a Deployment

A deployment tells Kubernetes:

- Which container image to use (e.g., WordPress)
- How many copies (replicas) to run
- How to identify the pods (labels)

**Step 1:** Create the file wordpress-deployment.yaml

```
wordpress-deployment.yaml
apiVersion: apps/v1 # Which Kubernetes API to use
```

```

kind: Deployment # Type of resource
metadata:
 name: wordpress # Name of this deployment
spec:
 replicas: 2 # Run 2 identical pods
 selector:
 matchLabels:
 app: wordpress # Pods with this label belong to this
deployment
 template: # Template for the pods
 metadata:
 labels:
 app: wordpress # Label applied to each pod
 spec:
 containers:
 - name: wordpress
 image: wordpress:latest # Docker image
 ports:
 - containerPort: 80 # Port inside the container

```

```

PS C:\Users\anshu> k3d cluster create mycluster
[INFO][0800] Prep. Network
[INFO][0800] Created network 'k3d-mycluster'
[INFO][0800] Created image volume 'k3d-mycluster-images'
[INFO][0800] Starting new tools node...
[INFO][0800] Starting node 'k3d-mycluster-tools'
[INFO][0800] Creating node 'k3d-mycluster-server-0'
[INFO][0800] Creating loadBalancer 'k3d-mycluster-serverlb'
[INFO][0800] Using the k3d-tools node to gather environment information
[INFO][0800] Starting new tools node...
[INFO][0800] Starting node 'k3d-mycluster-tools'
[INFO][0800] Starting cluster 'mycluster'
[INFO][0800] Starting servers...
[INFO][0800] Starting node 'k3d-mycluster-server-0'
[INFO][0800] All agents already running
[INFO][0800] Starting node 'k3d-mycluster-serverlb'
[INFO][0800] Starting helpers...
[INFO][0800] Injecting records for hostAliases (incl. host.k3d.internal) and for 3 network members into CoreDNS configmap...
[INFO][0800] Cluster 'mycluster' created successfully!
[INFO][0800] You can now use it like this:
kubectl cluster-info

```

```

apiVersion: apps/v1
kind: Deployment
metadata:
 name: wordpress
spec:
 replicas: 2
 selector:
 matchLabels:
 app: wordpress
 template:
 metadata:
 labels:
 app: wordpress
 spec:
 containers:
 - name: wordpress
 image: wordpress:latest
 ports:
 - containerPort: 80

```

## Step 2: Apply the deployment

```
kubectl apply -f wordpress-deployment.yaml
```

**What happens?** Kubernetes creates 2 pods running WordPress.

```

PS C:\Users\anshu> kubectl get nodes
NAME STATUS ROLES AGE VERSION
k3d-my-lab-server-0 Ready control-plane,master 11m v1.31.5+k3s1
PS C:\Users\anshu> notepad wordpress-deployment.yaml
PS C:\Users\anshu> kubectl apply -f wordpress-deployment.yaml
deployment.apps/wordpress created
PS C:\Users\anshu> kubectl get pods
NAME READY STATUS RESTARTS AGE
wordpress-7d6f6db8d8-2l6v5 0/1 ContainerCreating 0 6s
wordpress-7d6f6db8d8-hvhdt 0/1 ContainerCreating 0 6s
PS C:\Users\anshu>

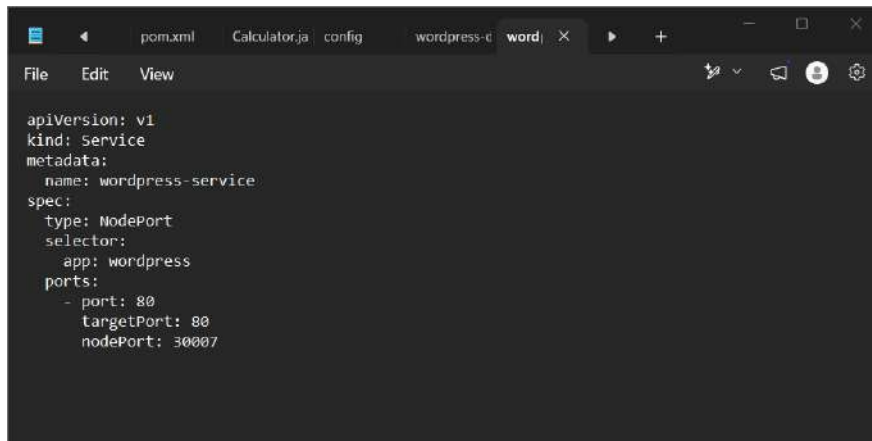
```

## 69.5.2 Task 2: Expose the Deployment as a Service

Pods are temporary (they can be deleted or recreated). A Service gives them a fixed IP and exposes them to the outside.

**Step 1:** Create the file `wordpress-service.yaml`

```
wordpress-service.yaml
apiVersion: v1
kind: Service
metadata:
 name: wordpress-service
spec:
 type: NodePort # Exposes service on a port of each node
 (VM)
 selector:
 app: wordpress # Send traffic to pods with this label
 ports:
 - port: 80 # Service port
 targetPort: 80 # Pod port
 nodePort: 30007 # External port (range: 30000-32767)
```



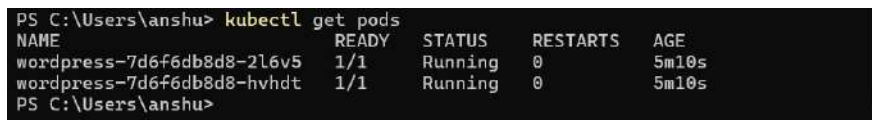
**Step 2:** Apply the service

```
kubectl apply -f wordpress-service.yaml
```

## 69.5.3 Task 3: Verify Everything

**Check if pods are running:**

```
kubectl get pods
```



**Check the service:**

```
kubectl get svc
```

```
PS C:\Users\anshu> kubectl get svc
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S)
kubernetes ClusterIP 10.43.0.1 <none> 443/TCP
wordpress-service NodePort 10.43.56.252 <none> 80:30007/TCP
```

### Access WordPress in your browser:

<http://localhost:30007>

- **Minikube:** Usually minikube ip
- **k3d:** Usually localhost

```
PS C:\Users\anshu> kubectl port-forward svc/wordpress-service 30007:80
Forwarding from 127.0.0.1:30007 -> 80
Forwarding from [::1]:30007 -> 80
Handling connection for 30007
Handling connection for 30007
Handling connection for 30007
Handling connection for 30007
Handling connection for 30007
Handling connection for 30007
```

Below you should enter your database connection details. If you are not sure about these, contact your host.

Database Name:   
The name of the database you want to use with WordPress.

Username:   
Your database username.

Password:  [Show](#)  
Your database password.

Database Host:   
You should be able to get this info from your web host, if localhost does not work.

Table Prefix:   
If you want to run multiple WordPress installations in a single database, change this.

[Submit](#)

## 69.5.4 Task 4: Scale the Deployment

Increase the number of pods from 2 to 4:

```
kubectl scale deployment wordpress --replicas=4
```



```
PS C:\Users\anshu> kubectl scale deployment wordpress --replicas=4
deployment.apps/wordpress scaled
PS C:\Users\anshu>
```

Verify:

`kubectl get pods`

```
PS C:\Users\anshu> kubectl get pods
NAME READY STATUS RESTARTS AGE
wordpress-7d6f6db8d8-2l6v5 1/1 Running 0 11m
wordpress-7d6f6db8d8-hvhdt 1/1 Running 0 11m
wordpress-7d6f6db8d8-ktdv9 1/1 Running 0 26s
wordpress-7d6f6db8d8-vfvw4 1/1 Running 0 26s
PS C:\Users\anshu>
```

You should now see 4 running pods.

**Why scale?** More traffic → more copies → better performance.

## 69.5.5 Task 5: Self-Healing Demonstration

Kubernetes automatically replaces failed pods.

**Step 1:** Get pod names

`kubectl get pods`

**Step 2:** Delete one pod (replace <pod-name>):

`kubectl delete pod <pod-name>`

**Step 3:** Check pods again:

`kubectl get pods`

You will still see 4 pods — the deleted one was **automatically recreated**.

**Why?** The deployment ensures the desired number (4) is always running.

```
PS C:\Users\anshu> kubectl get pods
NAME READY STATUS RESTARTS AGE
wordpress-7d6f6db8d8-2l6v5 1/1 Running 0 11m
wordpress-7d6f6db8d8-hvhdt 1/1 Running 0 11m
wordpress-7d6f6db8d8-ktdv9 1/1 Running 0 26s
wordpress-7d6f6db8d8-vfvw4 1/1 Running 0 26s
PS C:\Users\anshu> kubectl get pods
NAME READY STATUS RESTARTS AGE
wordpress-7d6f6db8d8-2l6v5 1/1 Running 0 12m
wordpress-7d6f6db8d8-hvhdt 1/1 Running 0 12m
wordpress-7d6f6db8d8-ktdv9 1/1 Running 0 91s
wordpress-7d6f6db8d8-vfvw4 1/1 Running 0 91s
PS C:\Users\anshu> kubectl delete pod wordpress-7d6f6db8d8-2l6v5
pod "wordpress-7d6f6db8d8-2l6v5" deleted
PS C:\Users\anshu> kubectl get pods
NAME READY STATUS RESTARTS AGE
wordpress-7d6f6db8d8-7vnpm 1/1 Running 0 17s
wordpress-7d6f6db8d8-hvhdt 1/1 Running 0 13m
wordpress-7d6f6db8d8-ktdv9 1/1 Running 0 2m29s
wordpress-7d6f6db8d8-vfvw4 1/1 Running 0 2m29s
PS C:\Users\anshu>
```

## 69.6 PART B - Cluster Information



```
kubectl get nodes
kubectl cluster-info
```

```
PS C:\Users\anshu> kubectl get nodes
NAME STATUS ROLES AGE VERSION
k3d-my-lab-server-0 Ready control-plane,master 28m v1.31.5+k3s1
PS C:\Users\anshu> kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:62113
CoreDNS is running at https://127.0.0.1:62113/api/v1/namespaces/kube-system/
services/kube-dns:dns/proxy
Metrics-server is running at https://127.0.0.1:62113/api/v1/namespaces/kube-
system/services/https:metrics-server:https/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info du
mp'.
PS C:\Users\anshu>
```

## 69.7 PART C - Swarm vs Kubernetes (Comparison)

| Feature      | Docker Swarm | Kubernetes                 |
|--------------|--------------|----------------------------|
| Setup        | Very easy    | More complex               |
| Scaling      | Basic        | Advanced (auto-scaling)    |
| Ecosystem    | Small        | Huge (monitoring, logging) |
| Industry use | Rare         | Standard                   |

Verdict: Learn Kubernetes — it’s what companies use.

## 69.8 PART D - Advanced Lab: Real Cluster with kubeadm

### 69.8.1 Lab Requirements

- 2 or 3 virtual machines (e.g., VirtualBox, VMware)
- Ubuntu 22.04 or 24.04
- Each VM: 2+ CPU, 2+ GB RAM

### 69.8.2 High-Level Steps

Step 1: Install kubeadm, kubelet, kubectl on all nodes

```
Update system
sudo apt update

Install required packages
sudo apt install -y apt-transport-https ca-certificates curl

Add Kubernetes signing key
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key | \
 sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

Add Kubernetes repository
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] \
 https://pkgs.k8s.io/core:/stable:/v1.29/deb/ /' | \
 sudo tee /etc/apt/sources.list.d/kubernetes.list
```

```
Install kubeadm, kubelet, kubectl
sudo apt update
sudo apt install -y kubeadm kubelet kubectl
```

```
Hold versions to prevent auto-update
sudo apt-mark hold kubeadm kubelet kubectl
```

### Step 2: Initialize the control plane (master node only)

```
sudo kubeadm init
```

This command:

- Sets up the control plane
- Generates a token for worker nodes to join
- Takes 2-3 minutes

### Step 3: Set up kubeconfig (to use kubectl)

```
Create .kube directory for your user
mkdir -p $HOME/.kube

Copy admin config
sudo cp /etc/kubernetes/admin.conf $HOME/.kube/config

Fix permissions
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Now you can run `kubectl get nodes` to see the master.

### Step 4: Install a network plugin (required for pods to communicate)

```
kubectl apply -f
https://docs.projectcalico.org/manifests/calico.yaml
```

Wait 1-2 minutes for Calico to start.

### Step 5: Join worker nodes

After `kubeadm init`, you'll see a join command like:

```
kubeadm join 192.168.1.100:6443 --token abcdef.0123456789abcdef \
--discovery-token-ca-cert-hash sha256:...
```

Run that exact command on each worker node.

### Step 6: Verify the cluster (on master)

```
kubectl get nodes
```

Expected output:

| NAME    | STATUS | ROLES         | AGE | VERSION |
|---------|--------|---------------|-----|---------|
| master  | Ready  | control-plane | 5m  | v1.29.0 |
| worker1 | Ready  | <none>        | 2m  | v1.29.0 |
| worker2 | Ready  | <none>        | 2m  | v1.29.0 |

---

## 69.9 Summary of Commands (Cheat Sheet)

---

| Goal               | Command                                                         |
|--------------------|-----------------------------------------------------------------|
| Apply a YAML file  | <code>kubectl apply -f file.yaml</code>                         |
| See all pods       | <code>kubectl get pods</code>                                   |
| See all services   | <code>kubectl get svc</code>                                    |
| Scale a deployment | <code>kubectl scale deployment &lt;name&gt; --replicas=N</code> |
| Delete a pod       | <code>kubectl delete pod &lt;pod-name&gt;</code>                |
| See all nodes      | <code>kubectl get nodes</code>                                  |
| Cluster info       | <code>kubectl cluster-info</code>                               |

## 69.10 Conclusion

In this experiment, we:

- Understood basic Kubernetes concepts (Pods, Deployments, Services, ReplicaSets)
- Deployed WordPress using a **Deployment** and **Service**
- **Scaled** the deployment from 2 to 4 replicas
- Demonstrated **self-healing** by deleting a pod and watching it auto-recover
- Compared **Docker Swarm** vs **Kubernetes**
- Learned how to build a real production-style cluster with kubeadm

**Next step:** Try deploying your own custom app (e.g., a simple Node.js or Python app).